

DESIGN PATTERNS

Thursday, December 8, 2022 1:10 PM



Course Overview

Monday, October 31, 2022 11:17 AM

Pluralsight: <https://app.pluralsight.com/library/courses/c-sharp-10-design-patterns/table-of-contents>

Every code base can be improved. In this course, you will learn what design patterns are. Many patterns are so common you have probably already used them.

Don't learn the pattern implementation from the top of your head, learn which problem a pattern solves.

You will learn how to vastly improve your code base by applying 23 different design patterns covering all sorts of challenges.

Patterns that Connect Senders and Receivers

- Chain of Responsibility:
Passes a request along a chain of receivers
- Command
Connects senders with receivers unidirectionally
- Mediator
- Observer

Each of the 23 patterns will be described in detail with a real-life example. The pattern structure will be covered as well so you can learn how to apply it to your code base.

Each pattern will be implemented in C# 10 on .NET 6. You will learn about use cases for the pattern and you will learn about pattern consequences and related patterns.

When you are finished with this course, you will have the skills and knowledge to use state-of-the-art design patterns in C# 10 to improve your code base.

Before beginning the course you should be familiar with C#.

Introduction to Design Patterns

Monday, October 31, 2022 1:41 PM

Coming up

Version check

It has been used

- .NET 6.0
- C# 10

Course Prerequisites and Frameworks

This course is a best practices course that cover a variety of design patterns. The focus is on the pattern itself, not on all the surrounding framework code. In other words, you should know C#.

Tooling and Setting up your Environment

As far as tooling is concerned, it will be used Visual Studio 2022.

Install Visual Studio at <https://visualstudio.microsoft.com/downloads/>

Course Structure

In this course, you will learn all about the Gang of Four design patterns, all 23 of them.

Each module covers one pattern, and each module also roughly adheres to the same structure.

- First, what the pattern in question is about, the pattern's intent, why it is used, and what it does. We will clarify this with a real-life example to make it more tangible to understand which problem the pattern solves.
- After that we will build a diagram so you will learn how the different components of the pattern work together.
- Next, Implement Real-life pattern, with use case used to describe the pattern in a demo.
- Depending on the pattern: you will learn variations, extensions, points of interest or alternatives.
- At the end of each module, we will cover pattern use cases, consequences and related patterns.

- With pattern use case, we are going to check out cases for which the pattern can help. "A pattern use case tells you for which cases the pattern might be a good match".
- With pattern consequences, it means the pros and cons of each pattern. "Pattern consequences can be positive and/or negative: consider implementing a pattern when the advantages outweigh the disadvantages for your use case".
- Don't just implement a design pattern because you can, implement it when the advantages outweigh the disadvantages.
- Patterns tend to solve related issues and often have comparable templates and implementations. "Patterns sometimes have comparable templates and implementations: learn how they compare, differ and can be combined".

Structure options:

- Follow the course from start to finish. (Recommended)
- Simply jump in wherever you want, each model can be regarded as a standalone module.

The patterns are grouped by type:

- Creational
- Structural
- Behavioral

See GitHub for solution: <https://github.com/KevinDockx/CSharp10DesignPatterns>

Introducing Design Patterns

A design pattern is a general, reusable solution to a commonly occurring problem within a given context in software design. You can look at it as a template you can use to solve common issues you will encounter during software design. They help with creating flexible, easily maintainable applications. In that regard, you can consider the best practice approaches to common problems.

It is very important to remember that you should see these patterns as a template to start from and not as a rule set in stone. Implementing a pattern can typically be done in multiple ways, depending on, for example, the language you are implementing them in. A good example is that you will notice a lot of patterns talk about abstract classes; however, in C# you can often use an interface instead if all you need is a contract and not an implementation to start from.

Throughout the course you will learn how to implement these patterns in various ways. That being said, each pattern has an intent, for example, the adapter pattern, which we will encounter, intends to let classes work together that otherwise cannot. No matter how you implement this pattern, the intent should remain the same, and that is true for all patterns.

A lot of these patterns are so common that you have undoubtedly used them before, maybe even without knowing that you were using them. Important is that you shouldn't learn the pattern implementation from the top of your head. Instead, learn which problem a pattern solves. The patterns we will cover in this course are the well-known Gang of Four design patterns. The gang of four are, Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. All the way back to 1994, these guys published a book that, to this day, is the de facto standard work on software design patterns, Design Patterns - Elements of Reusable Object-Oriented Software. It contains 23 patterns. Many is built-in and we do not notice them.

They also have common issues you might encounter when designing, in this case, object-oriented software.

A few examples of problems that are solved by them are:

- How do I ensure that only a single instance of a class exists.
- How do I make two objects with a different interface work together.
- How can I extend an object's interface without changing the underlying object.
- How do I enable support for undo functionality.
- And many more....

We will use modern-day language features to implement these patterns. Using current, real-life examples you might encounter in your day-to-day life as a developer. The intent will of course remain the same. As far as some of the use cases, consequences, and related patterns are concerned, not everything from the book is current today and some consequences or use cases that are common today weren't at the time the book was written.

Gang of Four Pattern Types

The gang of four patterns types contains three different pattern types

- Creational - It consist of patterns which all deal with object creation. The idea behind these patterns is to abstract the object instantiation process. These patterns will help making your system independent of how its objects are created, composed, and represented. If you notice, you have got a lot of new statements in your client code and tightly coupled classes. Using concrete implementations, these patterns will help with that.

Five patterns:

- o Abstract Factory
- o Builder
- o Factory method
- o Prototype

- Singleton
- Structural - These patterns deal with ways to define relation between classes or objects. In other words, they are concerned with how classes and objects are composed to form larger structures, hence the name.
Consists of seven patterns:
 - Adapter
 - Bridge
 - Composite
 - Decorator
 - Facade
 - Flyweight
 - Proxy
- Behavioral - These types of patterns define ways to communicate between classes or objects. They characterize complex control flow that is difficult to follow at runtime and they let you concentrate on the way objects are interconnected.
Consist of eleven patterns:
 - Chain of Responsibility
 - Command
 - Interpreter
 - Iterator
 - Mediator
 - Memento
 - Observer
 - State
 - Strategy
 - Template method
 - Visitor

Object-Oriented Principle Refresher

There is two important object-oriented principles that are followed by the Gang of Four in their design patterns.

- The first one is programmed to an interface and not an implementation.
- The second one is favor object composition over class inheritance.

Program to an interface, not an implementation

This has two main benefits:

- Clients remain unaware of the specific types of objects they use (as long as the objects adhere to the interface that the clients expect)
- Clients remain unaware of the classes that implement these objects. Clients only know about the interface.

This allows loose coupling.

When thinking of program to an interfaces and not an implementation in 2022, we will think about interface language features. However, what the Gang of Four is talking about is not a technical interface language feature we have defined with the interface keyword. They are talking about the set of requests that an object can respond to, which is the object's type. Now of course it is a close relation to the interface language features we know about as it can be implemented as such, but you can also implement the interface they are talking about with an abstract class.

It is important to remember that distinction. As mentioned before, these patterns can be implemented in various ways. Using the language feature interface versus the language feature abstract class is one of

the main ways that leads to different implementations. Most pattern that talk about the object interface or an interface can in C# be implemented with abstract classes or interfaces. To decide on which to use, a good thumb rule is use an abstract class to create functionality that subclasses can implement or override, and use interface when all you need to do is define functionality and not implement it. And whereas a class can extend only one abstract class, it can take advantage of multiple interfaces.

"Use an abstract base class when you need to provide some basic functionality that can potentially be overridden".

"Use an interface when you only need to specify the expected functionality of a class".

When a pattern starts working with interface as defined by this principle, you will notice that it will often start adhering to the open/closed principle from the SOLID coding principles. That is the principle that states that software entities, like classes, modules, and functions, should be open for extension but closed for modification.

Favor object composition over class inheritance

Both class inheritance and object composition are techniques for reusing functionality in object-oriented systems.

Class inheritance:

Class inheritance lets you define the implementation of one class in terms of another. You simply create a subclass of a base or parent class, both terms are used interchangeably, and you do that by inheriting the parent class. Like that, you are reusing functionality of the parent class in your subclass or derived class, however you prefer to call it. This is often referred to as white-box reuse meaning that the internals of parent class are often visible to subclasses.

Object composition:

And then there is object composition, here, new functionality is obtained by assembling or composing objects to get more complex functionality. This requires that the objects being composed have well-defined interfaces. It is referred to as black-box reuse because internal details of objects are not visible to the composing class.

Both have their advantages and disadvantages and the debate can get quite heated. In most systems, both reuse techniques are commonly used. What is important for this course is that these patterns will, wherever possible, favor object composition over class inheritance, as the Gang of Four felt that inheritance as a reuse technique is overused. And when you see some systems that regularly define classes that inherit from each other six or seven levels deep. It can be agreeable, that it is not a great design.

Designs are often made more reusable and simpler by relying on object composition, and that is a good thing because unnecessary complexity is something you should definitely avoid.

When a pattern starts working with composition as is defined by this principle, you will notice that we will often start adhering to the single responsibility principle (SOLID). That is the principle that states that a class should have one and only one reason to change.

Creational Pattern: Singleton

Monday, October 31, 2022 1:42 PM

Coming Up

In this module the Singleton pattern is covered, a very common and pretty simple one to start with.

First we will describe the pattern. We will do that with the help of real-life example, a very typical use case for a singleton, a logger. After that, we will describe the pattern structure and implement it in the demo. Then we will look into a variation to make the pattern thread-safe.

We will continue with learning about use cases for this pattern and pattern consequences, which can be positive or negative. And we will end the module by looking into related patterns.

Describing the Singleton Pattern

The singleton pattern is a creational pattern from the well-known Gang of Four design patterns. Creational patterns deal with object creation. The intent of the singleton pattern is to ensure that a class only has one instance, and to provide a global point of access to it.

If you have worked with an inversion of control container before, you have undoubtedly encountered it as registering a type with a singleton lifetime is functionality offered by almost all of these containers.

A very common example is a logger. You typically only want one logger instance for your whole application to avoid multiple instances interfering with each other, which could result in messages being logged multiple times or not at all.

Let's build that on a diagram; Now, we could simply hold the instance of the class in a global variable, but there is an issue with this approach. It does not prevent clients from creating other instances of the class. That is a responsibility that is too widely distributed to be reliable. Therefore, through the singleton pattern, we can make the class itself responsible for ensuring that only one instance of the class exists.

"Holding the class instance in a global variable doesn't prevent clients from creating other instances of the class".

"Make the class responsible for ensuring only one instance of itself exists".

Another option would be relaying the responsibility to another component, like an IOC container, as mentioned.

To make our logger class responsible for managing its only instance, we should make its constructor private. Like that, other parts of the code cannot directly instantiate our logger. If you want to allow subclassing the logger, a protected constructor is also acceptable.

Next to that, the Logger class will contain a method or property in a modern language, that will return the instance of itself, for example, an Instance property with a private backing field. This property is static, which ensures that when multiple instances of the Logger class exist, there still will be only one instance of the Instance property. When the property getter is accessed the first time, the instance gets created and stored. When it is accessed after that, the previously created instance is returned, thus ensuring that only one instance exists at any given time.

"Prefer lazy instantiation Create and store the instance when it is requested for the first time, and return that instance on subsequent requests"

It is a good idea to prefer lazy instantiation in this case. When the property getter is accessed the first time, the instance gets created and stored. When it is accessed after that, the previously created instance is returned, thus ensuring that only one instance exists at any given time. The instance isn't created before the instance property is accessed.

Next to that, a Logger class will typically contain methods like Log. These are accessed via the Instance property. And with that we have got our diagram.

```
Logger
Logger() (private/protected)
static Logger? Instance ----- return _instance
Log()
static Logger? _instance
```

Singleton Pattern Structure

In this part of the module, we are going to map the real-life example we ended up with to the pattern structure. The idea is that this pattern structure can be applied to other real-life scenarios as well. The reason for doing so is that from this structure, you will be able to apply the pattern to other real-life situations. Mapping our logger example to the singleton pattern structure is pretty straightforward.

You are looking at the diagram we ended up with. The Logger class itself is called the singleton. A singleton defines an instance operation that lets clients access its unique instance. On it, a public static property instance exists. A private backing field contains the stored instance value. The log method maps to what is called a singleton operation in the pattern structure. These are operations on the singleton instance.

```
Singleton
Singleton() (private/protected)
static Singleton? Instance ----- return _instance
SingletonOperation()
static Singleton? _instance
```

Demo - Implementing the Singleton Pattern

In this demo, we are going to implement the Singleton pattern. You can find the project files for all patterns in the starter solution. All of these contains a Program.cs file in which we will write the code to test the pattern and an implementation.cs file in which we will write the pattern code itself.

For the Singleton pattern, we want to start with a Logger class. To this class, we add a protected constructor so clients cannot instantiate it, but it can be subclassed. The next thing we want to add is the static Instance property and a private backing field for that property. This must be nullable. In the property getter, we can then check whether it is still null. If it is, we instantiate and store a Logger instance. And afterwards, we always return that instance. That takes care of ensuring that only one instance of the Singleton class, our Logger in this case, exists at any given time. We also add one method, Lg, for testing purposes. Every instance of the Logger that is returned when getting the instance property has access to it, and it is always exactly the same instance that is being returned.

```
In Implementation.cs
public class Logger
{
    protected Logger(){}

    private static Logger? _instance;
```



```

    public static Logger Instance
    {
        get
        {
            if (_instance == null)
            {
                _instance = new Logger();
            }
            return _instance;
        }
    }

    public void Log(string message)
    {
        Console.WriteLine($"Message to log: {message}");
    }
}

```

Let's write some code to test this. First, let's see if we can new this up. With: `var test = new Logger();` And apparently we can't. The constructor is inaccessible due to its protection level. What we can do, however, is get an instance via `Logger.Instance`. So, let's get two of those and compare them. If these two are the same instance and they match the `Logger.Instance` property, we write that out to the console output window, and then we log a few messages as an example. All the instances is the same instance and will be written out from the `instance1`, `instance2` and `Logger.Instance` variable resulted in exactly the same. So it seems to work, but there is a potential issue with this.

In Program.cs

```

// var test = new Logger(); // Add using statement

//call the property getter twice
var instance1 = Logger.Instance;
var instance2 = Logger.Instance;

if (instance1 == instance2 && instance2 == Logger.Instance)
{
    Console.WriteLine("Instances are the same");
}
instance1.Log($"Message from {nameof(instance1)}");
instance1.Log($"Message from {nameof(instance2)}");
Logger.Instance.Log($"Message from {nameof(Logger.Instance)}");
Console.ReadLine();

```

Demo - Making the Implementation thread-safe with Lazy<T>

We are looking at the implementation of the Singleton class. What we have done here is a naive form of lazy initialization. Lazy initialization is the principle that states that we are only going to create an instance of a class once we need it and not when it is constructed. The problem is that this code is not guaranteed to be thread-safe.

We could go and create our own logging mechanism, but there is no need for that anymore. .NET includes a way to deal with lazy initialization in a thread-safe manner using Lazy of T (Lazy<T>). When we use that, the compiler takes care of making this thread-safe, so let's do that (see blue code). We add a new static field to replace our static private `Logger` instance. This is of type Lazy of T, in this case Lazy of

Logger. And we state that it is read only because we won't need to change this object during the singleton lifetime.

We immediately initialize it, but we initially sites it as a Lazy of Logger, and that does not mean that the Logger instance itself will be constructed. Instead, we pass through a method that will be used for constructing the Logger, when we first access the lazyLogger's value property. We remove the blue `//` code in the get and instead implements a return of the value of our lazyLogger.

The first time this is accessed, the Logger will be constructed using the action we passed through when initializing the lazyLogger. Every time after that, the same instance will be returned, and all of this happens in a thread-safe manner. There is no need to change our client code. Normally, when we try this, we should get the exact same output as in the previous demo.

In Implementation.cs

```
public class Logger
{
    protected Logger(){}

    // Lazy<T>
    private static readonly Lazy<Logger> _lazyLogger = new Lazy<Logger>(() => new Logger());

    // private static Logger? _instance;

    public static Logger Instance
    {
        get
        {
            // if (_instance == null)
            // {
            //     _instance = new Logger();
            // }
            // return _instance;

            return _lazyLogger.Value;
        }
    }

    public void Log(string message)
    {
        Console.WriteLine($"Message to log: {message}");
    }
}
```

Use Cases for Singleton Pattern

You can use this pattern:

- When there must be exactly one instance of a class, and it must be accessible to clients from a well-known access point. Our logger is a prime example of that.
- It is also useful when the sole instance should be extensible by subclassing, and all clients should be able to use an extended instance without modifying their code. That is what we enabled by using a protected constructor.

Pattern Consequences

There is quite a few consequences to this pattern.

- Because the Singleton class encapsulates its sole instance, it has strict control over how and when clients access it. (This is an advantage)
- It is also better than global variables because it avoids polluting the namespace with global variables used for storing sole instances.
- As the class can be subclassed, which enabled by the protected constructor, you can configure the application with an instance of the class you need at runtime.
- Also, in case you change your mind and you want to allow multiple instances, you can achieve that without having to alter the client because instance management is now contained within the singleton itself.
- But there is also a disadvantage. The singleton pattern violates the single responsibility principle because objects not only control how they are created. They also manage their of lifecycle.

In modern languages you don't often write code like this anymore as all of this is mostly dealt with via an IOC container. In that case, it is no longer the responsibility of the singleton class itself to manage its own lifetime, but the IOC container takes on that responsibility.

Related Patterns

Many patterns are related to the singleton pattern, if only for the reason that they can be implemented as singletons. We will encounter three creational patters like that:

- The Abstract Factory pattern, that can be implemented as a singleton.
- The Builder pattern, that can be implemented as a singleton.
- The Prototype pattern, that can be implemented as a singleton.
- The State pattern, also relates as state objects are often implemented as singletons.

Creational Pattern: Factory Method

Monday, October 31, 2022 1:42 PM

Coming up

The idea is that this pattern let's a class differ instantiation to one or more subclasses.

We start by describing the pattern. As a real-life example, we will use a shopping cart checkout context. We will create a factory with the factory method to provide us with the correct discount service implementations because discounts might be different per country, per code and so on. We will use that example to learn about the pattern structure, and we will implement it. After that we will continue with learning about use cases for this pattern and pattern consequences. Lastly, we look at related patterns.

Describing the Factory Method

The factory method pattern is a creational pattern from the well-known Gang of Four design patterns. Creational patterns deal with object creation, and using a factory method is a very common way of doing that. It is also known as a virtual constructor. These days, we don't really talk about the factory method pattern a lot anymore. We simply talk about the factory pattern. But if we want to be correct, the factory pattern can refer to either the factory method, or the abstract factory pattern. The abstract factory pattern will be in the next module, after we done that we will compare both.

The factory method, its intent is to define an interface for creating an object, but to let subclasses decide which class to instantiate. Factory method let's a class defer instantiation to subclasses.

With the example we will clarify this:

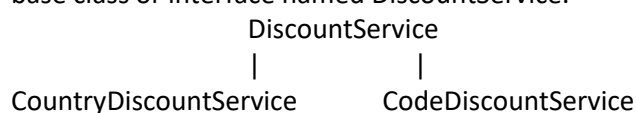
Imagine that you are creating part of a shopping cart system. At a certain moment, a discount can be applied. One of the services that can provide the discount is a service that provides a discount depending on the country the user is in. Another one provides a discount depending on a discount code. So both services are not constructed in the same way, nor is the logic to calculate the actual discount the same. Yet, they do have something in common, the discount percentage that needs to be exposed. We could write some code that create new instances of these classes up and gets the discount percentage. As shown below.

```
var codeDiscountService = new CodeDiscountService(GUID.NewGuid());  
var discount = codeDiscountService.DiscountPercentage;
```

//or

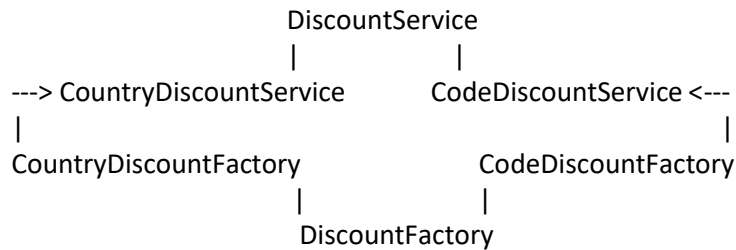
```
var countryDiscountService = new CountryDiscountService("BE");  
var discount = countryDiscountService.DiscountPercentage;
```

Yet this creates tight coupling between the client class, which is instantiating them, and the different discount service implementations. The factory method pattern helps avoid that. We do that by adding a base class or interface named DiscountService.



It is this class that any part of the application that needs to know the discount can be interact with. The concrete implementation isn't important for the client. Now the client, which is part of the shopping cart system that needs to know the discount, does not need to know which actual subclass it needs to instantiate. It doesn't care whether it is the code discount or the country discount service. All it needs to

know is when it needs such an instance. But how do we instantiate it then? For that, we create a base discount factory with one abstract method, `CreateDiscountService`. This is called the factory method. Concrete implementation of this factory are then a `CodeDiscountFactory` and a `CountryDiscountFactory`.



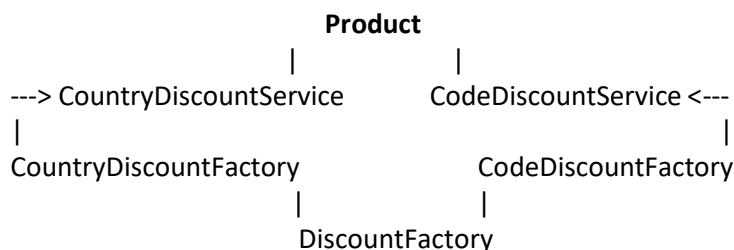
The client works on the `DiscountFactory` base class, so it uses whichever implementation is provided to get a hold of the concrete implementation of the discount service via the concrete implementation of the `DiscountFactory` with the concrete implementation for the `DiscountFactory` method.

Via the Factory method, we encapsulated the logic to create the actual subclass, so the client doesn't need to know about that anymore. And because we are working on an interface instead of a concrete implementation, we prevent tight coupling. Let's map this to the pattern structure.

Factory Method Pattern Structure

When we map this to the pattern structure, we start with the product. That could be an abstract base class or interface for the thing that needs to be created.

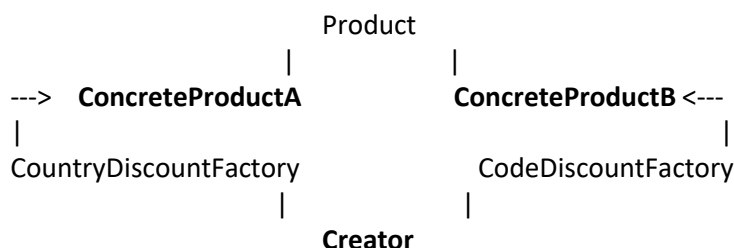
"Product defines the interface of objects that the factory method creates".



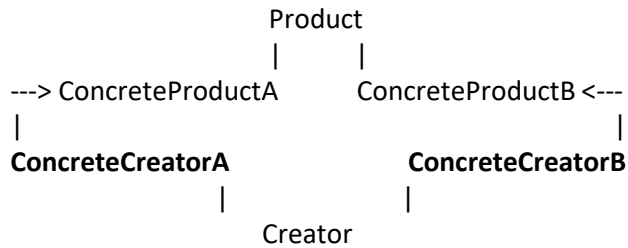
In our case, that's the `DiscountService` class.

Then we have concrete products, the implementations of that product interface. If we map that to our example, that is the `CodeDiscountService` and `CountryDiscountService`. "ConcreteProduct implements the Product interface".

The creator then declares the factory method. This factory method must return an object of type product. In our case that is the `DiscountFactory` with the abstract `CreateDiscountService` method on it. We can map that to factory method. The creator itself may also call the factory method to create a product object.



And lastly, we have the concrete creator. These are implementations of creator, which override the factory method to return an instance of `ConcreteProduct`. Mapped to our example, these are the `CodeDiscountFactory` and `CountryDiscountFactory`.



And with that, we have got our pattern structure.

Demo - Implement the Factory Method Pattern

Time to implement the Factory Method Pattern. Let's start with our product, the DiscountService.

This is an abstract class as we want it to serve as an interface for a concrete implementation. On it, we can add properties or methods like the DiscountPercentage that can be used by clients. This too is marked with the abstract keyword as we want the concrete products to implement the logic behind this. Note that as with most pattern implementations that require an abstract base class, you can use an interface as an alternative. As an example, it will be used as an interface in the next module where we will look into the Abstract Factory pattern, a pattern closely related to this Factory Method pattern. We also override the ToString method so it returns the actual type name. That will make it easier to see what is going on when we test this. Then let's implement this.

In Implementation.cs

```

public abstract class DiscountService
{
    public abstract int DiscountPercentage { get; }

    public override string ToString() => GetType().Name;
}
  
```

Below the first class, we add a CountryDiscountService, which implements our DiscountService abstract class. This is a concrete product. Depending on the passed-in countryIdentifier, a different percentage must be returned. So, we store the identifier in a readonly field. We do not want it changed after the object has been constructed. Then, we override the DiscountPercentage as it is up to the concrete implementation to decide on this. Let's say that, for this demo, when you are from Belgium, you get a better discount.

```

public class CountryDiscountService : DiscountService
{
    private readonly string _countryIdentifier;

    public CountryDiscountService(string countryIdentifier)
    {
        _countryIdentifier = countryIdentifier;
    }

    public override int DiscountPercentage
    {
        get
        {
            switch (_countryIdentifier)
            {
                // if you are from Belgium, you get a better discount :)
            }
        }
    }
}
  
```

```

        case "BE":
            return 20;
        default:
            return 10;
    }
}
}
}

```

Our next implementation then is a CodeDiscountService. We start in a likewise manner. The DiscountService is implemented, and we override the DiscountPercentage property. Let's say that each code result in the same discount percentage, but a code can only be used once. In real-life, this is where you could add that check. We are not going to implement that here because what is important for us is that we ended up with two different concrete products.

Insert below the second class:

```

public class CodeDiscountService : DiscountService
{
    private readonly Guid _code;

    public CodeDiscountService(Guid code)
    {
        _code = code;
    }

    public override int DiscountPercentage
    {
        //Each code returns the same fixed percentage, but a code is only valid once
        //Include a check to so whether the code's been used before ....
        get => 15;
    }
}

```

Up next is the factories. As we learned, we start with an abstract DiscountFactory class with one abstract method, CreateDiscountService, which returns the DiscountService, the base class for our concrete discount services.

Insert below the third class:

```

public abstract DiscountFactory
{
    public abstract DiscountService CreateDiscountService();
}

```

And then we need to implement the concrete implementations. ContryDiscountFactory news up a ContryDiscountService for which it needs to countryIdentifier. So it is this factory that is responsible for creating a CountryDiscountService.

Insert below the fourth class:

```

public class ContryDiscountFactory : DiscountFactory
{
    private readonly string _countryIdentifier;

    public CountryDiscountFactory(string countryIdentifier)
    {
        _countryIdentifier = countryIdentifier;
    }
}

```

```

    {
        _countryIdentifier = countryIdentifier;
    }

    public override DiscountService CreateDiscountService()
    {
        return new CountryDiscountService(_countryIdentifier);
    }
}

```

And in a likewise manner, our CodeDiscountFactory is responsible for creating and managing the lifetime of the CodeDiscountService, passing through a code it gets from the CodeDiscountFactory constructor.

Insert below the fifth class:

```

public class CodeDiscountFactory : DiscountFactory
{
    private readonly Guid _code;

    public CountryDiscountFactory(Guid code)
    {
        _code = code;
    }

    public override DiscountService CreateDiscountService()
    {
        return new CodeDiscountService(_code);
    }
}

```

And with that, we have all the moving parts of our pattern in place. we can test this. Let's switch to Program.cs file. And first, let's create the two factories. For that, we need to import the Factory Method namespace. We store these in a list, which makes it easy to run through them to see what is going on. It is through these factories that the actual services can be created.

In Program.cs
using FactoryMethod;

```

var factories = new List<DiscountFactory> //Quick action to add using statement
{
    new CodeDiscountFactory(Guid.NewGuid()),
    new CountryDiscountFactory("BE")
}

```

Below the newly added code. We run through our list of factories, and on each factory, we call CreateDiscountService. That means that this client is doing the call on the DiscountFactory base class, and then we write out the percentage and where it is coming from. Like that, we can see what was actually created. Important to remember here is that the client does not know about any concrete implementations. The factories it works with are abstract and so are the discount services.

```

foreach (var factory in factories)
{
    var discountService = factory.CreateDiscountService();
}

```



```
        Console.WriteLine($"Percentage {discountService.DiscountPercentage} " + $"coming from  
        {discountService}");  
    }  
  
    Console.ReadKey();
```

When testing these codes, we get two concrete products that were created by our two concrete factories without the client having to know about the concrete implementations.

Use Case for the Factory Method

Good use cases for this pattern are when a class can't anticipate the class of objects it must create, in other words when you don't know in advance which types of object your code should work with. When that is the case, the concrete creators can take on this responsibility.

Another good use case is when a class wants its subclasses to specify the objects it creates. This enables the scenario in which standard components or classes can be extended via subclasses. Here too, this means the concrete creator takes on the responsibility for creating the objects.

Up next, when classes delegate responsibility for creation to one of several helper subclasses and you want to localize the knowledge of which helper subclass is the delegate. Localization of that knowledge is now at level of whichever part of your application is responsible for providing the concrete implementation of the creator that is needed.

And you can also use it as a way to enable reusing of existing objects.

The logic inside of the factory method can take care of either creating an object or reusing an existing one as it sees fit. If you are renewing the instances yourself, you will either have to write code like that wherever you renew it or put it in a reusable method. And that reusable method, that is the factory method.

Pattern Consequences

The main consequence of this pattern is that factory methods eliminate the need to bind application-specific classes to your code, to the client for example. And that is because the code only deals with the interface, the product, and not with the concrete implementation.

That means it can work with any concrete product implementation. In other words, you are avoiding tight coupling. It is this that enables adding new types of products without breaking client code. And that is the open/closed principle at work. The single responsibility principle is also adhered to.

Creating the products is moved to one specific place in your code, the creator, that is only responsible for that.

A potential disadvantage is that clients might need to create subclasses of the creator class just to create a particular ConcreteProduct object. If only one subclass, so one concrete creator, is expected to be needed, you would have just added unnecessary complexity.

Related Patterns

Closely related to this pattern is the Abstract Factory pattern. It is often implemented with factory methods as we will see very soon in the module about Abstract Factory.

There is also a relation to another creational pattern, the prototype pattern. Prototypes don't require subclassing creators. They are not based on inheritance, which is a plus when you favor composition over inheritance, which the Gang of Four does. However, prototype pattern often require an initialize operation on the Product class, which the factory method doesn't. So, factory method is based on inheritance, but it doesn't require the initialization step.

Lastly, there is also a relation to a behavioral pattern. Factory methods are usually called within template methods.

- Abstract Factory
Often implemented with factory methods.
- Prototype
No subclassing is needed (not based on inheritance), but an initialize action on Product is often required.
- Template
Factory methods are often called from within template methods.

Creational Pattern: Abstract Factory

Monday, October 31, 2022 1:42 PM

Coming up

The intent of the abstract factory pattern is to provide an interface for creating families of related or dependent objects without specifying their concrete classes. In this module you will learn what that means.

We will start by describing the pattern, when to use it, and why. After we described the pattern, we will look at the pattern structure. As a real-life example, we will continue with our shopping cart example from the Factory method module. We will also compare the abstract factory pattern to the factory method pattern. After that we will continue to learn about use cases for abstract factory pattern and its consequences. We will end the module by looking into related patterns.

Describing the Abstract Factory

The abstract factory is a creational pattern from the well-known Gang of Four design patterns. It is closely related to the factory method pattern which we already covered. It is often implemented by using factory methods.

The abstract factory pattern is sometimes also referred to as the factory pattern, and so is the factory method better.

The intent of the abstract factory pattern is to provide an interface for creating families of related or dependent objects without specifying their concrete classes. When we looked into the factory method pattern, we used a discount service example to clarify it. We can extend that example to get a good use case for the abstract factory pattern.

In the example for factory method, we stated that everyone from Belgium got a 20% discount and everyone else got a 10% discount. This is something typically used in a shopping cart scenario, but if you want to make that scenario a bit closer to life, you need more than just a discount service that is dependent on the country. Shipping costs will also differ per country, and they belong together. We could implement this by simply renewing all of this, but then we would run into likewise issues as we had before, implementing the factory method pattern. Tight coupling.

```
var belgiumDiscountService = new BelgiumDiscountService();  
var discount = belgiumDiscountService.DiscountPercentage;
```

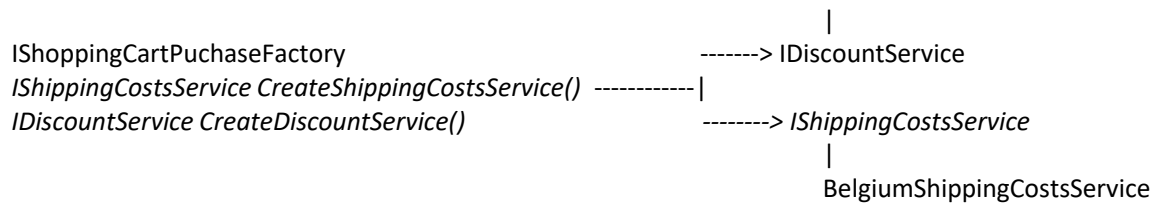
```
var belgiumShippingCostsService = new BelgiumShippingCostService();  
var shippingCosts = belgiumShippingCostService.ShippingCosts;
```

Moreover, when doing it like this, we don't convey the intent that these objects belong together. In other words, this would be a good use case for a factory that creates a family of related objects, a shopping cart purchase factory. This is an interface which contains methods to create an `IDiscountService`, that method is named `CreateDiscountService`, and a method to create an `IShippingCostsService`. That one is named `CreateShippingCostsService`. This is the family of services required for one purchase. That also means that we need to define these interfaces, an `IDiscountService` which exposes a `DiscountPercentage` and an `IShippingCostsService` which exposes the `ShippingCosts`.

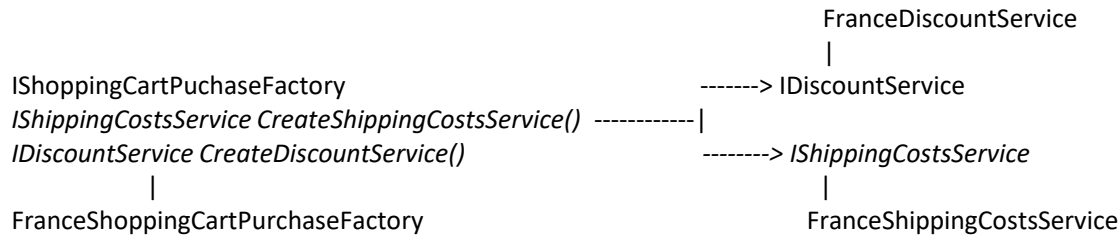
```
IShoppingCartPurchaseFactory -----> IDiscountService  
IShippingCostsService CreateShippingCostsService() -----|  
IDiscountService CreateDiscountService() -----> IShippingCostsService
```

Note that this time we are working on interfaces and not abstract base classes. If you prefer an abstract base class, you can use that as well. Most of the time these days interfaces are used. If you need to provide some basic functionality that can potentially be overridden, you could go for the abstract base class route; otherwise, interfaces will do. They already allow you to specify the expected functionality of a class. Anyway, concrete implementation could then be a `BelgiumShoppingCartFactory`, which related `BelgiumDiscountServices` and `BelgiumShippingCostsServices`. The responsibility for creating these concrete service implementations lies with the `BelgiumShoppingCartPurchaseFactory` via the `CreateDiscountService` and `CreateShippinCostsService` methods, respectively. Note that these still return the interfaces instead of the concrete implementations.

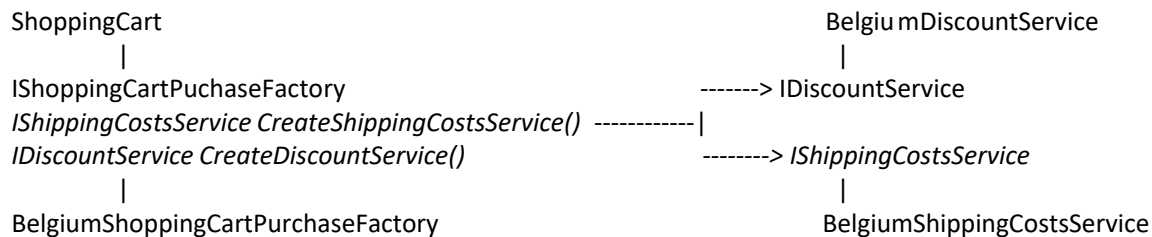
FranceDiscountService



Another implementation could then be a FranceShoppingCartPurchaseFactory, which is related to FranceDiscountService and FranceShippingCostsService classes. These follow likewise logic as we explained for Belgium.



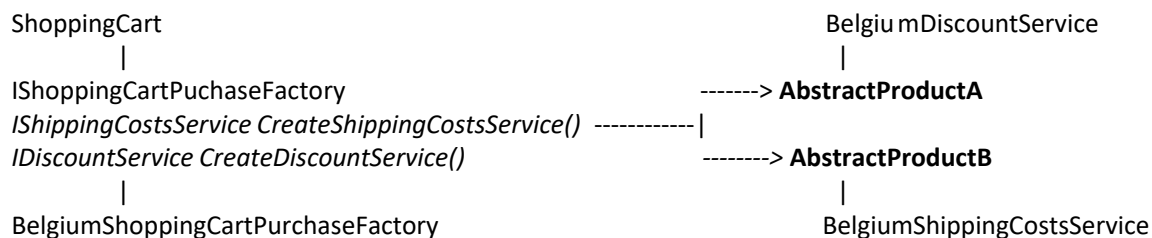
On top of that, we then have a client, that is the ShoppingCart, which uses one of the shopping cart factories to create the two services it needs. The ShoppingCart works with an IShoppingCartPurchaseFactory, an interface, not an implementation. This allows it to work with any concrete implementation.



A common way to provide the concrete factory implementation is via the constructor. Optionally add methods that use the concrete service implementations. As an example, we could add a method to this client, CalculateCosts, which calculates the actual purchase costs using the two concrete service implementations. Seeing the client works on the interface using one of these factories or adding a new factory, like the FranceShoppingCartPurchaseFactory we covered, can be done without having to change the client because the contracts remain the same and the client does not need to know about the concrete implementations. That is one of the main reasons for using this pattern. The client is decoupled from the concrete implementation.

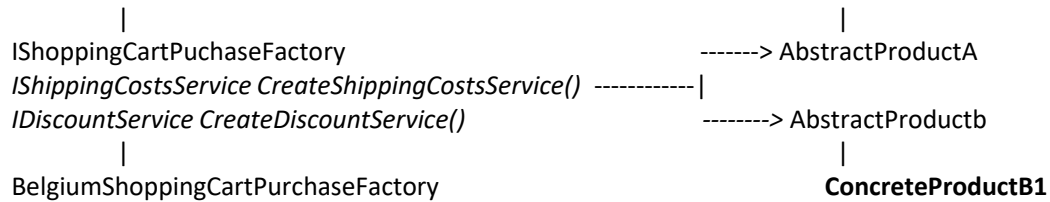
Abstract Factory Pattern Structure

The first player in the pattern structure is the abstract product. This declares an interface for a type of product object. Mapped to our example, IDiscountService and IShippingCostsService are abstract products.

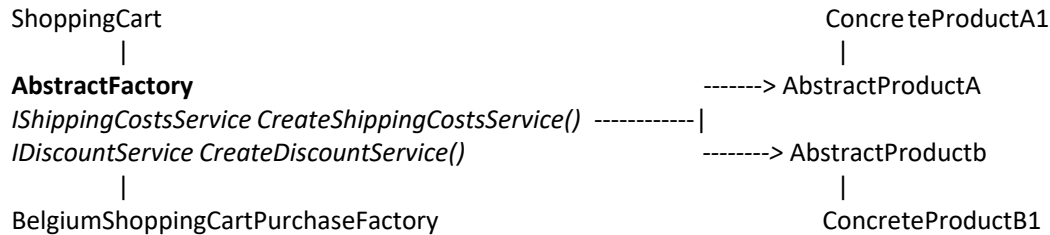


Then we have got the concrete product. This defines the product that has to be created by a corresponding factory and it implements the abstract product interface. In our example, these are the implementations of IDiscountService, in other words BelgiumDiscountService, and IShipping CostsService, in other words BelgiumShippingCostsService. Note that we note these down as belonging to the same family of products, Family1. Were we to work with another family of products, for example those related to the France as we looked into before, they would map to concrete products of family 2.

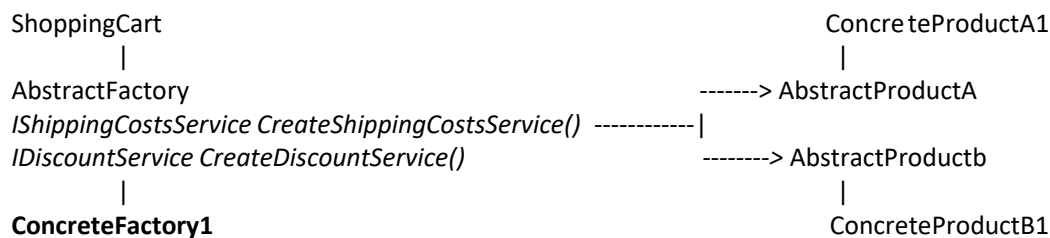




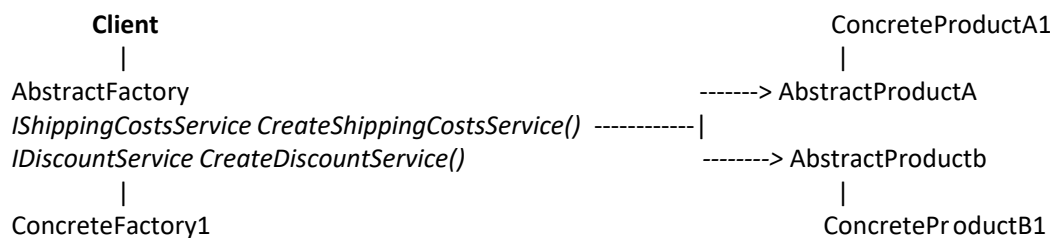
Up next, is the abstract factory, which lends its name from the pattern and declares an interface for operations that create abstract product objects. In other words this is the IShoppingCartPurchaseFactory interface from our example. The methods on it are CreateProduct A and CreateProductB, which returns the respective abstract products.



Implementations of this abstract factory are called concrete factories. These are responsible for effectively creating the related concrete products. Note that these methods create concrete products, but return them as abstract products, which it can because a concrete product implements an abstract product interface. Mapped to our use case, the BelgiumShoppingCartPurchaseFactory maps to ConcreteFactory1. The FranceShoppingCartPurchaseFactory from our example would map to ConcreteFactory2.



And lastly, there is the client. That is the part of our application which requires an abstract product implementation. Important is that it should only use the abstract implementations of the product and factory and not a concrete implementation to keep it decoupled from them. It, thus, only uses interfaces, and that is the shopping cart from our example.



And like that, we fully mapped our example to the pattern structure.

Demo - Implementing the Abstract Factory pattern

Let's start our implementation with adding our abstract factory. For our use case, that was the IShoppingCartPurchaseFactory interface. This interface describes how to create a family of related abstract products.

For our use case, it exposes two methods, CreateDiscountService, which returns an IDiscountServiceInterface, we still need to create that, and CreateShippingCostsService returning an IShippingCostsService interface, which we also need to create. Note that we are not working with any concrete implementations here. We won't implement this yet.

In Implementation.cs

```
public interface IShoppingCartPurchaseFactory
```

```

{
    IDiscountService CreateDiscountService();

    IShippingCostsService CreateShippingCostsService();
}

```

Let's first add these abstract products so this resolves again. We start with the IDiscountService interface, which will have exposed a discount percentage.

```

public interface IDiscountService
{
    int DiscountPercentage { get; }
}

```

Our IShippingCostsService then exposes a shipping costs decimal.

```

public interface IShippingCostsService
{
    decimal ShippingCosts { get; }
}

```

As you notice, these are different contracts. Then, let's implement these. First we need two implementations of IDiscountService, one concrete product, BelgiumDiscountService, which sets the discount percentage at 20. And one concrete product, FranceDiscountService, which sets the discount percentage at 10. As you see, we are starting to create families here. The BelgiumDiscountService will belong in one family, and the FranceDiscountService will belong to another family.

Insert these classes under IDiscountPercentage:

```

public class BelgiumDiscountService : IDiscountService
{
    public int DiscountPercentage => 20;
}

public class FranceDiscountService : IDiscountService
{
    public int DiscountPercentage => 10;
}

```

And we create likewise concrete products for our abstract product, IShippingCostsService. The BelgiumShippingCostsService implementation of IShippingCostsService sets the ShippingCost to 20 and the FranceShippingCostsService sets it to 25.

Insert under IShippingCostsService:

```

public class BelgiumShippingCostsService : IShippingCostsService
{
    public decimal ShippingCosts => 20;
}

public class FranceShippingCostsService : IShippingCostsService
{
    public decimal ShippingCosts => 25;
}

```

And then we can start implementing our factories. As you remember we need one factory per family. So, the first implementation of IShoppingCartPurchaseFactory becomes a BelgiumShoppingCartPurchaseFactory. This is responsible for creating the concrete BelgiumDiscountService and BelgiumShippingCostsService implementations. Yet, what they return are still the interfaces we work on.

```

public class BelgiumShoppingCartPurchaseFactory : IShoppingCartPurchaseFactory
{
    public IDiscountService CreateDiscountService()
    {
        return new BelgiumDiscountService();
    }

    public IShippingCostsService CreateShippingCostsService()
    {
        return new BelgiumShippingCostsService();
    }
}

```

And we create a likewise concrete factory implementation for our France family, the FranceShoppingCartPurchaseFactory. As you can see this is responsible for creating a family of concrete implementations of IDiscountService and IShippingCostsService, yet the methods themselves work on the underlying interfaces.

```

public class FranceShoppingCartPurchaseFactory : IShoppingCartPurchaseFactory
{
    public IDiscountService CreateDiscountService()
    {
        return new FranceDiscountService();
    }

    public IShippingCostsService CreateShippingCostsService()
    {
        return new FranceShippingCostsService();
    }
}

```

And then we need a client. Let's call that our shopping cart. We add two fields for our services to get to our family of products. These are our two abstract products. When we construct this shopping cart, we want to convey that one certain family of products should be used and there is no mixing and matching. That is why we create the discount service and shipping cost service using the same factory. You can also let the client renew the factory to use, which is a common way of handling this. Important, however, is that its implementation should remain composition-based. In this context, that means that if you want to change the factory, it is a matter of changing the factory object by replacing it with another factory object. We will also add a field to store the total cost of the order. And we are going to assume that the products we ordered have a total cost of 200 euro. What will be added to this are the costs related to our discount and shipping costs services.

```

public class ShoppingCart
{
    private readonly IDiscountService _discountService;
    private readonly IShippingCostsService _shippingCostsService;
    private int _orderCosts;

    public ShoppingCart(IShoppingCartPurchaseFactory factory)
    {
        _discountService = factory.CreateDiscountService();
        _shippingCostsService = factory.CreateShippingCostsService();

        //Assume that the total cost of all the items we ordered = 200 euro
        _orderCosts = 200;
    }
}

```

And now we add one more method, to the ShoppingCart. CalculateCosts, which will make it easy for us to test this. In this method, we write out the orderCosts minus the discount for which we use the DiscountPercentage from our discountService

plus the ShippingCosts.

Insert under public ShoppingCart(...){..}:

```
public void CalculateCosts(=
{
    Console.WriteLine($"Total cost = {_orderCosts - (_orderCosts / 100 * _discountService.DiscountPercentage) +
        _shippingCostsService.ShippingCosts }");
}
```

And that's it for our pattern implementation. Let's write some code to test this. The first thing we need to do is renew a factory because our shopping cart requires that. For that we import the AbstractFactory namespace. And then our client application renews the shopping cart, passing through the factory that is responsible for creating the concrete implementations. And now we call CalculateCosts on our shopping cart, which will write out the actual costs to the console output window. And we do the same for another family of services, the France family.

In Program.cs

using AbstractFactory;

```
var belgiumShoppingCartPurchaseFactory = new BelgiumShoppingCartPurchaseFactory(); //quick action to add using statement
var shoppingCartForBelgium = new ShoppingCart(belgiumShoppingCartPurchaseFactory);
shoppingCartForBelgium.CalculateCosts();
```

```
var franceShoppingCartPurchaseFactory = new FranceShoppingCartPurchaseFactory();
var shoppingCartForFrance = new ShoppingCart(franceShoppingCartPurchaseFactory);
shoppingCartForFrance.CalculateCosts();
```

```
Console.ReadKey();
```

Let's give this a try, the total cost are written out to our console output window, and they are different, meaning that the different factories we used, resulted in different concrete products. Our services being created and eventually used.

Use Cases for the Abstract Factory

You can use the abstract factory pattern when a system should be independent of how its products are created, composed and represented. Our real-life example nicely illustrated this. The client doesn't know about these concerns.

Closely related to this, use it when you want to provide a class library of products and you only want to reveal their interfaces and not their implementations. The decoupling between client and factory and product is a win in this regard.

Also, use it when a system should be configured with one of multiple families of products. As we learned, the abstract factory deals with product families, products that belong together. In that way, it is different from the factory method pattern.

Also, use it when a family of related product objects is designed to be used together and you want to enforce this constraint.

Pattern Consequences

This pattern, as most, has a few benefits and liabilities.

First, it isolates concrete classes because it encapsulates the responsibility and the process of creating product objects. In other words, clients are isolated from implementation classes, and that is mostly an advantage because it means that we are avoiding tight coupling.

You can thus, easily introduce new products without breaking client code. That means we are adhering to the open/closed principle from the SOLID coding principles.

Also, seeing the code to create product is contained in one place, we are adhering to the single responsibility principle as well. It also makes exchanging product families easy. Seeing the class of a concrete factory only appears once in an application where you instantiate it, it is easy to change that concrete factory. You can switch it out for another one by changing that one line of instantiating code as the rest of the system works on the interface and not on the implementation.

It also promotes consistency among products. This is because products in the same family are created by the same factory. Seeing an application only uses one abstract factory at a time, the products that are used are related and compatible with each other.

A potential disadvantage is that supporting new kinds of products is rather difficult. If you want to extend the abstract factory, you have to change the factory interface, which also means changing the implementation of all of its subclasses.

Comparing the Abstract Factory to Factory Method

Abstract factory is closely related to factory method. In fact, both are sometimes referred to as the factory pattern. As they are creational patterns they are both about creating objects. But there is more ways of doing that. In fact, two more creational patterns are coming up next. So, why are these two so closely related?

Factory method	Abstract factory
Exposes an interface with a method on it, the factory method, to create an object of a certain type.	Exposes an interface to create related objects: families of objects. Multiple methods are exposed on the abstract factory
The return type is never an object, it is in fact a interface or an abstract base class.	The return types of these methods are interfaces of abstract classes.
The factory method only produces one product.	Abstract factory produces families of products.
Creates objects through inheritance by implementing the interface or abstract base class with the factory method on it.	Creates families of objects through composition instead of inheritance. It contains fields that hold the concrete factories that are used for creating concrete products

Abstract factory and factory method can be combined.

Related patterns

Abstract factory classes are often implemented with factory methods, as we just covered.

But they can also be implemented using the prototype pattern.

Also a concrete factory, so an abstract factory implementation, is often implemented as a singleton.

- Factory method
Abstract factory can be implemented using factory methods.
- Prototype
Abstract factory can be implemented using prototypes.
- Singleton
A concrete factory is often implemented as a singleton

Creational Pattern: Builder

Monday, October 31, 2022 1:43 PM

Coming up

In this module we will cover the builder pattern. The intent of the builder pattern is to separate the construction of a complex object from its representation. So, the same construction process can create different representations.

We will start by describing the pattern, when to use it, and why. After we described the pattern, we will map that to the pattern structure. As a real-life example, we will create a vehicle builder, which can be used to create cars and motorcycles.

We will also implement that example. After that we will continue to learn about use cases for abstract factory pattern and its consequences. We will end the module by looking into related patterns.

Describing the Builder Pattern

The builder pattern is a common creational pattern from the Gang of Four.

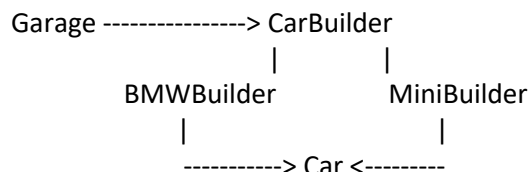
It deals with object creation, complex objects in this case, because the intent of the builder design pattern is to separate the construction of a complex object from its representation. In other words, it lets us construct complex objects step by step. By doing so, the same construction process can create different representations.

Let's explain that definition with a real-life example:

A very typical example of the builder pattern in real life is a piece of software for a garage that, at a certain point, requires you to construct cars. We can consider such a car as a complex object consisting of different parts, say a frame, and an engine in this example. But not all cars are constructed in the same manner. A three-door Mini has a different frame and engine than a five-door BMW for example, so the builder for these is different. This is a prime example of where the builder pattern shines.

We will have a builder for a car, and on it methods to build the engine, build a frame, and so on. And also a way to return a car, for example via a car property.

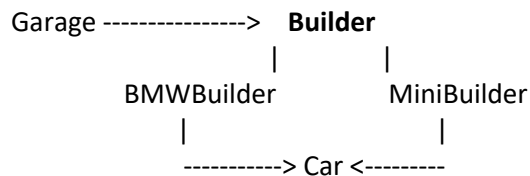
Seeing we are returning a car, we will also have a Car class. On this class, we will have a method to add a part, for example adding an engine, adding a frame and so on. How the car class adds and store these parts isn't of importance for our pattern implementation. And then we will have different implementations of this builder, one for building the mini, and one for building a BMW and so on. Important here is that these different implementations still have the same methods. So, creating the complex car is transparent for the consumer of the builder. No matter which builder implementation is used, it has to call into the same methods.



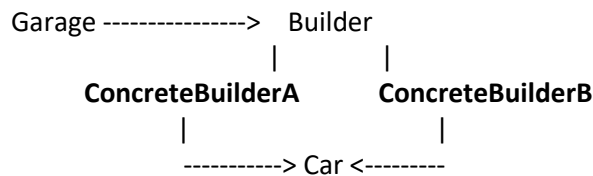
This is possible because, just as with other patterns in the creational pattern category, we are working on interfaces and not on implementations. We have again avoided tight coupling. The consumer, in our case, would be a garage. It needs a builder to effectively construct a car. One way of passing that through is via a construct method, which accepts a car builder.

Builder Pattern Structure

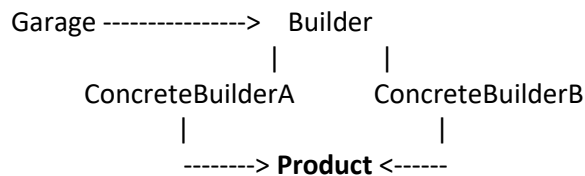
The first part of the pattern structure is the builder. This specifies an abstract interface for creating parts of a product object. In our case, this maps to our car builder. It is an interface for creating parts of the car.



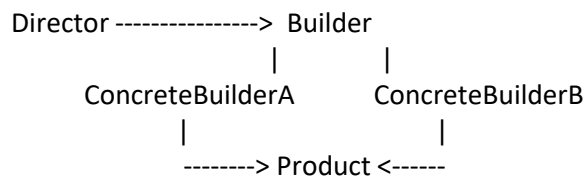
Then, we have the concrete builder. They construct and assemble the product, which in our case, and they define and keep track of the representation it creates. That is the product. It also provides a way to return that product. In our example, this maps to our **BMWBuilder** and our **MiniBuilder**.



Then there is the product itself. That represents a complex object under construction. In our case, that maps to the car. It potentially includes classes that represent the constituent parts. In our case, that could be a **Frame** class, an **Engine** class, and so on.



And lastly, we have got the director. This constructs an object using the builder interface. In our example, we could map that to our garage. In some implementations though, director allows setting the builder interface to use via a property. In others, it allows passing through via the construct method.



These are implementation details we can decide on ourselves.

Demo - Implementing the Builder pattern

Before we start coding, remember that, as always with these design patterns, we should see them as a template. It is important to stick to the intent of the pattern, but implementations can vary.

We are going to start with the product. On it, we will define a set of car parts as a **List of string**. This is for demo purposes to avoid having to create separate classes for engine, for frame, etc. Each car also has a type, which we will accept via a constructor. Like that we can convey meaning to consumers, letting them know that we expect a non-null **carType**. And then we add one method, **AddPart**. This method will be used to add parts to our part list, in other words to construct the car. And lastly, we override the **ToString** method simply to make it a bit easier to test. In this method, we write out the type of the car and the parts.

In Implementation.cs

```
public class Car
{
    private readonly List<string> _parts = new();
    private readonly string _carType;

    public Car(string carType)
    {
        _carType = carType;
    }

    public void AddPart(string part)
    {
        _parts.Add(part);
    }

    public override string ToString();
    {
        var sb = new StringBuilder(); //Quick action to add using statement (system.text)
        foreach (string part in _parts)
        {
            sb.Append($"Car of type {_carType} has part {part}. ");
        }
        return sb.ToString();
    }
}
```

Up next, is the builder class, which is the abstract base class for concrete builders. It has a car property. That is the product it is building. We initialize that in the constructor by passing through the carType. Important is that we also define two abstract methods on this CarBuilder, BuildEngine and BuildFrame. These are the methods which will have to be implemented by concrete builders to actually build the engine and build the frame.

```
public abstract class CarBuilder
{
    public Car Car {get; private set; }

    public CarBuilder(string carType)
    {
        Car = new Car(carType);
    }

    public abstract void BuildEngine();
    public abstract void BuildFrame();
}
```

That brings us to the concrete builders. Let's start with the builder to build a Mini. It implements the CarBuilder, which adds the two methods from CarBuilder, BuildEngine and BuildFrame. Let's start with the constructor. We construct this MiniBuilder by passing through the carType to the base constructor. That is something we can do thanks to the fact that CarBuilder is an abstract base class and not just an interface. The BuildEngine implementation is then responsible for adding the engine to the car. Car is defined on the CarBuilder base class. And BuildFrame is responsible for adding a frame. In our case it is

just a string. In a more extensive implementation, you would effectively use classes for this.

```
public class MiniBuilder : CarBuilder //Quick Action to get the two missing methods
{
    public MiniBuilder() : base("Mini"){

        public override void BuildEngine()
        {
            Car.AddPart("not a V8");
        }

        public override void BuildFrame()
        {
            Car.AddPart("3-door with stripes");
        }
    }
}
```

And then we do something similar to the second concrete builder, a BMWBuilder. We invoke the base class constructor, and we add the implementations for BuildEngine and BuildFrame.

```
public class BMWBuilder : CarBuilder //Quick Action to get the two missing methods
{
    public BMWBuilder() : base("BMW"){

        public override void BuildEngine()
        {
            Car.AddPart("a fancy V8 engine");
        }

        public override void BuildFrame()
        {
            Car.AddPart("5-door with metallic finish");
        }
    }
}
```

Lastly, we have the director, a Garage. A director needs a builder to effectively construct something, a car in our case, so we store that in a field. Then we need to be able to inject that builder somehow. And for this example, we will do that via a construct method. The builder is thus injected, stored, and using the builder, the car is built. Seeing, we are working on an interface here, the CarBuilder abstract base class, and not on a concrete implementation, we can inject any concrete builder.

```
public class Garage
{
    private CarBuilder? _builder;

    public Garage(){

        public void Construct(CarBuilder builder)
        {
            _builder = builder;
            _builder.BuildEngine();
            _builder.BuildFrame();
        }
    }
}
```

```
}
```

Now let's test this. First, we renew the director. That is our garage. To be able to do that, we need to import the BuilderPattern namespace. And then we need instances of our builders because the construct method on the garage expects that. Then we create MiniBuilder and a BMWBuilder instance. And to test this, we call the Construct method on the garage, passing through the MiniBuilder and BMWBuilder, respectively.

In Program.cs

```
var garage = new Garage(); //Insert using statement
```

```
var miniBuilder = new MiniBuilder();  
var bmwBuilder = new BMWBuilder();
```

```
garage.Construct(miniBuilder);  
Console.WriteLine(miniBuilder.Car.ToString());  
//or  
garage.Show();
```

```
garage.Construct(bmwBuilder);  
Console.WriteLine(bmwBuilder.Car.ToString());  
//or  
garage.Show();
```

```
Console.ReadKey();
```

A small variation. What is often done with this pattern is adding a slight extension. To avoid having to call into the Car to visualize it, you can add a method on the Garage class to show it. So, let's add a new method, Show, on our Garage.

```
Insert below the construct(){..}:  
public void Show()  
{  
    Console.WriteLine(_builder?.Car.ToString());  
}
```

Use Cases for the Builder pattern

You can use this pattern when you want to make the algorithm for creating a complex object independent of the parts that makes up the object and how they are assembled. A Mini or BMW is a complex object to build, and it is the builder that becomes responsible for constructing it, not the director.

You can also use it when you want the construct process to allow different representations for the object that is constructed. In our case, we want to build a car. Different builder implementations allow for different representations of that car.

Pattern Consequences

One of the basic consequences of the builder pattern is that it lets us vary a product's internal representation. As you know the builder provides the director with an interface for constructing the product. Like that, the interface lets the builder hide the representation and internal structure of the product. This also makes it easy to change a product's representation. All you need to do is add a new kind of builder.

Next to that, it isolates code for construction and representation. This is the single responsibility principle from the SOLID coding principles. It then improves modularity by encapsulating the way a complex object is constructed and represented. Clients don't need to know anything about the internal structure of the product.

It also gives us finer control over the construction process. The product is created in different steps instead of in one go, and each of these steps is under the director's control.

A negative consequence is that the complexity of our code increases because we are often creating multiple new classes.

Related patterns

The builder pattern is related to the abstract factory pattern as both can be construct complex objects. The main difference is that the builder constructs the complex objects step by step.

It also relate to the singleton pattern as the builder can be implemented as a singleton.

And lastly, it is related to a structural pattern, the composite pattern, as composites are often built by builders.

- Abstract factory
Both can be used to construct complex objects, but the builder constructs the complex objects step by step.
- Singleton
A builder can be implemented as a singleton.
- Composite
Composite are often built by builders.

Creational Pattern: Prototype

Monday, October 31, 2022 1:43 PM

Coming up

In this module, we will cover the prototype pattern. Its intent is to specify the kinds of objects to create using a prototypical instance and create new objects by copying this prototype.

We will start by describing the pattern, and looking into the pattern structure. As a real-life example, we will use the creation of an employee and a manager, both deriving from Person via a Clone method. After that we dive a bit deeper into shallow versus deep copies, which we will both implement. We will cover the ICloneable interface as well. After that we will continue to learn about use cases for prototype pattern and its consequences. We will end the module by looking into related patterns.

Describing the Prototype Pattern

The prototype pattern is one of Gang of Four design patterns. It belongs to the creational patterns group, which means it is a pattern that deals with object creation. This is the last creational pattern that will be covered.

The intent of the pattern is to specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype. In other words, it lets us copy existing objects without making the client code dependent on their concrete classes.

It is a pretty easy pattern, but as with most of these pattern definitions, they are easier to understand with an example. Imagine we have a system which we work with people. We can simply create new people by renewing instances of a Person class or, more realistically, assuming that Person is an abstract class, an Employee or Manager class, that derives from Person. Now imagine the use case where most property or field values of an existing object are required in a new object. Or maybe you even need all property values of the new object to match the existing object. In that case, we can simply copy them over. But that requires the client to have intrinsic knowledge of the concrete classes we are working with, Employee and Manager. It also needs to know how to create them.

```
var existingManager = new Manager("Cindy");  
var existingEmployee = new Employee("Kevin", existingManager);
```

```
var newEmployee = new Employee();  
newEmployee.Name = existingEmployee.Name;  
newEmployee.Manager = existingEmployee.Manager;
```

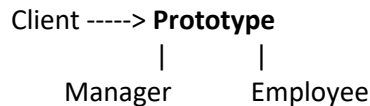
We can go about this in another way as well. We could create a method on our Employee and Manager classes, Clone, that will return a copy of itself. Moreover, as we know, both Employee and Manager derive from Person, we can define this method on the person class and override it in the subclasses. The client application only needs to know about the Person class's Clone method to be able to clone the subclasses, like Employee and Manager. In essence, the prototype pattern describes a way to create a new object by cloning an existing one.

```
Client -----> Person  
                  |   |  
                Manager Employee
```

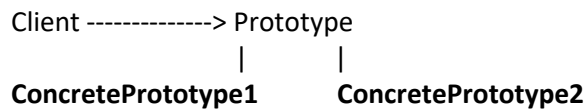
Prototype Pattern Structure

The prototype pattern obviously contains a Prototype class. The prototype declares an interface for

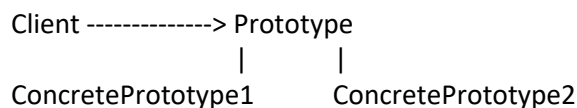
cloning itself. It maps to the person class. A way to implement this would be as an abstract base class with an abstract Clone method on it. But as is often the case, an interface are possible as well.



And now we have got the ConcretePrototype. These are implementations of the prototype that implement the actual clone operation, which returns a copy of itself. This maps to our Manager and Employee classes.



And lastly, we have the client. This one creates a new object by asking a Prototype to clone itself. So the only thing we still need to change to our prototype pattern structure is that the client doesn't execute clone on Person, but it does so on a Prototype. And that is it.



Demo - Implementing the Prototype pattern

Time to implement the pattern. Let's start with the prototype itself first. We are going to use an abstract base class for that, defining a Name property and a Clone method, which returns a Person object.

```

In Implementation.cs
public abstract class Person
{
    public abstract string Name { get; set; }

    public abstract Person Clone();
}
  
```

And now we need to implement this. Let's start with a Manager class deriving from Person. That means we need to implement the abstract base class, so let's do that. And we see that we need to provide a Name. What we can do for that is add a constructor that accepts that name, conveying that we expect this object, the Manager, to be constructed with a name. We can now simply change the name property to contain getter and setter via auto property syntax. In the constructor, we set the property name to the passed-through name, and now we also need to implement the Clone method. We could implement this by renewing Person and manually copying over all the fields, but there is an easier, built-in way to do this, by calling into MemberwiseClone. This is a method that is defined on object. So, every instance you use in C# has access to this. It creates a shallow copy of the existing object.

```

public class Manager : Person //Quick action to implement the necessary elements
{
    public override string Name { get; set; }

    public Manager(string name)
    {
  
```

```

        Name = name;
    }

    public override Person Clone()
    {
        return (Person)MemberwiseClone();
    }
}

```

Then, let's add another concrete prototype, Employee. It is more or less the same as the Manager class, with one difference. An Employee has an additional property. It's Manager. So, we construct this one by accepting a name and a manager object.

```

public class Employee : Person //Quick action to implement the necessary elements
{
    public Manager Manager { get; set; }

    public override string Name { get; set; }

    public Employee(string name, Manager manager)
    {
        Name = name;
        Manager = manager;
    }

    public override Person Clone()
    {
        return (Person)MemberwiseClone();
    }
}

```

Let's test this. Start by renewing the manager first. For that we need to import the Prototype namespace. We will pass through a name, Cindy. And then we will try to clone this manager. For that, we can call Clone on the manager object. Note that we do need to cast this back to a manager if we want to use it as a manager because the Clone method returns a person object, the abstract base class. And then we do the same for employee. The difference here is that we pass through the manager just created. But let's make a small change. Right before the ReadKey statement, let's change the manager name and write out the employee info again. The manager name on employeeClone was changed, even though the clone was created before we changed the name. That may be what you want, but often it is not. You might expect to get an actual copy of the manager object as well when you clone the employee object. This is a limitation of MemberwiseClone. That statement always creates shallow copies. Getting an actual copy of the manager object as well would mean we need support for deep copies or clones.

In Program.cs

```

var manager = new Manager("Cindy"); //Quick Action to insert using statement
var managerClone = (Manager)manager.Clone();
Console.WriteLine($"Manager was cloned: {managerClone.Name}");

var employee = new Employee("Kevin", managerClone);
var employeeClone = (Employee)employee.Clone();
Console.WriteLine($"Employee was cloned: {employeeClone.Name}");

managerClone.Name = "Karen";

```

```
Console.WriteLine($"Employee was cloned: {employeeClone.Name}, " + $"with manager  
{employeeClone.Manager.Name}");
```

```
Console.ReadKey();
```

Shallow Copy vs Deep Copy

A shallow copy is a copy of primitive type values. So, strings, integers, and so on will be copied. These are independent of the original object. But complex types that might be properties of the object you are cloning will be shared across the different clones. Because of that, changing a value from that complex object, for example the manager's name, results in changes on both the original object and the clone.

A deep copy, on the other hand, also results in copies of the complex types. In that case, we can modify the complex object properties of the original object without it affecting those of the cloned object and vice versa.

Demo - Supporting Deep Copies

In this demo, we will add support for deep copies or clones. Let's start by adding a Boolean, `deepClone`, as a parameter to the clone method signature. We do that in the `Person` abstract base class, which means we also need to do this in the `Manager` class and the `Employee` class in their override `Person Clone` method. "Personally, I like passing through a default value for this, `false`, so clients don't have the obligation to be explicit about it".

```
In implementation.cs  
public abstract class Person  
{  
    public abstract string Name { get; set; }  
  
    public abstract Person Clone(bool deepClone);  
}  
  
//both in Employee & Manager class  
public override Person Clone(bool deepClone = false)  
{  
    return (Person)MemberwiseClone();  
}
```

Now, all that is left is effectively creating such a deep clone. We could manually traverse the object tree, creating shallow copies of each complex type and assign it as a property value. But the more common approach is making a binary copy using a binary format. The code for that would look like something you can see below:

```
public override Person Clone(bool deepClone = false)  
{  
    if (deepClone)  
    {  
        var formatter = new BinaryFormatter();  
        using (var stream = new MemoryStream())  
        {  
            formatter.Serialize(stream, this);  
            stream.Seek(0, SeekOrigin.Begin);  
            return (Person)formatter.Deserialize(stream);  
        }  
    }  
}
```

```

    }
    return (Person)MemberwiseClone();
}

```

However, since that format was introduced in .NET, deserialization vulnerabilities were discovered. As a result, this does not follow modern best practices, and the `Deserialize` method can be used as a vector for attackers to perform denial-of-service attacks against consuming apps. This is not something that can be solved with this binary format though. What Microsoft currently recommends is to simply no longer use the binary formatter anymore. (If you want to read more about that, visit:

<https://docs.microsoft.com/en-us/dotnet/standard/serialization/binaryformatter-security-guide>)

Another common approach is to use a `JsonSerializer` instead. `System.Text.Json` is built in, but it currently doesn't support parameterless constructor classes, and that happens to be what we have here. But luckily, `Json.NET` does, so let's give that a try.

First, let's open the NuGet dialog, and let's look for `Json.NET`. `Newtonsoft.Json` is how you can find it. Install it. Import the `Newtonsoft` namespace. And from that moment on, we can call `SerializedObject` on our current object and, after that, deserialize that object.

So first, we serialize to a string, and then we deserialize that string back to an object we can return. The warning you see here is that this may return a null value, but that is ok for us. And we do the same in `Manager` class. But change so we deserialize it to a `Manager`.

```

public override Person Clone(bool deepClone = false) //Employee
{
    if (deepClone)
    {
        var objectAsJson = JsonConvert.SerializeObject(this); //Quick Action to insert using
        statement.
        return JsonConvert.DeserializeObject<Employee>(objectAsJson);
    }
    return (Person)MemberwiseClone();
}

public override Person Clone(bool deepClone = false) //Manager
{
    if (deepClone)
    {
        var objectAsJson = JsonConvert.SerializeObject(this); //Quick Action to insert using
        statement.
        return JsonConvert.DeserializeObject<Manager>(objectAsJson);
    }
    return (Person)MemberwiseClone();
}

```

Lets test this. To do that , we change our clone call, this time requesting a deep clone. We do that when caling the employee, and let's give that a try.

In `Program.cs`

```

var manager = new Manager("Cindy"); //Quick Action to insert using statement
var managerClone = (Manager)manager.Clone();
Console.WriteLine($"Manager was cloned: {mangerClone.Name}");

```

```

var employee = new Employee("Kevin");
var employeeClone = (Employee)employee.Clone(true);
Console.WriteLine($"Employee was cloned: {employeeClone.Name}");

managerClone.Name = "Karen";
Console.WriteLine($"Employee was cloned: {employeeClone.Name}, " + $"with manager
{employeeClone.Manager.Name}");

Console.ReadKey();

```

What About the ICloneable Interface

.NET includes an interface named the ICloneable interface. We could use that. It enable us to provide a customized implementation that creates a copy of an existing object. The ICloneable interface contains one member, the clone method, which we could implement.

So why not use that. We could, but there are a few disadvantages.

- One, the ICloneable interface simply requires that your implementation of the Clone method returns a copy of the current object instance. It does not specify whether the cloning operation performs a deep copy, a shallow copy, or something in between.
- Two, it also doesn't require all property values of the original instance to be copied to the new instance.
- Three, it returns an object, which means the client could need an additional cast.

So, you can indeed use it, but I would advise not to.

Use Cases for the Prototype pattern

This pattern is useful when a system should be independent of how its objects are created and to avoid building a set of factories that mimics the class hierarchy, for example, class A deriving from class B, deriving from class C, potentially with all of them having properties of type class D, E, or F. Creating such a hierarchy with factories would lead to a lot of boilerplate code, which is to be avoided.

Another good use case is when a system should be independent of how its objects are created, and when instances of a class can have one of only a few different combinations of states. In those cases, starting from a clone will often be more convenient than starting from an instance with default values.

Pattern Consequences

Much of the consequences are related to the abstract factory and builder consequences.

Prototype hides the ConcreteProduct classes from the client, which reduces what the client needs to know. It doesn't need to know about the concrete classes of the objects of which you want a copy.

A positive prototype-specific consequences is reduced subclassing. Instead of having to create a factory method to make a new object, you can simply clone an existing one.

A potential liability is that each implementation of the prototype base class must implements its own clone method, which might not always be convenient.

Related patterns

Prototype and abstract factory can be used together.

A factory might store a set of prototypes from which it clones when a new instance is requested. There is also an relation to another creational pattern, the factory method pattern.

Prototypes don't require subclassing creators. They are not based on inheritance, which is a plus when you favor composition over inheritance, like the Gang of Four. However, they often require an initialize operation on the Product class, which the factory method doesn't. Factory method is based on inheritance, but doesn't require that initialization step. Prototype can also be implemented as a singleton, which means it is related to that pattern.

Lastly, when using the composite and decorator patterns, both structural, the prototype can be beneficial for convenient object creation.

- Abstract factory
A factory might store a set of prototypes from which it clones when a new instance is requested.
- Factory method
Factory method is based on inheritance, but doesn't require an initialization step.
- Singleton
Prototype can be implemented as a singleton.
- Composite
Can use prototype for convenient object creation.
- Decorator
Can use prototype for convenient object creation.

Structural Pattern: Adapter

Monday, October 31, 2022 1:43 PM

Coming up

In this module, we will cover the adapter pattern, also known as a wrapper pattern. It is a very common one used to let classes work together that couldn't before because of incompatible interfaces.

We will start by describing the pattern, which will quickly lead us to two different variations, the class adapter and the object adapter. We will look into the pattern structure. And, as a real-life example, we will use a travel agent scenario that needs to load cities from an external system. We will continue learning about the use cases for this pattern and the pattern consequences, which can be both positive and negative. We will end the module by looking into related patterns.

Describing the Object Adapter Pattern

The adapter pattern is a commonly used structural pattern from the well-known Gang of Four design patterns. Its intent is to convert the interface of a class into another interface clients expect. So, the adapter pattern lets classes work together that couldn't otherwise because of incompatible interfaces.

There are two variations of this pattern, the first one being the object adapter pattern. Imagine that you need to integrate with an external system that exposes a city. You could for example, be creating an application that needs to talk to an API that exposes these cities. Or you could just be talking to another application layer that works on a different model than the one you eventually need. For our use case, let's say we got a city from ExternalSystem class with the name, a nickname, and the amount of inhabitants as an integer.

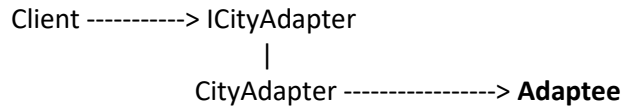
This CityFromExternalSystem class is exposed by an ExternalSystem class, which contains one method, GetCity, which returns our city from ExternalSystem. But you cannot work with this city from ExternalSystem class. Your system needs a City class, which consists of a FullName property and an Inhabitants property, which is a long or int64 instead of a regular int32. So, now you somehow need to convert that CityFromExternalSystem into a City. Worded slightly differently, you need to adapt the ExternalSystem.

One way of doing that is via a wrapper. You can create a new class, which adheres to a certain interface and which calls into the ExternalSystem. After it gets the result of the GetCity call at level of the external system, it converts it to a city object and returns it. That wrapper is what we would call a city adapter, and the contractor interface it must adhere to is what we could call our ICity adapter. The client then calls GetCity on ICity adapter.

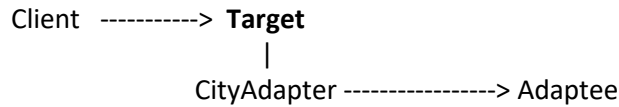


Object Adapter Pattern Structure

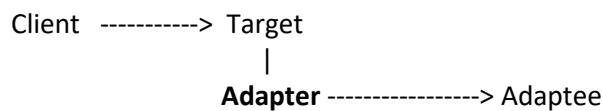
Our two classes, CityFromExternalSystem and City, are simple helper classes that helped us describe the pattern. They are not the part of the pattern structure. Our external system, which contains the GetCity method that returns a CityFromExternalSystem, that is what would be called an adaptee, the thing that needs to be adapted. This defines an existing interface that needs to be adapted, the specific request. So in our diagram, we switch ExternalSystem to Adaptee.



The ICityAdapter interface is named the Target. In other words, the thing we need to adapt the adaptee to. It defines the domain-specific interface that the client works with. So we switch ICityAdapter to Target



The client works with objects conforming to the Target interface. The implementation of that target, the CityAdapter, is named the adapter, or if you want, the wrapper that adapts the adaptee to the target. So we change the CityAdapter to Adapter, and with that we got our pattern structure.



Demo - Implementing the Object Adapter pattern

Let's implement the object adapter pattern. In this module we will implement two different implementations from the adapter pattern, the object and the class adapter, it is not created a implementation class in advanced. Let's add a class for the object adapter pattern, and name it, ObjectAdapterImplementation, and that takes care of that.

We will start with the CityFromExternalSystem class. This is a simple class with three properties. We accept three matching fields in the constructor. Like that, we convey that this object must be created with these three field values. With that we have got the helper class that our adaptee needs, so we can we can add that adaptee.

In ObjectAdapterImplementation:

```

public class CityFromExternalSystem
{
    public string Name { get; private set; }
    public string NickName { get; private set; }
    public int Inhabitants { get; private set; }

    public CityFromExternalSystem(string name, string nickName, int inhabitants)
    {
        Name = name;
        NickName = nickName;
        Inhabitants = inhabitants;
    }
}
  
```

The ExternalSystem class, it exposes one method, GetCity, which returns a new CityFromExternalSystem.

Below CityFromExternalSystem{..}:

```

public class ExternalSystem
{
  
```



```

    public CityFromExternalSystem GetCity()
    {
        return new CityFromExternalSystem("Antwerp", "'t Stad", 500000);
    }
}

```

The city class, is another helper class we need, and it looks a bit like the CityFromExternalSystem class. But instead of a name and nickname, it only has one string property, a FullName. The inhabitants property is of type long. That is an int64. The CityFromExternalSystem's inhabitants is an int, which means int32. So these classes are alike, yet different.

```

Below ExternalSystem {..}
public class City
{
    public string FullName { get; private set; }
    public long Inhabitants { get; private set; }

    public City(string fullName, long inhabitants)
    {
        FullName = fullName;
        Inhabitants = inhabitants;
    }
}

```

The next thing we do, now that we got this helper class is create or define the target. We use an interface for this. An abstract class is also a possibility, but "I" prefer to use an interface in this case, as we don't need base implementation for this example. What the interface does is define a GetCity method, which returns a City instead of a CityFromExternalSystem. In other words, we have now got two incompatible systems.

```

Below City:
public interface ICityAdapter
{
    City GetCity();
}

```

We can make them compatible by implementing this ICityAdapter, and that is what we name the adapter. When we implement the ICityAdapter interface, we see that we need to implement one method, GetCity. But to make this compatible, we also want to store an ExternalSystem property. In other words we are going to be using composition to make these two different interfaces or systems compatible. For the demo, I'm doing it up ad hoc, but you can inject it if needed or otherwise ensure that the CityAdapter gets ahold of this. Again, as already mentioned, the design patterns should be seen as a template to start from. You can change the template if it fits your needs, as long as you stick to the intent of the pattern. And now we only need to implement the GetCity method. We do that by calling into the ExternalSystem and then adapting the result, which is a CityFromExternalSystem. In this method, we call into the external system, and then we adapt the result, which is a CityFromExternalSystem to a City object, and that is what we return. With that, we have created our pattern.

```

Below ICityAdapter:
public class CityAdapter : ICityAdapter
{
    public ExternalSystem ExternalSystem {get; private set; } = new();
}

```

```

public City GetCity()
{
    // call into the external system
    var cityFromExternalSystem = ExternalSystem.GetCity();

    // adapt the CityFromExternalCity to a City
    return new City(
        $"{cityFromExternalSystem.Name} - {cityFromExternalSystem.NickName}",
        cityFromExternalSystem.Inhabitants);
}
}

```

Now let's create some sample code to try it out. First, we create a new instance of our adapter. For that, we need to import the Adapter namespace, and then we try to get a city from it. This should call into the external system and return a City object. To check whether that is really the case, we simply write out the FullName and the Inhabitants. Note that what should happen now is that the ExternalSystem.GetCity call, which would result in Antwerp being returned will be transformed into a city object. So we should still see the string Antwerp being written out, including the nickname this time, Stad.

```

In Program.cs
ICityAdapter adapter = new CityAdapter(); //Quick action to import using statement
var city = adapter.GetCity();

```

```

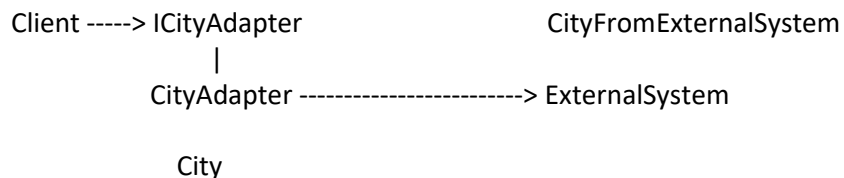
Console.WriteLine($"{city.FullName}, {city.Inhabitants}");
Console.ReadKey();

```

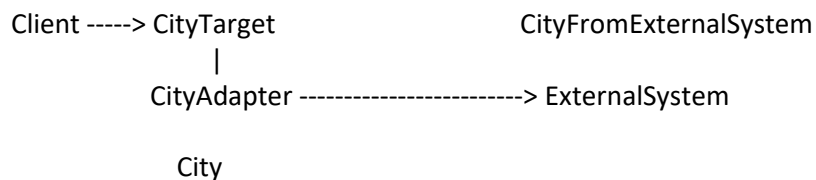
Let's try that, and that is indeed the case. So far for our object adapter. Let's look at a variation of this pattern, the class adapter.

Describing the Class Adapter pattern

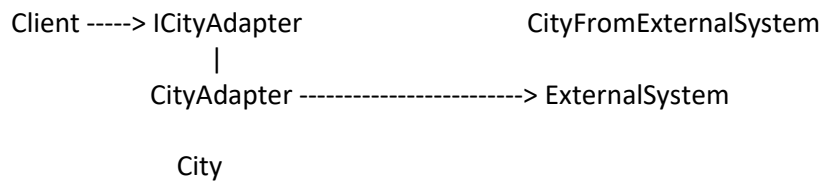
The object adapter pattern we looked into works via object composition. What is shown below is the diagram we ended up with when describing the object adapter.



The class adapter pattern is a variation which relies on multiple inheritance to adapt one interface to another. In our case, the CityAdapter would derive from the External System and from the Target, assuming both are abstract base classes. There is only one little problem with this. C# is a language that prefers composition over inheritance, and it does not support multiple inheritance. We could have done this in Java, but not in C#. However, we can implement it with a slight variation.

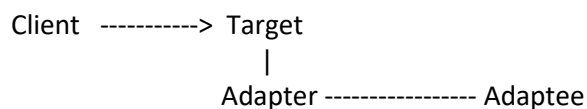


We could let our CityAdapter derive from ExternalSystem and implement the ICity adapter interface. Like that, we don't need multiple inheritance. We are implementing a base class, the ExternalSystem, and we adhering to an interface. And that combination is supported by C#. Let's have a look at the pattern structure.



Class Adapter Pattern Structure

We are looking at the object Adapter structure with a slight change in which we state that the Adapter implements the adaptee, we get the class pattern variation we can work with. The Adapter no longer has to store an Adaptee object to call into the specific request on the Adaptee object. Because the Adapter derives from the Adaptee, it can now call into the specific request method directly from its parent class, the Adaptee.



Demo - Implementing the Class Adapter pattern

In this demo, we are going to implement the class adapter. For that, let's add a new class to our adapter project. Name it ClassAdapterImplementation. Change the namespace to ClassAdapter, and copy the ObjectAdapterImplementation code as it is about 90% the same code

In ClassAdapterImplementation.cs:

```

public class CityFromExternalSystem
{
    public string Name { get; private set; }
    public string NickName { get; private set; }
    public int Inhabitants { get; private set; }

    public CityFromExternalSystem(string name, string nickName, int inhabitants)
    {
        Name = name;
        NickName = nickName;
        Inhabitants = inhabitants;
    }
}

public class ExternalSystem
{
    public CityFromExternalSystem GetCity()
    {
        return new CityFromExternalSystem("Antwerp", "'t Stad", 500000);
    }
}

public class City
{

```

```

    public string FullName { get; private set; }
    public long Inhabitants { get; private set; }

    public City(string fullName, long inhabitants)
    {
        FullName = fullName;
        Inhabitants = inhabitants;
    }
}

public interface ICityAdapter
{
    City GetCity();
}

public class CityAdapter : ICityAdapter
{
    public ExternalSystem ExternalSystem {get; private set; } = new();

    public City GetCity()
    {
        // call into the external system
        var cityFromExternalSystem = ExternalSystem.GetCity();

        // adapt the CityFromExternalCity to a City
        return new City(
            $"{cityFromExternalSystem.Name} - {cityFromExternalSystem.NickName}",
            cityFromExternalSystem.Inhabitants);
    }
}

```

There is only one change we have to make, and that is at level of the CityAdapter class. We let it derive from the ExternalSystem class and implement the ICityAdapter. That means we need to implement GetCity a little bit differently. Instead we call GetCity on the base class, the ExternalSystem class. We can do that as we are deriving from it, and that should be it.

Change:

```

public class CityAdapter : ExternalSystem, ICityAdapter
{
    public City GetCity()
    {
        // call into the external system
        var cityFromExternalSystem = base.GetCity();

        // adapt the CityFromExternalCity to a City
        return new City(
            $"{cityFromExternalSystem.Name} - {cityFromExternalSystem.NickName}",
            cityFromExternalSystem.Inhabitants);
    }
}

```

To test this, simply change the namespace we import in Program.cs to ClassAdapter. Which means that now the classes from our ClassAdapterImplementation are used. The result will be exact the same as

with object adapter.

In Program.cs
using ClassAdapter

Use Cases for the Adapter pattern

This pattern is useful when you want to use an existing class, but the interface does not match the one you need, adapting a city class from one system to another one for example.

You can also use it when you want to create a reusable class, the adapter, that works with classes that don't have compatible interfaces.

Lastly, we use it when you need to use several existing subclasses, but you don't want to create additional subclasses for each of them, but still need to adapt their interface. In that case, you can use the object adapter pattern to adapt the interface of its parent class.

Pattern Consequences

As every pattern, this one has a few consequences. It will be focused on the object adapter pattern here, as a true class adapter with multiple inheritance isn't possible in C#.

One of the consequences is that it lets a single adapter work with many adaptees, the adaptee itself and all of its subclasses as you can cast to those. The adapter can also add functionality to all adaptees at once as calls to the external system, the adaptee, pass through the adapter.

The single responsibility principle from the SOLID coding principles is adhered to in the way that you are separating the interface, the adapter code, out from the rest of the code. The adapter thus has a single responsibility.

As we are working on interfaces, new types of adapters can easily be introduced without breaking client code. That is the open/closed principle at work.

The object adapter makes it harder to override adaptee behavior. If you want to do that, you will have to subclass the adaptee and make the adapter refer to the subclass rather than the adaptee itself.

Another potential disadvantage, as is often the case, is that additional complexity is introduced because you introduced additional interfaces and classes.

Related patterns

As usual, a few patterns are related to the adapter pattern.

From the family of structural patterns, there is a relation to the bridge. Both have a similar structure, but a different intent. A bridge is meant to separate an interface from its implementation so they can be varied easily and independently. An adapter, on the other hand, is meant to change the interface of an existing object.

It is also related to the decorator pattern. The decorator changes an object without changing its interface. It is just more transparent than the adapter, which changes the interface of an existing object.

It is also related to the facade pattern. With the facade pattern, you are defining a new interface for an entire subsystem of objects. With adapter, you are making an existing interface usable by wrapping one object.

Lastly, there is a relation to the proxy pattern. This defines a surrogate for another object. It does not change its interface, while the adapter does provide a different interface.

- Bridge

Bridge separates interface from implementation, adapter changes the interface of an existing object.

- Decorator

Decorator changes an object without changing its interface, adapter changes the interface of an existing object.

- Facade

With facade you define a new interface for an entire subsystem, with adapter you are making an existing interface useable via wrapping

- Proxy

Defines a surrogate for another object, but does not change its interface.

Structural Pattern: Bridge

Monday, October 31, 2022 1:44 PM

Coming up

In this module we will cover the bridge pattern, another common one. The intent is to decouple an abstraction from its implementation so the two can vary independently.

First, bridge pattern will be described, what it is, and which problems it solves. For that it will be looked into a real-life scenario in the context of a cash register. From that real-life example, we will learn about the pattern structure and we will implement it. It will be shown use cases for this pattern and pattern consequences, and we will end the module with related patterns.

Describing the Bridge Pattern

The bridge pattern is a commonly used structural pattern from the well-known Gang of Four design patterns.

They describe the intent of this pattern as to decouple an abstraction from its implementation so the two can vary independently, in other words, be developed, be changed, independently. As with all these intent definitions, this requires a bit more explanation.

You could rephrase this as this pattern aims to separate an abstraction of the class from the implementation. As a result, this provides the means to replace an implementation with another implementation without modifying the abstraction.

Abstraction in this context can be seen as a way to simplify something complex. Take for example, a console app that wants to write something. `Console.WriteLine()` is an abstraction to whatever is contained within the `WriteLine` method. So every action and function, for example, are abstractions. We are constantly using and creating them.

Abstractions handle complexity by hiding the parts that we don't need to know about. And this abstraction is what we are trying to decouple from its implementation.

When I hear things like separate abstraction from implementation, "I'm" immediately thinking about interfaces and abstract classes, yet it is a bit more complicated than that.

Let's clarify this with a real-world example. Say we are creating a software for managing a restaurant register. There is two menus available, a meat-based menu and a vegetarian menu. Both menu implementation expose a method, `CalculatePrice`. Depending on the menu, other fields are available, but those are not of interest right now. The class that needs to add up the total amount owed can simply call into `CalculatePrice` on each menu.

```
public class VegetarianMenu
{
    public int CalculatePrice() { ... }
}

public class MeatBasedMenu
{
    public int CalculatePrice() { ... }
}
```

We could implement this with an abstract base `Menu` class or with an `IMenu` interface. Now let's imagine we are introducing something new a coupon, one that offers a €1 discount, another one that

offers a €2 discount, and so on. These affect the price of both our menus, obviously.

```
public abstract class Menu
{
    public abstract int CalculatePrice();
}

public class VegetarianMenu : Menu
{
    public override int CalculatePrice() { ... }
}

public class MeatBasedMenu : Menu
{
    public override int CalculatePrice() { ... }
}
```

How do we implement that? Well, one way of doing that might be to create additional menu specializations, deriving from MeatBasedMenu, MeatBasedMenuWithOneEuroCoupon, MeatBasedMenuWithTwoEuroCoupon, and two more deriving from VegetarianMenu, VegetarianMenuWithOneEuroCoupon, VegetarianMenuWithTwoEuroCoupon.

```
public abstract class Menu
{
    public abstract int CalculatePrice();
}

public class VegetarianMenu : Menu
{
    public override int CalculatePrice() { ... }
}

public class VegetarianMenuWithOneEuroCoupon : VegetarianMenu { ... }

public class VegetarianMenuWithTwoEuroCoupon : VegetarianMenu { ... }

public class MeatBasedMenu : Menu
{
    public override int CalculatePrice() { ... }
}

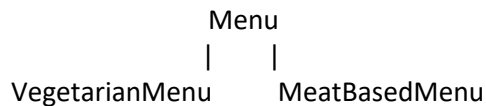
public class MeatBasedMenuWithOneEuroCoupon : MeatBasedMenu { ... }

public class MeatBasedMenuWithTwoEuroCoupon : MeatBasedMenu { ... }
```

This is not ideal. Imagine that another menu is added, Vegan Menu. we have to create additional subclasses for that as well. Or imagine another coupon value being added. We would need a new subclass for each menu. (Subclassing is one way to add functionality). So as you can imagine approaching it like this, by adding subclasses, leads to many additional classes and complexity. That is unnecessary and something we want to avoid, and that is where the bridge pattern comes in.

We are going to put the abstraction, which is MeatBasedMenu and VegetarianMenu, i.e., the ones based on the Menu abstract base class, or the IMenu interface, in one hierarchy. The abstract base class

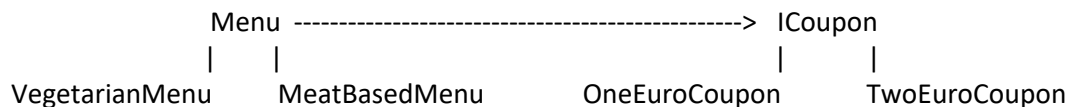
contains the definition of CalculatePrice and MeatBasedMenu and VegetarianMenu implements it, but to be able to calculate the price, our menus need to know the coupon value.



These are named implementers, which we can put in a separate hierarchy. So we will have a Coupon abstract base class or an ICoupon interface which states that a coupon value must be exposed. And now we have implementations of it, OneEuroCoupon and TwoEuroCoupon, who expose the actual coupon value.



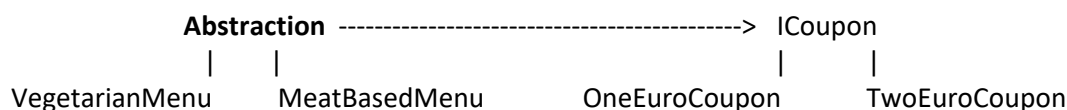
Each menu then needs to access to a coupon, on which it can calculate the coupon amount when calculating its price, and we can do that with a bit of composition. The abstract base Menu class will require an abstract coupon object, or something that implements the ICoupon interface. As that states that there should be a coupon value that can be accessed, the correct price can be calculated. That relationship between the menu and the coupon, that is what is named to bridge, as it bridges the abstraction and implementation, letting them vary independently.



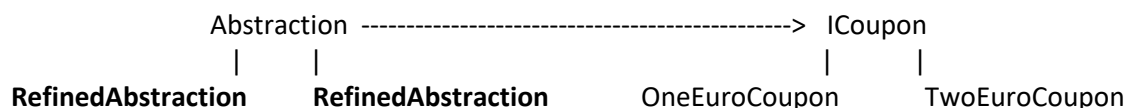
From this real-world example, we can now extract the structure of this pattern.

Bridge Pattern Structure

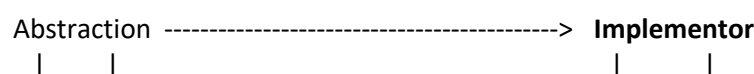
The abstract Menu class structurally is called an abstraction as it hides the complexity of getting the coupon value. The abstraction does define the abstraction's interface. It holds a reference to an implementor. So we switch out our Menu class for abstraction.



A concrete implementation of this, like our VegetarianMenu and MeatBasedMenu classes, is named RefinedAbstraction. This extends the interface defined by Abstraction.

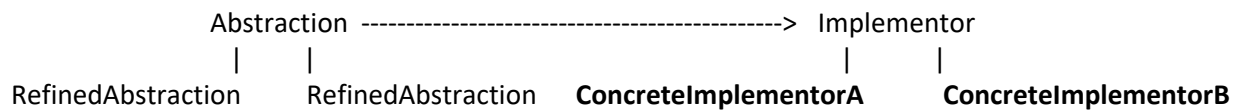


We can then implement getting the coupon value as a property we can get on a base coupon object or interface or as a method. But no matter how we do that, the fact remains that the implementation is hidden. The coupon base is then the implementor. This defines the interface for concrete implementations. The interface doesn't need to correspond to the Abstractions interface. In fact, most of the time, it is very different.



RefinedAbstraction RefinedAbstraction OneEuroCoupon TwoEuroCoupon

Classes like OneEuroCoupon and TwoEuroCoupon are concrete implementations of that implementor interface. A concreteImplementor implements the Implementor interface and defines its concrete implementation.



Like that we now know how the bridge pattern is structured.

Demo - Implementing the Bridge pattern

Now, it is time to learn how to implement the bridge pattern. We are going to start by adding our abstraction base, the menu. We will use an abstract class for that, which we will use as a base class for the refined abstractions. We give it one method, CalculatePrice, which is an abstract method as it will have to be implemented in the classes that implement this menu class.

In implementation.cs

```
public abstract class Menu
{
    public abstract int CalculatePrice();
}
```

As we know this menu class requires access to an operation implemented by an implementer. We are going to expose that via a Coupon object. This can be a simple abstract class exposing one property getter to get the value. These are the implementation of the operation that will effectively be implemented on the classes deriving from this abstract coupon class.

Below Menu {...}:

```
public abstract class Coupon
{
    public abstract int CouponValue { get; }
}
```

Note, that this can also be an interface. In fact, let's make it an interface as it doesn't contain any implementation details. So we change it to interface and name it ICoupon instead, and it will require one integer, a CouponValue.

Change the Coupon class:

```
public interface ICoupon
{
    int CouponValue { get; }
}
```

Then let's add some concrete implementations. We will start with NoCoupon. And add the two other coupons, one for €1 discount and one for €2 discount.

Below ICoupon:

```
public class NoCoupon : ICoupon
{
    public int CouponValue { get => 0; }
}
```

```
public class OneEuroCoupon : ICoupon
{
    public int CouponValue { get => 1; }
}
```

```
public class TwoEuroCoupon : ICoupon
{
    public int CouponValue { get => 2; }
}
```

Back to our abstraction. Here we can now add that coupon as a read only field. We don't want to see this changed after the menu has been constructed. We also add a constructor here that accepts such a coupon and sets the field. Combined with the read-only modifier, this ensures that our Menu class always has a non-null coupon value, and that is why it is written null! If we don't do that we will see a warning in classes that derive from the Menu class, and that is because our compiler cannot be sure of the fact that this will not be null, even though we are. By using the ! operator, we can convey that.

Change Menu:

```
public abstract class Menu
{
    public readonly ICoupon _coupon = null!;

    public abstract int CalculatePrice();

    public Menu(ICoupon coupon)
    {
        _coupon = coupon;
    }
}
```

And now let's create our two menu types. These are named refinedAbstractions. We start with the VegetarianMenu. Via the constructor we call the base constructor ensuring that the coupon value is set. So like this, we ensure that coupon is not null. In the CalculatePrice method, we use the CouponValue getter, which is the actual operation implementation from the pattern structure diagram. We use that to get the actual CouponValue which we deduct from the standard menu price, and then we do the same for the MeatBasedMenu.

Below menu:

```
public class VegetarianMenu : Menu
{
    public VegetarianMenu(ICoupon coupon) : base(coupon){}

    public override int CalculatePrice()
    {
        return 20 - _coupon.CouponValue;
    }
}

public class MeatBasedMenu : Menu
{
    public MeatBasedMenu(ICoupon coupon) : base(coupon){}
```

```

    public override int CalculatePrice()
    {
        return 30 - _coupon.CouponValue;
    }
}

```

Now let's test this. First we need to create a few new instances of coupons, and then we create new instances of a MeatBaseMenu passing through one of our coupons, the noCoupon coupon. We write out the price, then we passed through a oneEuroCoupon for that, we have to make a new instance again and we write out the price. We can do the same for vegetarianMenu. Note that we can safely pass through the same coupon instances we used for the meatBasedMenu.

In Program.cs

```

var noCoupon = new NoCoupon(); //Quick Action to insert using statement
var oneEuroCoupon = new OneEuroCoupon();

var meatBasedMenu = new MeatBasedMenu(noCoupon);
Console.WriteLine($"Meat based menu, no coupon: {meatBasedMenu.CalculatePrice()} euro.");

meatBasedMenu = new MeatBasedMenu(oneEuroCoupon);
Console.WriteLine($"Meat based menu, coupon: {meatBasedMenu.CalculatePrice()} euro.");

var vegetarianMenu = new VegetarianMenu(noCoupon);
Console.WriteLine($"Vegetarian menu, no coupon: {meatBasedMenu.CalculatePrice()} euro.");

vegetarianMenu = new VegetarianMenu(oneEuroCoupon);
Console.WriteLine($"Vegetarian menu, coupon: {meatBasedMenu.CalculatePrice()} euro.");

Console.ReadKey();

```

Now we can test this. We can see that our coupons are added/subtracted from the price. Thanks to the bridge pattern it is now very easy to add a new menu or a new coupon. And will no longer result in an explosion of classes.

Use Cases for the Bridge pattern

It is quite a few use cases for the bridge pattern. There is a reason for this pattern is very common. A few of the more common ones are, first of all, when you want to avoid a permanent binding between an abstraction and its implementation, for example when the implementation must be selected at runtime. Our coupon example showed that. In other words, the bridge pattern enables switching implementations at runtime.

Then, use it when abstraction and implementations should be extensible by subclassing. The bridge pattern allows combining the different abstractions and implementations and extends them independently.

Another good use case is for when you don't want changes in the implementation of an abstraction have an impact on the client. As far as the client is concerned, this should be completely transparent thanks to loose coupling

Pattern Consequences

Let's start with the first consequence, decoupling. The implementation isn't permanently bound to the abstraction. It can be configured and potentially even changed at runtime. Because of this decoupling, layering is encouraged, which typically results in a system that is better structured.

Another one is extensibility being improved. As the abstraction and implementation hierarchies are separate, they can evolve and be extended independently. You can then introduce new abstractions and implementations independently of each other, which is the open/closed principle at work.

Three, you can hide implementation details away from clients, which is a consequence of abstraction. The pattern also allows focusing on high-level logic in the abstraction and on the details in the implementation. By doing so, we can adhere to the single responsibility principle.

Related patterns

The bridge pattern is related to the abstract factory pattern from the family of creational patterns. Such a factory can create and configure a bridge, which hides complexity away from client code from the family of structural patterns their relation to the adapter. That pattern is used for making unrelated classes work together.

The adapter adapts the adaptee for that. The bridge pattern, as we learned, is used to let abstractions and implementations vary independently.

Strategy, Bridge, and State are all related and look similar because they are based on composition, even though they solve different problems.

- Abstract factory
Factory can create and configure a bridge
- Adapter
Adapter lets unrelated classes work together, bridge lets abstractions and implementation vary independently
- Strategy
Based on composition, like bridge
- State
Based on composition, like bridge

Structural Pattern: Decorator

Monday, October 31, 2022 1:44 PM

Coming up

In this module we will learn about the Decorator pattern. This pattern allows adding behavior dynamically to an individual object without affecting the behavior of other instances of the same class. It is sometimes referred to as a wrapper, which is something the adapter is also sometimes referred to as.

We will start this module by describing the Decorator pattern, where we learn what it is and which problems it solves. The real-life scenario we will cover while doing so is related to collecting statistics and storing a list of sent email messages. From that real-life sample, we will learn about the pattern structure, and we will implement it. After that, we will cover the pattern use cases, and consequences, and finish up with related patterns.

Describing the Decorator Pattern

The decorator pattern is a commonly used structural pattern from the well-known Gang of Four design patterns. The intent of the pattern is to attach additional responsibilities to an object dynamically. A decorator then provides a flexible alternative to subclassing for extending functionality. Just like the adapter pattern, this pattern is sometimes called a wrapper.

Attaching additional responsibility or behavior to a class can easily be done by simply adding an additional method to the class. In this case, however, we need a way to attach additional responsibilities to an object and not to a class. We want to allow this dynamically, so at runtime. That is a bit more complicated, but not that much really. The fact that this pattern is sometimes called a wrapper should already give you a good idea of how something like this can be implemented.

Let's clarify this with a real-life example.

In one of the other courses given by Kevin, he use a CloudMailService and a regular OnPremiseMailService to send mail.

```
public class CloudMailService
{
    public void SendMail() { ... }
}
```

```
public class OnPremiseMailService
{
    public void SendMail() { ... }
}
```

Both of these implement a simple IMailService interface, which exposes one method, SendMail.

```
public interface IMailService
{
    void SendMail();
}
```

```
public class CloudMailService : IMailService
{
    public void SendMail() { ... }
}
```

```
public class OnPremiseMailService : IMailService
{
    public void SendMail() { ... }
}
```

Now imagine that we will want to add some functionality to this service, say the wish is to start collecting some statistics. For

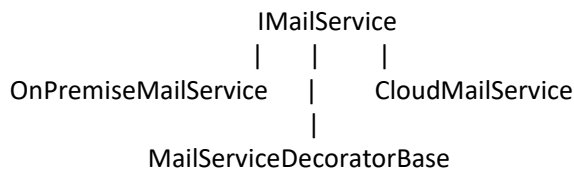
example, how long does it take sending a mail, or how often is the SendMail method accessed? It could be added functionality to the different mail service implementations, but that violates the single responsibility principle for that class. Moreover, imagine that I want to add even more responsibilities, like logging when a mail is being sent or storing all the sent mail messages in a database somewhere. Our classes would get littered with code that doesn't belong there, and that is where the decorator pattern comes in.

```
public interface IMailService
{
    void SendMail();
}

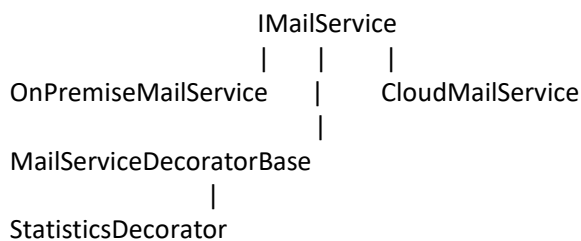
public class CloudMailService : IMailService
{
    public void SendMail() { //Additional functionality }
}

public class OnPremiseMailService : IMailService
{
    public void SendMail() { //Additional functionality }
}
```

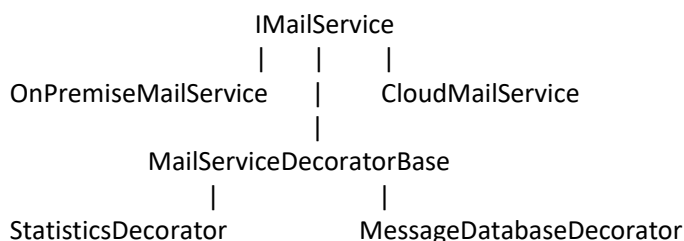
Let's start with our IMailService and both implementing classes. We are going to create a wrapper around our IMailService to add the functionality we need by overriding the SendMail method in that wrapper. We can call the wrapper MailServiceDecoratorBase, for example. It is an abstract base class, which also implements the IMailService interface, thus exposing the SendEmail method.



Next to that, we also store the IMailService instance we want to decorate in this decorator. It is now up to concrete implementations of the decorator to effectively add functionality. We could have a StatisticsDecorator that derives from MailServiceDecoratorBase, calculating some statistics, and then calls into the sendMailMethod.



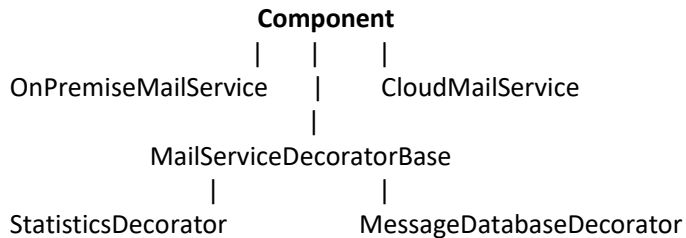
We could also have a MessageDatabaseDecorator, which stores all the messages that are being sent in a database and so on. Instead of directly calling, for example, CloudMailService.SendMail, we can now call StatisticsDecorator.SendMail or MessageDatabaseDecorator.SendMail.



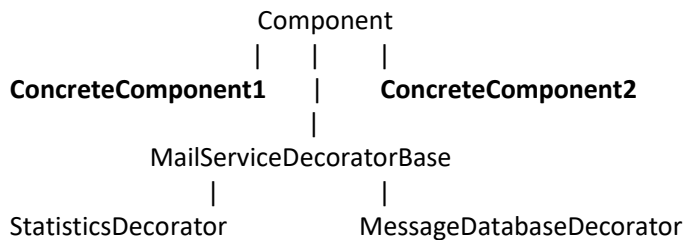
So with a few simple interventions, we can easily extend the functionality of the mail service objects without having to resort to an explosion of subclasses or large inheritance trees and while, at the same time, sticking to the single responsibility principle. From this real-world example, we can now extract the structure of this pattern.

Decorator Pattern Structure

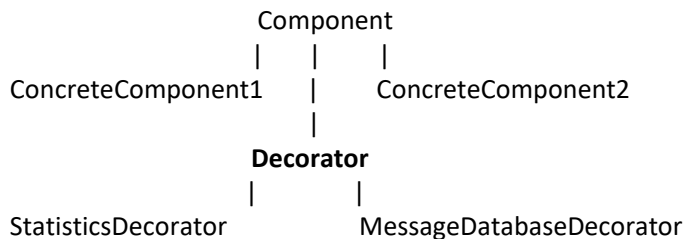
The IMailService interface structurally is called a component. The component defines the interface for objects that can have responsibilities added to them dynamically.



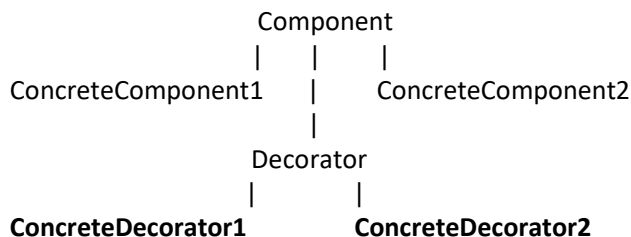
Concrete implementation like our **CloudMailService** and **OnPremiseMailService** classes are called concrete components, **ConcreteComponent1** and **ConcreteComponent2**, for example. They define an object to which additional responsibilities can be attached.



The other implementation of that IMailService interface or of that component, in this case, is structurally called the decorator. That is our abstract mail service decorator base class. It intercepts, if you want, the **SendMail** operation, thus allowing us to extend the functionality of the object. It maintains a reference to a component object and defines an interface that conforms to the component's interface.



Concrete implementations like **statisticsDecorator** and **MessageDatabaseDecorator**, which contain added behavior like storing statistics and/or added state like a mail message store are called concrete decorators, for example, **ConcreteDecorator1** and **ConcreteDecorator2**. They are responsible for adding responsibilities to the component.



And like we know how the decorator pattern is structured.

Demo - Implementing the Decorator pattern

In this module we will learn how to implement the decorator pattern. Let's start by adding our component, the `IMailService` interface. Note that as with many of these patterns, this can also be implemented with an abstract class instead of an interface. Our interface contains one method, `SendMail`, accepting the message to send. It returns a `Boolean`, so we know whether sending the mail was successful or not.

In `Implementation.cs`

```
public interface IMailService
{
    bool SendMail(string message);
}
```

Then we can create our concrete components. We do that by implementing the interface. In the first concrete component implementation, named `CloudMailService`, we fake sending the mail via a cloud service, and we do something likewise with an `OnPremiseMailService`, another concrete component implementation.

Below `IMailService`:

```
public class CloudMailService : IMailService
{
    public bool SendMail(string message)
    {
        Console.WriteLine($"Message \"{message}\" " +
            $"sent via {nameof(CloudMailService)}.");
        return true;
    }
}

public class OnPremiseMailService : IMailService
{
    public bool SendMail(string message)
    {
        Console.WriteLine($"Message \"{message}\" " +
            $"sent via {nameof(OnPremiseMailService)}.");
        return true;
    }
}
```

So far, so good. Up next is the decorator itself. Let's add a new class, `MailServiceDecoratorBase`. This will become our decorator. As we learned on the slides, it should implement the `IMailService` interface. Like that, we ensure that the same method, `SendMail`, must be implemented. First, we ensure that this class can only be constructed with the `IMailService` implementing object being passed in. That is the object we want to add functionality to. We store that in a private read-only field. We do still need to call `SendMail` on it, and that is what we do in the `SendMail` method on our `MailServiceDecoratorBase`. And we return the result, but this isn't done yet.

This class doesn't really do anything yet. It wraps the `IMailService`, but it doesn't add functionality to the `mailService`. This serves as an abstract class to store the `IMailService` instance. You could name this a base wrapper. We don't want it to be used on its own, so we mark it as abstract. So we make the class abstract. And we want to make sure we mark the `SendMail` method as virtual so implementing classes can effectively override this and add functionality. And that is it for this class.

Below `OnPremiseMailService`

```
public abstract class MailServiceDecoratorBase : IMailService
{
    private readonly IMailService _mailService;

    public MailServiceDecoratorBase(IMailService mailService)
    {

```

```

        _mailService = mailService;
    }

    public virtual bool SendMail(string message)
    {
        return _mailService.SendMail(message);
    }
}

```

Now let's add the concrete decorator implementations.

The first one is a StatisticsDecorator. It implements the MailServiceDecoratorBase abstract class, which ensures that we get a constructor generated. We call into that base constructor from the StatisticsDecorator constructor. Like that, our mailService is nicely stored in the private field. We also override the SendMail method, we collect some statistics, and we send the mail. We have to call base.SendMail for that, as the actual implementation of sending that mail is at that level.

Onto our second concrete decorator, our MessageDatabaseDecorator. Likewise, it implements the MailServiceDecoratorBase. It stores a list of SentMessages, acting as a data store for messages sent through this decorator. In the SendMail method, we store the message if it is been successfully sent. And that is it for our pattern implementation.

Below MailServiceDecoratorBase:

```

public class StatisticsDecorator : MailServiceDecoratorBase
{
    public StatisticsDecorator(IMailService mailService) : base(mailService){}

    public override bool SendMail(string message)
    {
        // Fake collecting statistics
        Console.WriteLine($"Collecting statistics in {nameof(StatisticsDecorator)}.");
        return base.SendMail(message);
    }
}

```

```

public class MessageDatabaseDecorator : MailServiceDecoratorBase
{
    public List<string> SentMessages { get; private set; } = new List<string>();

    public MessageDatabaseDecorator(IMailService mailService) : base(mailService){}

    public override bool SendMail(string message)
    {
        if(base.SendMail(message))
        {
            // Store sent message
            SentMessages.Add(message);
            return true;
        };
        return false;
    }
}

```

Let's test this. The first thing we want to do is instantiate our MailServices. For that, we need to import the Decorator namespace. So we add a new instance of a cloudMailService and an onPremiseMailService, and for testing purposes, we have sent a mail through them. That is the default behavior of these MailServices. what we want to do now is add behavior to it to collect statistics. To do that, we decorate the cloudMailService, for example, with the StatisticsDecorator, and then we send the mail through that StatisticsDecorator. This should show us some added behavior, in other words, some added functionality. And then we also add behavior to the OnPremiseMailService. As you can see, this works in a likewise manner.

The decorators work on the MailService interface and not on the concrete implementations. So we send a few messages through the MessageDatabaseDecorator, and then we run through the stored messages to see whether or not our decorator is successfully executed. Then we add a ReadKey statement at the bottom.

```
In Program.cs
using Decorator;

//instantiate mail services
var cloudMailService = new CloudMailService();
cloudMailService.SendMail("Hi there.");

var onPremiseMailService = new OnPremiseMailService();
onPremiseMailService.SendMail("Hi there!");

// add behavior
var statisticsDecorator = new StatisticsDecorator(cloudMailService);
statisticsDecorator.SendMail($"Hi there via {nameof(StatisticsDecorator)} wrapper.");

var messageDatabaseDecorator = new MessageDatabaseDecorator(onPremiseMailService);
messageDatabaseDecorator.SendMail(
    $"Hi there via {nameof(MessageDatabaseDecorator)} wrapper, message 1.");
messageDatabaseDecorator.SendMail(
    $"Hi there via {nameof(MessageDatabaseDecorator)} wrapper, message 2.");

foreach (var message in messageDatabaseDecorator.SentMessages)
{
    Console.WriteLine($"Stored message: \"{message}\"");
}

Console.ReadKey();
```

Let's test this. The first thing we see are the two messages we sent via the SendMail method on the services themselves. Then we decorated one of them with the StatisticsDecorator, and that is what we see next, Collecting statistics in StatisticsDecorator, so we pass through that. The next line is the message passed through to the cloudMailService from the StatisticsDecorator, "Hi there via StatisticsDecorator wrapper. Up next, we used the MessageDatabaseDecorator to send messages via the OnPremiseMailService. We did that twice, which is what is shown. After that, we ran through the stored messages, because the MessageDatabaseDecorator stores the messages that are sent through it, and we indeed see our two messages being sent. With that we successfully implemented a decorator.

Use Cases for the Decorator pattern

The first good use case for this pattern is when you have a need to add responsibilities to individual objects dynamically without affecting other objects. The example we just covered was, thus, a good use case.

Another good use case for this pattern is when you need to be able to withdraw responsibilities you attached to an object. Decorators can be applied dynamically, and thus, also removed dynamically. That is an advantage when, for example, you want to switch out storing mail messages in the database to another user store. Simply remove the message database decorator and, if needed, replace it.

Lastly we can use this pattern when extension by subclassing is impractical or impossible, for example, when the class is sealed.

Pattern Consequences

The first consequence of this pattern is that it is more flexible than using static inheritance via subclassing. Responsibilities can be added and removed at runtime ad hoc when needed. If you compare that to subclassing, which leads to increased complexity of your system when many different behaviors must be added, the decorator pattern is definitely preferable.

The second consequence is that it avoids feature-loaded classes that don't respect the single responsibility principle. With this pattern, you can split that class up until there is just one responsibility per class, thus adhering to the single responsibility principle from the SOLID coding principles.

The third consequence is that, this pattern does tend to litter your system with lots of small, simple classes. That is a disadvantage that potentially increases the effort required to learn and debug the system.

Related patterns

A few patterns are related to the decorator pattern. From the family of structural patterns, there is relation to the adapter pattern, but the adapter pattern gives a completely new interface to an object while the decorator pattern only changes its responsibilities.

Another related structural pattern is the composite pattern. A decorator can be viewed as a composite with only one component. That being said, the decorator isn't intended for object aggregation while the composite pattern is.

Lastly, a behavioral pattern, the strategy pattern. While the decorator pattern lets you change the skin, which is the outer side of an object, the strategy pattern lets you change the inner workings. So, both allow you to change an object, but in a different way.

- Adapter
Adapter gives a new interface to an object, decorator only changes its responsibilities.
- Composite
Decorator can be seen as a composite with only one component.
- Strategy
Decorator lets you change the skin of an object, strategy lets you change its inner workings.

Structural Pattern: Composite

Monday, October 31, 2022 1:44 PM

Coming up

In this module, we will cover the composite pattern. That is a pattern that takes advantage of recursion and is often used when your data resembles a tree-like structure.

We will start by describing the pattern using a real-life example. As the real-life example, we will use a file system scenario. After that we will map that to a structural example, and we will implement the real-life example in a demo. We will continue with learning about use cases for this pattern and pattern consequences, which can be positive or negative. We will end the module by looking into related patterns.

Describing the Composite Pattern

The composite pattern is a structural pattern from the well-known Gang of Four design patterns. Its intent is to compose objects into tree structures to represent part-whole hierarchies. The composite pattern lets clients treat individual objects and compositions of objects uniformly, as if they all were individual objects.

What would this translate to in real life? Well, composing objects into tree-like structures is something we are all very much used to. Imagine the structure of an XML document, where each element, in turn, consist of other elements, which can again consist of elements. Another example might be an application in which a user can draw. You can represent that as something that consists of one or more lines and potentially one or more pictures, which in turn consists of lines and pictures too, and so on.

The example we are going to work with is another one you can undoubtedly immediately picture, the file system.

We have essentially got items in there, lets name them `FileSystemItems`, that are either a file or a directory. Such a directory can consist of one or more files and/or other directories, and so on. So there is our tree structure already. We have got two classes in our tree structure, a file and a directory. Now imagine we want to get the size of all the items in our tree structure. We would have to run through all these objects. We need to know the exact classes, `File` and `Directory`. We need to know the method to call on them to get their size. We need to know how deep nesting goes to be able to access those levels of files and directories and so on. This can become very cumbersome, very fast, and that is what the composite pattern helps with.

Composite pattern:

Intrinsic knowledge about the object type is required.

```
public class File { }
```

```
public class Directory
{
    // other directories and/or files
}
```

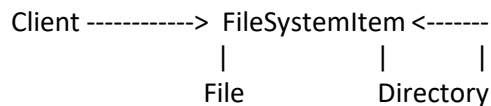
```
public class Client
{
    // Calculate the size of a tree of Directory / File
}
```

}

What if, instead of working with Directory and File classes, we'd work with the base FileSystemItem class, a common interface through which we can calculate the total size? Let's put that into practice. On the FileSystemItem base class, we define a method to calculate its size, GetSize. This is implemented by our File class and by our Directory class.

The directory class, next to that, also stores its children and includes methods to add additional FileSystemItems, so the tree can be built. On the file class itself, we don't need that, as such a file cannot have children, only a directory can. The other common thing we want to do is define a common operation on the FileSystemItem, a method to get the size of it.

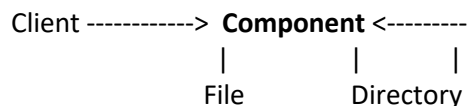
In case of a single file, implementing this is simply a matter of returning the actual size, but in case of a directory, that method would count the file size of the directory itself and then run over all FileSystemItems in its FileSystemItem collection, get the size for each and add them up, and like that, it continues. That is a very nice example of recursiveness. All of this is transparent for the client. It is the FileSystemItem itself that will handle counting the size of all the items in its tree structure. This is a lot more convenient to work with.



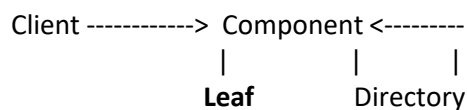
Composite Pattern Structure

Our base object, that is a FileSystemItem, is what maps to a component. This can be implemented as an interface or an abstract base class. This component, this declares the interface for objects in the composition and potentially implements default behavior, which is possible if you implement it as an abstract class. It also contains a common operation.

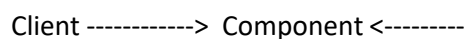
That common operation in our example was the method to get the FileSystemItem size. It can also potentially define an interface for accessing and managing its children. This was obligatory in the initial design pattern template, but in more current iterations of the pattern, this type of behavior is often defined and implemented on the composite itself instead of on the base class. We will get to that.



One item would be a leaf, in our case, that is the File class. This one does not have children. It derives from the component, and it implements the common operation. "Leaf represent leaf objects in the composition, and has no children. It defines behavior for primitive objects in the composition." So behavior for the primitive objects in the composition is the responsibility of the leaf.

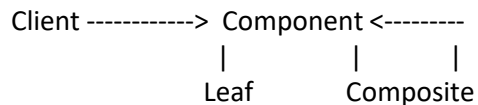


Another item that would be a composite. This one also derives from the component base, and it stores a set of components, children. It implements the common operations and defines operations to add or remove children. It does define behavior for components having children.





And lastly, there is the client, of course. This one manipulates objects in the composition through the component interface. In the structure, as its detailed in the Gang of Four design patterns book, the component abstract base class also defines an operation to get a specific child by index. But as mentioned before, you should not see that as a literal must to have such an operation. Instead, see it as a reminder that you can add that type of functionality if you need it. In the same spirit, we have varied this a bit. In the original implementation, the add and remove operation were defined on the abstract base class and not overridden on the leaf class. We simply didn't define among the base class and instead added them to the composite class. So this just goes to show that there are multiple ways of implementing a pattern.



Demo - Implementing the Composite pattern

Now we will implement our example. In this demo we will implement the composite pattern using the real-life example we just described. We will start with the abstract base class, the `FileSystemItem`. This is the component. We give it a name and define an abstract method on it to get its size. We also add a constructor that accepts the name, signifying we expect consumers to pass through a name when constructing the item. This is an abstract class so it cannot be used on its own.

In `Implementation.cs`

```

public abstract class FileSystemItem
{
    public string Name { get; set; }
    public abstract long GetSize();

    public FileSystemItem(string name)
    {
        Name = name;
    }
}
  
```

The next step this is implementing this class. Our first implementation is the leaf. In our case, that is a file. Next to the name, we also expect a size this time, so we had a constructor that accepts that size and stores it in a private field. Implementing the `GetSize` method is then simply a matter of returning the size variable.

Below `FileSystemItem{}`:

```

public class File : FileSystemItem
{
    private long _size;

    public File(string name, long size) : base(name)
    {
        _size = size;
    }

    public override long GetSize()
  
```

```

    {
        return _size;
    }
}

```

The directory from our example is the composite, and it too derives from `FileSystemItem` and requires a size. So we have a likewise constructor as we had for the file yet we haven't implemented `FileSystemItem` yet.

When we do that, we see that we need to provide an implementation for the `GetSize` method, and that is a bit more complicated in this case. As the directory is a composite it can contain other components, `FileSystem` in our case. So we need a variable to store these in. We make sure this one is initialized to avoid null reference exceptions. We also wanted to add and remove functionality. There is multiple ways of doing this.

We could simply expose our private list `FileSystemItems`, but this would break encapsulation. It would allow other classes to directly manipulate this list. Whether or not you want to do that depends on how much your use case requires you to shield this list. For our case, we are going to make encapsulation as strict as possible. We actually don't need this list to have a property getter and setter.

So we add the two methods to add and remove `FileSystemItems` by manipulating the `FileSystemItems` list. Lastly, we can now implement the `GetSize` operation. This shouldn't just return the size value. It should return the sum of the current size value and that of each size of each `FileSystemItem`. In turn, one of those could be a directory, which would result in the size getting calculated by running through all of its children and so on. So that is a very nice example of a recursive method at work. From inside of the `GetSize` method, we call `GetSize` on each `FileSystemItem`. And that is it for our implementation.

Below File {}:

```

public class Directory : FileSystemItem
{
    private List<FileSystemItem> _fileSystemItems {get;set;} = new List<FileSystemItem>();

    private long _size;

    public Directory(string name, long size) : base(name)
    {
        _size = size;
    }

    public void Add(FileSystemItem itemToAdd)
    {
        _fileSystemItems.Add(itemToAdd);
    }

    public void Remove(FileSystemItem itemToRemove)
    {
        _fileSystemItems.Remove(itemToRemove);
    }

    public override long GetSize()
    {
        var treeSize = _size;
        foreach (var fileSystemItem in _fileSystemItems)

```



```

        {
            treeSize += fileSystemItem.GetSize();
        }
        return treeSize;
    }
}

```

To test this. The first thing we need to do is create our tree structure. So, let's create a few directories and files. We start with the root, which we give no specific value for size. To that we want to add one file and two directories, so we make new instances of a file and two directories.

We give the file and these directories a size, and we add them to the root. We are not going to stop there. We will create a tree that is a bit bigger than this, so we will create two sublevel files as well. We make these instances and give them a size, and we add them to one of the top-level directories. Like that, we have something that is starting to resemble a nice tree hierarchy. Now let's test whether our GetSize method actually works.

So we write out to the console window the GetSize statement on the topLevelDirectory1, one on topLevelDirectory2, and one on the root itself, which should result in the full size of our tree. Interesting to notice here is that at level of the client we are getting the size. But as far as the client is concerned, interacting with the leaf is exactly the same as interacting with the component. We are calling GetSize on the directories here, but we might as well call GetSize on one of these files as well. This is completely transparent to the client.

In Program.cs

```

var root = new Composite.Directory("root", 0);
var topLevelFile = new Composite.File("toplevel.txt", 100);
var topLevelDirectory1 = new Composite.Directory("topleveldirectory1", 4);
var topLevelDirectory2 = new Composite.Directory("topleveldirectory2", 4);

root.Add(topLevelFile);
root.Add(topLevelDirectory1);
root.Add(topLevelDirectory2);

var subLevelFile1 = new Composite.File("sublevel1.txt", 200);
var subLevelFile2 = new Composite.File("sublevel2.txt", 150);

Console.WriteLine($"Size of topLevelDirectory1: {topLevelDirectory1.GetSize()}");
Console.WriteLine($"Size of topLevelDirectory2: {topLevelDirectory2.GetSize()}");
Console.WriteLine($"Size of root: {root.GetSize()}");
topLevelFile.GetSize();
Console.ReadKey();

```

Use Cases for the Composite pattern

This pattern can be useful when you want to represent part-whole hierarchies of objects, in other words, when you want to represent a tree-like object structure.

It is also something worth considering when you want clients to be able to ignore the difference between composition of objects and individual objects. As we have noticed in the example, as far as the client is concerned, it doesn't matter if the operation, GetSize in our example, is called on a leaf or a composite.

Pattern Consequences

This pattern has a few consequences. First of all, it makes the clients simple, as clients don't know whether they are dealing with a leaf or a composite component.

It is also easy to add new kinds of components. That is because newly defined subclasses automatically work with existing structures and client code, so clients don't have to be changed for new component classes. Imagine you'd have another type of `FileSystemItem`. You could simply implement it and work with it. The client doesn't care. It simply calls `GetSize` on the `FileSystemItem` base type, not on the subclass type. This is the open/closed principle from the SOLID coding principles at work.

A disadvantage is that it can make the overall system too generic. By making it easy to add and remove items to the composite, we also make it a bit harder to restrict the components of a composite. It is no longer the compiler that checks for this, as `add` and `remove` accept the abstract base type and not the subclass. Take our `FileSystemItem` example. If we want to restrict the directory component to only contain files and not subdirectories, we cannot rely on the compiler for that as `add` and `remove` accept the base `FileSystemItem`. We have to write additional code to check for the actual type at runtime.

Related patterns

There is quite a few related patterns to this one. From the behavioral pattern family, there is the chain of responsibility. The component parent link is often used for that. When the leaf component gets a request, it can potentially pass it through the chain of all parent components down to the root of the object tree.

It is also related to the iterator because an iterator can be used to traverse composites.

Lastly the visitor pattern can localize operations and behavior that would otherwise be distributed across composite and leaf classes. You can, thus use it to execute an operation over the full composite tree.

- Chain of responsibility
Leaf component can pass a request through a chain of all parent components.
- Iterator
Can be used to traverse composites.
- Visitor
Can be used to execute an operation over the full composite tree.

Structural Pattern: Facade

Monday, October 31, 2022 1:44 PM

Coming up

In this module we will cover the Facade pattern. It is intended to provide a higher-level interface over a subsystem to make the subsystem easier to use.

We will start by describing the pattern using a real-life example. As a real-life example, which we will also implement, we will use a discount calculator service. After having described the pattern, we will map it to a structural example. We will continue with learning about use cases for this pattern and pattern consequences and will end the module with related patterns.

Describing the Facade Pattern

The facade pattern is a commonly used structural pattern from the well-known Gang of Four design patterns. The intent of the facade pattern is to provide a unified interface to a set of interfaces in a subsystem. It defines a higher-level interface that makes the subsystem easier to use. Let's think about that by studying a real-life example.

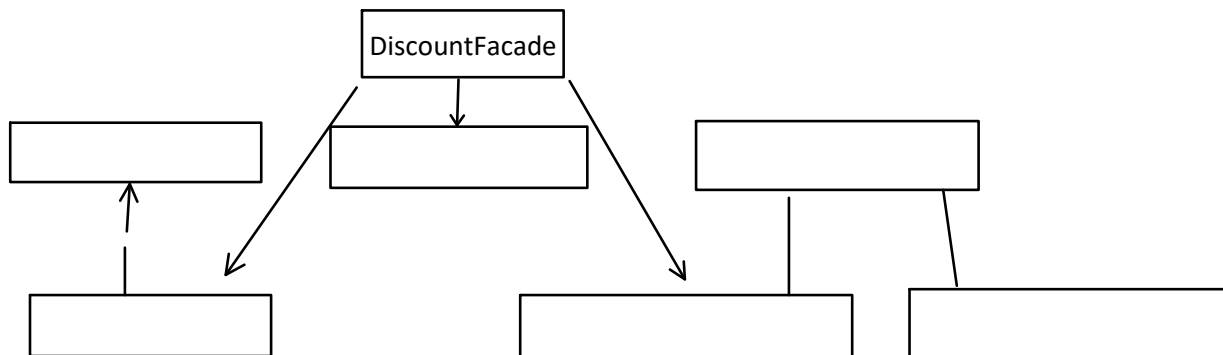
You are building a system in which you want to know the discount a customer will receive. However, calculating this can be quite complicated. First of all, we need to call into a service that checks whether the customer has enough orders. Then, we need to calculate the discount base percentage, which depends on the customer. For example, the percentage can be different depending on the country the customer is from. Lastly, a factor must be taken into account depending on the day of the week. This is something you sometimes see at airfare sites. This calculation thus isn't to reveal, and it has to be done in different parts of your application. Putting a facade in front of it hides away the complexity. So, it is an abstraction and encourages reuse.

A facade hides away the complexity of this calculation and encourage reuse.

```
OrderService orderService = new ();
CustomerDiscountBaseService customerDiscountBaseService = new ();
DayOfTheWeekFactorService dayOfTheWeekFactorService = new ();

if (!orderService.HasEnoughOrders(customerId)) {discount = 0;}
else
{
    discount = customerDiscountBaseService.CalculateDiscountBase(customerId) *
               dayOfTheWeekFactorService.CalculateDayOfTheWeekFactor();
}
```

We could name that DiscountFacade. It contains one method, CalculateDiscount, which is responsible for calling into the various services and calculating the actual discount percentage. So this is our real-life example.



Facade Pattern Structure

The structure of the facade pattern is pretty straight forward.

First of all, we have a facade that maps to our DiscountFacade. This facade has knowledge about the subsystem classes that are responsible for a request, and it delegates requests to the appropriate subsystem classes. That subsystem, that are all our other services or classes. The OrderService, CustomerDiscountBaseService, DayOfTheWeekFactorService, and more are all subsystems. They implement their own functionality as needed, and they handle the work assigned by the facade object. In OtherWords, methods called by the facade are implemented in a subsystem. The subsystem themselves don't have knowledge of the facade.

Demo - Implementing the Facade pattern

The first thing we want to do in this demo is creating the subsystem classes.

Our first subsystem class is an OrderService. It should check whether the customer has ordered enough in the past to be eligible for a discount. It has one method, HasEnoughOrders, which accepts a customerId. We do a fake calculation here for demo purposes. Any customer with an ID larger than 5 is assumed to have made enough orders.

In implementation.cs

```
public class OrderService
{
    public bool HasEnoughOrders(int customerId)
    {
        //Does the customer have enough orders?
        //Fake calculation for demo purposes
        return (customerId > 5);
    }
}
```

The second subsystem class is the CustomerDiscountBaseService, which is used to calculate the discount percentage base. These calculations can be quite extensive. You could take the country into account. You could take the amount of orders or total order price into account and so on. We will fake these two for demo purposes. If the ID of the customer is larger than 8, he or she gets a 10% discount, otherwise a 20% discount.

Below orderService{}:

```
public class CustomerDiscountBaseService
{
    public double CalculateDiscountBase(int customerId)
    {
        //Fake calculation for demo puposes
        return (customerId > 8) ? 10 : 20;
    }
}
```

The last subsystem class is the DayOfTheWeekFactorService. We use a switch statement on the day of the week, stating that during the weekend, the factor is 0.8 and otherwise it is 1.2 which means orders done during the weekend get discounted less while orders done on other days get discounted more. So these are our helper classes.

Below CustomerDiscountBaseService{}:

```
public class DayOfTheWeekFactorService
{
    public double CalculateDayOfTheWeekFactor()
```

```

    {
        //fake calculation for demo purpose
        switch (DateTime.UtcNow.DayOfWeek)
        {
            case DayOfWeek.Saturday :
            case DayOfWeek.Sunday :
                return 0.8;
            default :
                return 1.2;
        }
    }
}

```

Onto the facade itself. We will name it DiscountFacade. It is a new class, which should contain references to our subsystems. So, we add three references to the subsystems. We could inject these instance's file constructor injection, but in our case, we don't have an IOC container set up, so this time we will just make new instances. We could also add the constructor anyway and let the client class provide us with the subsystem class instances. Now we need to calculate the discount. We add one public method, CalculateDiscountPercentage, which accepts the customerId. First, we call into the orderService. If the customer doesn't have enough orders, we return + as the discount percentage. Otherwise, we multiply the discount base with the day of the week factor and return that value. And that is it for the facade. Our pattern is already done.

Below DayOfTheWeekFactorService{ }:

```

public class DiscountFacade
{
    private readonly OrderService _orderService = new();
    private readonly CustomerDiscountBaseService _customerDiscountBaseService = new();
    private readonly DayOfTheWeekFactorService _dayOfTheWeekFactorService = new();

    public double CalculateDiscountPercentage(int customerId)
    {
        if (!_orderService.HasEnoughOrders(customerId))
        {
            return 0;
        }
        return _customerDiscountBaseService.CalculateDiscountBase(customerId) *
            _dayOfTheWeekFactorService.CalculateDayOfTheWeekFactor();
    }
}

```

Let's test this. We create new instance of the facade object. For that, we need to add a using statement to Facade, and then we test our facade by trying to get a discount for customer with ID 1 and one for customer with ID 10. Let's give this a try. No discount for the customer with ID1 because of ID of this customer isn't larger than 5, customer with ID 10, on the other hand, does get a discount.

In Program.cs

using Facade;

```
var facade = new DiscountFacade();
```

```
Console.WriteLine($"Discount percentage for customer with id 1: " + $"{facade.CalculateDiscountPercentage(1)}");
```

```
Console.WriteLine($"Discount percentage for customer with id 10: " + $"{facade.CalculateDiscountPercentage(10)}");
```

```
Console.ReadKey();
```

Use Cases for the Facade pattern

The first use case for this pattern is obviously when you want to provide a simple interface into a complex subsystem consisting of a bunch of subsystem classes. In that case, when the subsystem becomes more complex, this has no effect on clients using the facade.

It is also useful to consider this pattern when there are many dependencies between a client and the implementation classes of the abstraction. By putting a facade in between, you decouple the subsystem from the clients.

Pattern Consequences

There are three main consequences to this pattern. First of all, it shields clients from subsystems. That means that the number of objects that clients have to deal with are reduced.

It also promotes weak coupling between the subsystem and its clients. The client is coupled to the facade, not to the subsystems per se. Thanks to that weak coupling, the subsystem components can vary without affecting the client. That is the open/closed principle from the SOLID coding principles at work.

And lastly, even though it shields clients from subsystems, clients are not forbidden to use the subsystem classes if they want to.

Related patterns

From the creational family, there is the abstract factory pattern that this one relates to. It can be used together with the facade to provide an interface for creating subsystem objects while remaining independent of the subsystem. Abstract factory can also be used as an alternative to facade to hide platform-specific classes.

Up next, from the behavioral family, the mediator pattern. Like facade, it abstracts functionality of existing classes, those subsystems in our example, but it is not the same. The purpose of the mediator is abstracting communication between objects, often centralizing functionality that doesn't belong in those objects. Facade on the other hand, simply abstracts the interface to the subsystem objects to promote ease of use.

It is also related to the adapter pattern. With adapter, you are making an existing interface usable by wrapping one object. With the facade, you are defining a new interface for an entire subsystem of objects.

- Abstract Factory
Can provide an interface for creating subsystem objects.
- Mediator
Also abstracts functionality of existing classes, but its purpose is abstracting communication between objects, while facade is about promoting ease of use.
- Adapter
Adapter makes existing interfaces useable by wrapping one object, while with facade you are defining a new interface for an entire subsystem.

Structural Pattern: Proxy

Monday, October 31, 2022 1:44 PM

Coming up

In this module, we will cover the proxy pattern, also known as the surrogate pattern. It is intended to provide a surrogate or placeholder for another object to control access to it.

We will start by describing the pattern using a real-life example. As an example, we will use a remote API call for starters. After that we will map that to a structural example. Then we will look into variations of the proxy pattern, because there are a few. We will also choose an example to implement in a demo. We will continue with learning about the use cases for this pattern, and the consequences. And we will end the module with related patterns.

Describing the Proxy Pattern

The proxy pattern is a structural pattern from the well-known Gang of Four design patterns. Its intent is to provide a surrogate or placeholder for another object to control access to it. That is important to know because this means that the reason for this pattern, the problem it solves, is one of access control. If you need to control access to another object for security reasons, performance reasons or other reasons, then the proxy pattern is a good one to look into.

The archetypical example of a proxy, which some of you have undoubtedly encountered, is accessing a remote API from an application. Those of you who remember the Add service reference option in Visual Studio will know about this. The idea here is that your application, let's say it is an MVC app, uses a proxy to interact with the actual remote API. That proxy can be completely written from scratch. But as mentioned, there is something like the Add service reference dialog which would generate a proxy for you.

The proxy is responsible for controlling the actual access to the remote API. It can contain code that deals with the underlying protocol, for example the actual HttpClient calls. It can also contain code that ensures that the most performant, reliable way to access that remote API is used and so on. In principle these proxies have the same interface as the object they are controlling access to. So for the client, it feels transparent. The client shouldn't care whether it is directly interacting with remote API or it is going via the proxy. But truth be told, sometimes you see code generators generating what they call proxies that don't adhere to this principle, which makes them more of a combination between a facade or adapter and a proxy maybe.

The question is, why do we want to that? Why not directly call the API with an HttpClient instance? You can see an example of that below.

```
HttpClient client = new();  
var response = await client.SendAsync("api/remotecall", ...);
```

Well imagine that you need to execute something right before or after calling the API with HttpClient, which isn't uncommon. When you have a proxy in between, the proxy can take responsibility. That means that you don't have to change the client class anymore. Note that a proxy does not change the interface. If that is a requirement, you might want to consider the adapter pattern.

Proxy can take on the responsibility of executing code before or after calling the API.

Structure of the Proxy Pattern

The first player in the proxy pattern is called the subject. This is what defines the common interface

between two other classes, the RealSubject and the proxy. It is the contract both of these adhere to. Map this to our remote API call example, and you could see this as a service contract, say, with one operation on it, Request. The subject can be written as an abstract class, but it can also be implemented as an interface.

Client -----> Subject

RealSubject implements this interface. This would be the remote API service. It is the real object that the proxy represents.

Client -----> Subject
|
RealSubject

And then we have the proxy class which also implements this interface, and therefore, also has a Request method. It provides an interface identical to the subject so that the proxy can be substituted for the real subject. It also maintains a reference to and controls access to the RealSubject, and potentially, is responsible for creating and deleting it.

Client -----> Subject
| |
RealSubject <----- Proxy

This is the pattern structure. A few variations of this pattern exist. The example we looked into was the remote proxy variation.

Variations of the Proxy Pattern

The first variation is the remote proxy. As the one we used in the example. The idea here is that the client can communicate with the proxy, thus treating it as a local resource. The pattern hides the fact that the actual object exists in a different network space. The client does not need to know that a remote service is called. The proxy hides away these network and protocol details. More often than not, these proxies are generated from a service definition. That can be a WSDL or, more common, an OpenAPI specification.

The second variation is the virtual proxy. This one allows creating expensive objects on demand. You can look at it as a stand-in for an object that is expensive to create. Performance optimization is a big advantage of this type of proxy.

The third variation is the smart proxy. The purpose is to allow adding additional logic around the subject. Typical examples are managing caching or locking access to shared resources.

The fourth and last variation, is the protection proxy. This is used to control access to an object. It can be useful when objects should have different access rights. Like that, checking for that no longer has to be done by the client code or in the object itself, it is the protection proxy that is responsible for this.

- Remote Proxy
Client can communicate with the proxy, feels like local resource.
- Virtual Proxy
Allows creating expensive objects on demand.
- Smart Proxy
Allows adding additional logic around the subject.
- Protection Proxy

Used to control access to an object.

Sometimes the separation between these different types of proxy is not clear. It is not uncommon for code generators to, for example, generate a proxy that both hides away the fact that a remote service is accessed and takes care of permissions and access restrictions. This of course, is in violation of the single responsibility principle. A better approach would be to chain proxies, in other words, proxy-to-proxy, sticking with a single responsibility per proxy. These variations all have their own responsibilities and consequences, but as far as implementation is concerned, they all follow the same structure.

What we are going to do in an example is implement two of these. We are going to implement a virtual proxy and we are going to implement a protection proxy. That one is very closely related to the smart proxy, because both allow additional housekeeping tasks when an object is accessed. As an example we will use a document object, which represents a PDF or a text file.

Imagine that this is an object that is expensive to create, which is common. You might, for example, want to read out the underlying file from disk as part of the creation process, and that is quite expensive. The virtual proxy pattern can be used to ensure that creation only happens when needed. And then we will add a business rule stating that the document can only be viewed by people in the viewer role. We will use a protection proxy for this, and we will chain that with the virtual proxy.

Demo - Implementing the Proxy pattern

In this demo we will implement the proxy pattern. Let's start by adding a document class, our RealSubject. This consist of a few common properties you might expect from a document, like title, content, authorId, and LastAccessed. We also add a private fileName field to it, so we know where to load it from. We do not want the document to be constructed without a file name having been passed through, so we have a constructor to convey that. The constructor expects a fileName parameter, and we set the private _fileName field to the passed through parameter. Up next, we want to load the document from disk.

For that we want to add a LoadDocument method, which expects the fileName. This is an expensive method. We fake that by adding a Thread.Sleep statement so it takes a bit of time to execute this. Once the document has been loaded from disk between quotation marks, we fill out its properties. This method currently gets called from the constructor, so the expensive operation happens at the moment the document object is being constructed.

We add one more method, something a client can call into to display the document. In that, we write out some property values to the console output window. That makes this a bit easier to test.

And with that, we got our RealSubject, a Document class. This does not have anything to do with a proxy yet.

In Implementation.cs

```
public class Document
{
    public string? Title { get; private set; }
    public string? Content { get; private set; }
    public int AuthorId { get; private set; }
    public DateTimeOffset LastAccessed { get; private set; }
    private string _fileName;

    public Document(string fileName)
    {
```

```

        _fileName = fileName;
        LoadDocument(fileName);
    }

    private void LoadDocument(string fileName)
    {
        Console.WriteLine("Executing expensive action: loading a file from disk");
        // fake loading...
        Thread.Sleep(1000);

        Title = "An expensive document";
        Content = "Lots and lots of content";
        AuthorId = 1;
        LastAccessed = DateTimeOffset.UtcNow;
    }

    public void DisplayDocument()
    {
        Console.WriteLine($"Title: {Title}, Content: {Content}");
    }
}

```

From this Document class, we can now extract an interface. This will act as the subject from the proxy pattern structure. Both the RealSubject and the proxy must adhere to this. Let's say we want to put one method in it, DisplayDocument. LoadDocument is private, so the only one we are going to put in it is DisplayDocument. Seeing RealSubject must implement it, we do that. By adding it with : behind the class name.

```

Above Document{}:
public interface IDocument
{
    void DisplayDocument();
}

```

```

public class Document : IDocument{...}

```

Let's give this a try already. Again, nothing proxy related in there at the moment, so we simply write out we are constructing the document, we load it, and we write out that the document has been constructed. After that we display it. Let's add a ReadKey statement, and let's give this a try.

```

In Program.cs:
// Without proxy
Console.WriteLine("Constructing document.");
var myDocument = new Proxy.Document("MyDocument.pdf");
Console.WriteLine("Document constructed.");
myDocument.DisplayDocument();

Console.ReadKey();

```

We see that the expensive operation happens when the document is being constructed and not when it is first being used, in other words, not when it is being displayed. This may not be what you want, as this can make applications that require a lot of objects like this unnecessarily slow. Expensive operations like loading the files from disk may be executed without them being needed. Because if you don't display

what is in the file, you don't really need it to be loaded from disk either. It is better to offload this to the moment the document is first accessed, and that is where the virtual proxy comes in.

Let's add a new class. We will name it DocumentProxy. As the name implies this is the proxy. It too must implement the IDocument interface. That means we need to implement a DisplayDocument method on it. But first we construct our DocumentProxy. We do that by accepting a fileName, but we don't construct the underlying RealSubject, our document, yet, so we don't call into the base constructor or anything like that. At this moment, the expensive operation has not been executed yet. We do need to implement this DisplayDocument method.

In this method we need the underlying document, which may not have been constructed yet. So we add a private variable, _document, which will hold the document. It is nullable because the document hasn't been constructed yet after we leave the DocumentProxy constructor. Constructing the document, that is what we do when a method on the DocumentProxy is called, DisplayDocument. So it is here that we check whether the document is null. If it is, that is when we create it. Afterwards, we call DisplayDocument on the document variable.

This means that the first time we call into this method, the document is constructed and the expensive operation will run. After that, the document stored in the _document variable will be used.

In implementation, below Document{...}:

```
public class DocumentProxy : IDocument
{
    private Document? _document;
    private string _fileName;

    public DocumentProxy(string fileName)
    {
        _fileName = fileName;
    }

    public void DisplayDocument()
    {
        //Avoid creating the document until we need it
        if(_document == null)
        {
            _document = new Document(_fileName);
        }
        _document.DisplayDocument();
    }
}
```

Let's test this. For that, we write a bit of extra code in our Program file. We write out an empty line, and then we use the DocumentProxy to call DisplayDocument on. As you notice, the way to interact with the Document and the DocumentProxy is transparent for the client. The client executes exactly the same statements. For the client, interacting with one of both is the same.

In Program.cs above Console.ReadKey();:
Console.WriteLine();

```
//With proxy
Console.WriteLine("Constructing document proxy.");
var myDocumentProxy = new Proxy.DocumentProxy("MyDocument.pdf");
```

```
Console.WriteLine("Document proxy constructed.");  
myDocumentProxy.DisplayDocument();
```

The expensive operation is only executed when we actually need it to be. The advantage here is that the cost of creation of the actual object can be deferred to when it actually needs to be used. The proxy this lazily initializes the actual object that is interacted with. And this brings me to a small improvement we can make, something we also did when we implemented the singleton pattern. .NET includes support for lazy initialization out of the box via `Lazy<T>`. When correctly used it ensures that the object of type `T` is only used when needed and not when initialized, so we can change our `DocumentProxy` to support this.

Instead of working on a document directly, we now want to work on a document that is of type `Lazy<Document>`. We also don't need it to be nullable, as the lazy object is constructed in the constructor of our proxy. We construct it by passing through an action which tells it how it should construct an instance of `Document`. When the `Lazy<Document>` object instance is created, construction of the actual document doesn't happen yet. Our `DisplayDocument` method then becomes easier. We call into `document.Value.DisplayDocument`. If it is the first time the value is accessed and value is our document instance, the actual document object is created. Each time after that, the previously created instance will be used.

In `Implementation.cs` change `DocumentProxy`

```
public class DocumentProxy : IDocument  
{  
    //avoid creating the document until we need it  
    private Lazy<Document> _document;  
  
    private string _fileName;  
  
    public DocumentProxy(string fileName)  
    {  
        _fileName = fileName;  
        _document = new Lazy<Document>(() => new Document(_fileName));  
    }  
  
    public void DisplayDocument()  
    {  
        _document.Value.DisplayDocument();  
    }  
}
```

If you test this, you don't need to change anything in `Program.cs`. And it would have the same result as the previous test. But this time it is in a thread-safe manner, because that is what `Lazy<T>` makes sure of.

Demo - Chaining Proxies

In this demo, we will learn how to chain proxies. Let's think about a new requirement. Our document should only be displayed when users have the `Viewer` role. We could build this in the document itself or in the document proxy, but that would lay too many responsibilities on one class. Instead we can load of this access right decision to a new proxy. Let's add such a proxy!

We will name it `ProtectedDocumentProxy`. This too should implement the `IDocument` interface, which makes this another proxy over the same type. We construct the protected document proxy by storing

fileName and user role, and by constructing a document proxy, which we also store. As to remember the DocumentProxy is our proxy from the previous demo, and it is this one that makes sure that the actual document will not be constructed at the moment DocumentProxy is constructed.

That also means that when the protected DocumentProxy constructor is executed, the eventually underlying document isn't constructed yet. We are now effectively creating a chain of proxies, first to ProtectedDocumentProxy and after that, the DocumentProxy. In DisplayDocument(), we check the role and throw an unauthorized exception if the user is not Viewer, otherwise we display the document. We surround this with a few Console.WriteLine statements so we can see exactly what happens when. And with that, we have chained our two proxies.

In Implementation below DocumentProxy{...}:

```
public class ProtectedDocumentProxy : IDocument
{
    private string _fileName;
    private string _userRole;
    private DocumentProxy _documentProxy;

    public ProtectedDocumentProxy(string fileName, string userRole)
    {
        _fileName = fileName;
        _userRole = userRole;
        _documentProxy = new DocumentProxy(_fileName);
    }

    public void DisplayDocument()
    {
        Console.WriteLine($"Entering DisplayDocument " +
            $"in {nameof(ProtectedDocumentProxy)}.");

        if (_userRole != "Viewer")
        {
            throw new UnauthorizedAccessException();
        }

        _documentProxy.DisplayDocument();

        Console.WriteLine($"Exiting DisplayDocument " + $"in
            {nameof(ProtectedDocumentProxy)}.");
    }
}
```

Let's test this. First we add a new instance of ProtectedDocumentProxy, passing through the file name and role. We surround it with a few WriteLine statements so we can see what happens when. After the WriteLine statements, we call DisplayDocument. Seeing the role is Viewer, we should have access. And let's write something likewise, which will not give us access.

In Program.cs above Console.ReadKey();:
Console.WriteLine();

//With chained proxy

```
Console.WriteLine("Constructing protected document proxy.");
var myProtectedDocumentProxy = new Proxy.ProtectedDocumentProxy("MyDocument.pdf", "Viewer");
```

```

Console.WriteLine("Protected document proxy constructed.");
myProtectedDocumentProxy.DisplayDocument();

Console.WriteLine();

//With chained proxy, no access
Console.WriteLine("Constructing protected document proxy.");
var myProtectedDocumentProxy = new Proxy.ProtectedDocumentProxy("MyDocument.pdf",
"AnotherRole");
Console.WriteLine("Protected document proxy constructed.");
myProtectedDocumentProxy.DisplayDocument();

```

The result is, that we hit an `UnauthorizedAccessException`. That sounds like what we expect because of the last proxy chain instance we created, shouldn't give us access. Anyway, let's continue so we can see what is been written out to the console output window. The first two paragraphs are from the previous demo, so we are interested in the third paragraph. We see that it is the `ProtectedDocumentProxy` that is being constructed.

And once we enter `DisplayDocument` in the `ProtectedDocumentProxy`, the expensive action is executed, so that is the original proxy at work, and the content is written out. From this statement, we can nicely see the chain of proxies. And the next paragraph, as expected, tries the same, but throws an `UnauthorizedException` because we didn't pass through a role that would grant access.

Use Cases for the Proxy pattern

The basic use case for this pattern is when you need to add some form of access control to an actual object, something that is more sophisticated than a simple object reference. The use cases come from the pattern variations, so let's have a look. For the remote proxy, the use cases providing a local representative for an object in a different network address space. Any time you need to call into a remote API or service, this can be helpful. For virtual proxies, the main use case is creating expensive objects on demand. Smart proxies are useful for caching and locking scenarios, and protection proxies are useful when objects should have different access rules. A good real-life use case was the one we just implemented.

- Remote proxy
When you want to provide a local representative.
- Virtual proxy
When you only want to create expensive objects on demand.
- Smart proxy
When you are in need of a caching or locking scenario.
- Protection proxy
When objects should have different access rules.

Pattern Consequences

What this pattern does is introduce a small layer or wrapper between the client and the actual object. The consequences from that, again, vary depending on the proxy type.

For a remote proxy, a consequence is that the proxy hides the fact that an object resides in a different network space.

For a virtual proxy a consequence is that an object can be created on demand.

And for the protection and smart proxies, a consequence is that additional housekeeping tasks can be executed whenever an object is accessed.

- Remote proxy
Hides the fact that an object resides in a different network space
- Virtual proxy
The object can be created on demand.
- Smart proxy
Additional housekeeping tasks can be executed when an object is accessed.
- Protection proxy
Additional housekeeping tasks can be executed when an object is accessed.

As a general consequence, the pattern allows introducing new proxies without changing the client code, that is the open/closed principle from the solid principles at work.

It also has some potential disadvantages. One is added complexity because a new set of classes are introduced. Another one is a potential performance impact because you are passing through one or more additional layers.

Related patterns

Two structural patterns are related to the proxy, the adapter and the decorator.

The adapter provides a different interface to an objective that adapts, while proxy provides the same interface.

And decorators have similar implementations as proxies, but serve a different purpose. As we remember, the decorator adds responsibilities to an object. The proxy, on the other hand, controls access to an object.

- Adapter
Adapter provides a different interface, proxy provides the same interface.
- Decorator
Decorator adds responsibilities to an object, while proxy controls access to an object.

Structural Pattern: Flyweight

Monday, October 31, 2022 1:44 PM

Coming up

In this module we will cover the flyweight pattern, which is a pattern that uses sharing to support large numbers of fine-grained objects efficiently.

We will start to describe the pattern using a real-life example, the characters that make up a document. While doing so, we will cover intrinsic and extrinsic state. After that we will map that to a structural example and implement it. Then we will look into a special variation of the flyweight, the unshared concrete flyweight. We will continue by learning about the use cases for this pattern, and the pattern consequences. And will finish the module with related pattern.

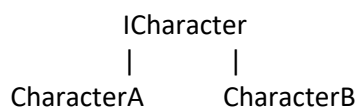
Describing the Flyweight Pattern

The flyweight pattern is a structural pattern from the Gang of Four patterns. The intent of the pattern is to use sharing to support large number of fine-grained objects efficiently. It does that by sharing parts of the state between these objects instead of keeping all that state in all of the objects. To explain this, by mapping it to a real-life situation. It is not that hard to come up with a good typical real-life example. We are going to use this pattern to share the characters of the document. That is a variation of the example that was originally used in the book from 1994. Afterwards, we will see why a lot of potential real-life situations that could call for this pattern actually don't. But in our case, it does look like a good fit. If each character in a document would have to be a new object instance, the amount of these that are required would explode.

We start with an ICharacter interface. All characters must implement this. This can also be an abstract base class. In our example, there is one method on it defined, Draw, which gets the fontFamily and Size passed through that the character needs to draw itself in.

ICharacter

Implementations of these are CharacterA and CharacterB and so on. We can consider these shared objects that can be used in multiple contexts simultaneously. Each of these acts as an independent object in each context. In that regard, it cannot be distinguished from an instance that isn't shared. And shared it because even though our document might have lots of A, B, and C characters, we will only hold onto one instance of each related class.



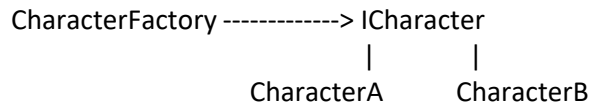
The key concept to allow for this is intrinsic versus extrinsic state.

The intrinsic state can be described as state data that is independent of the context. In other words, it is data that does not change if the object is used in different contexts. The character itself, the glyph if you want, A, B, C, remains as it is. That is intrinsic state.

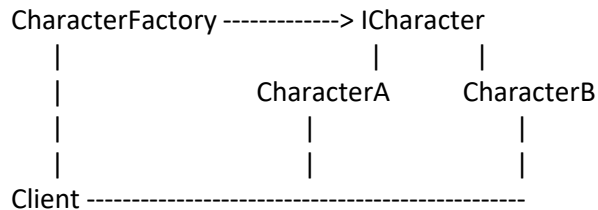
The extrinsic state then is data does vary with context. Different class instances might have different extrinsic state data, which cannot be shared. In our case, the fontFamily and fontSize would be extrinsic state.

We could expose that by adding a Draw method to the characters that accepts this fontFamily and fontSize. Set it and then draw the character. But with that, we are not there yet. We would need something that will manage these flyweights for us, a component that would parse the document and,

for each letter in the document, check whether the underlying characters has already been instantiated. And if it has, ensure that it can be reused. That is our character factory.



And lastly, there is of course, a client, something that consumes these characters and the factory. This is the example that we are going to implement.



Now there is a lot more examples you can think about that you could put in a pattern like this. Think about products with a fixed category and non-fixed weight and names, and ordering system with a few fixed intrinsic values per order, a library system, and so on. Yet, often it does not make sense to do that.

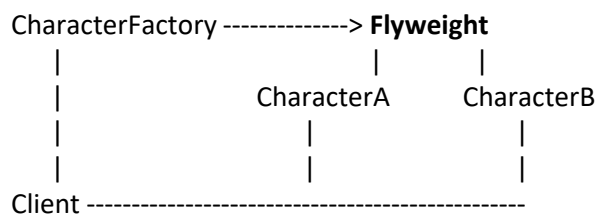
There is a few things to consider before implementing this pattern.

1. Does the app use a large number of objects? In our case, documents definitely use a lot of character objects.
2. Are storage costs, that is memory use, high because of the large amount of objects? In case of a document, yes.
3. Can most of the objects state be made extrinsic? Definitely in our case. The actual character glyphs are intrinsic, but things like color, font, size, etc., are extrinsic.
4. If you remove extrinsic state, can a large group of objects be replaced by relatively few shared objects? Yes, that is true too. In that case, we just end up with all the different characters in the alphabet.
5. Does the application require object identity? If it doesn't it is a good candidate. True for our example. The application does not need to know an ID or unique way to identify each character.

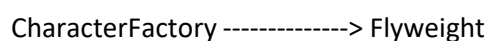
The general rule is only use this pattern when all five are true.

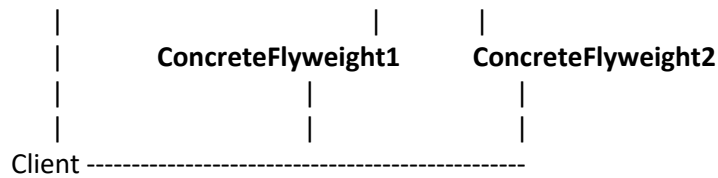
Structure of the Flyweight Pattern

The first player in the flyweight pattern is the flyweight itself. That is an abstract base class or interface through which flyweights can receive and act on extrinsic state. In our case, that would be the ICharacter interface that exposes a method, Draw, which accepts a fontFamily and fontSize.

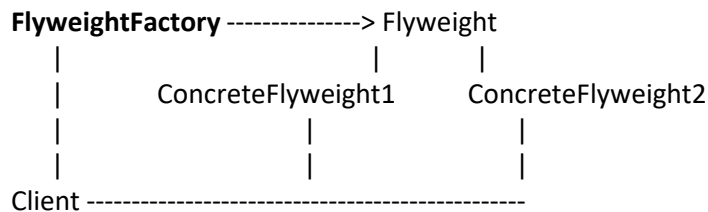


The second player is then the concrete implementation, the ConcreteFlyweight. These are our CharacterA, CharacterB, etc. classes implementing the Flyweight interface and thus implementing the draw method. It also adds storage for intrinsic state, our character itself, A, B, etc.

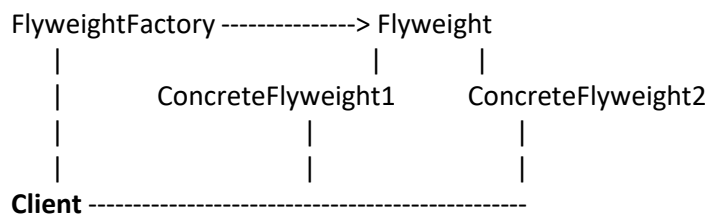




Then we have the FlyweightFactory. That is our CharacterFactory. It creates and manages flyweight, and it ensures that they are properly shared. In other words, when the flyweight does not exist yet, it creates an instance, and otherwise, it reuses the previously created one. On it, typically a method to get a flyweight exists.



And lastly, we have the client itself. It maintains a reference to flyweights and computes or stores the extrinsic state of flyweights. And with that we have our pattern structure.



Demo - Implementing the Flyweight pattern

In this demo, we will implement the flyweight pattern. We will start by creating the flyweight itself. This is the interface concrete implementations must adhere to. We will name it `ICharacter`, and we ensure that one method is required, `Draw`, which gets the extrinsic state passed in, `fontFamily` and `fontSize`.

```

In Implementation.cs
public interface ICharacter
{
    void Draw(string fontFamily, int fontSize);
}
  
```

Then onto the concrete implementations. We won't do this for all letters of the alphabet two, A and B, will suffice. So we create a new class, `CharacterA`, and have it implement `ICharacter`. We add a field for the actualCharacter, this is intrinsic state. The two other fields, the `fontFamily` and `fontSize`, are extrinsic state. These can be changed, and changing that is done via the `Draw` method. We get the extrinsic state passed in, we store it, and we write out a message to the console output window to mimic drawing the actual character on screen. And then we do the exact same thing for `CharacterB`.

```

Below ICharacter {}
public class CharacterA : ICharacter
{
    private char _actualCharacter = 'a';
    private string _fontFamily = string.Empty;
    private int _fontSize;
  
```

```

    public void Draw(string fontFamily, int fontSize)
    {
        _fontFamily = fontFamily;
        _fontSize = fontSize;
        Console.WriteLine($"Drawing {_actualCharacter}, {_fontFamily} {_fontSize}");
    }
}

public class CharacterB : ICharacter
{
    private char _actualCharacter = 'b';
    private string _fontFamily = string.Empty;
    private int _fontSize;

    public void Draw(string fontFamily, int fontSize)
    {
        _fontFamily = fontFamily;
        _fontSize = fontSize;
        Console.WriteLine($"Drawing {_actualCharacter}, {_fontFamily} {_fontSize}");
    }
}

```

Up next, is implementing the factory. Let's name that class CharacterFactory. A dictionary, which provides fast lookup by key is, a good way to store our concrete flyweights. As key, we give a character, and as value, an ICharacter. We are working on interface here and not on implementation. We add one method to it, GetCharacter, accepting an identifier and returning an ICharacter implementing instance, or potentially, null if we cannot match the identifier to a flyweight. In it, we first check whether our character is already in the dictionary, and we return that instance if it is. While doing so, we write that out to the console output window. Like that, we can easily follow along with what is happening. If the character requested isn't in the dictionary, we create the correct instance, store it, and return it. A switch statement is a convenient way of handling something like that, in our case. When none of these options apply, we return null.

Below CharacterB{ }:

```

public class CharacterFactory
{
    private readonly Dictionary<char, ICharacter> _characters = new();

    public ICharacter? GetCharacter(char characterIdentifier)
    {
        // Does the character dictionary contain the one we need?
        if (_characters.ContainsKey(characterIdentifier))
        {
            Console.WriteLine("Character reuse");
            return _characters[characterIdentifier];
        }

        // The character isn't in the dictionary.
        // Create it, store it, return it.
        Console.WriteLine("Character construction");
        switch (characterIdentifier)
        {

```

```

        case 'a':
            _characters[characterIdentifier] = new CharacterA();
            return _characters[characterIdentifier];
        case 'b':
            _characters[characterIdentifier] = new CharacterB();
            return _characters[characterIdentifier];
            // and so on...
    }

    return null;
}

```

That leaves us with writing the client code to test. The first thing we do is create a string which we are going to use for testing, and after that, we create a new instance of the CharacterFactory. For that, we need to import the flyweight namespace. Up next, we use the CharacterFactory to get the flyweight. We pass through the first character of our aBunchOfCharacters string seeing this is the first time the flyweight should be created. Afterwards, we pass through the extrinsic state, which is the fontFamily and the fontSize to the Draw method on the newly created characterObject. After that we do the exact same thing for the other three characters in our aBunchOfCharacter string. This should result in CharacterA and CharacterB instances being created. The first two times we call into GetCharacter on our factory, and those two instances being reused with another extrinsic state, the third and fourth time we call into GetCharacter and Draw. Let's give this a try.

In Program.cs

```

var aBunchOfCharacters = "abba";
var characterFactory = new CharacterFactory(); //Quick action to import using statement.

```

```

//Get the flyweight(s)
var characterObject = characterFactory.GetCharacter(aBunchOfCharacters[0]);
//Pass through extrinsic state
characterObject?.Draw("Arial", 12);

```

```

var characterObject = characterFactory.GetCharacter(aBunchOfCharacters[1]);
characterObject?.Draw("Trebuchet MS", 14);

```

```

var characterObject = characterFactory.GetCharacter(aBunchOfCharacters[2]);
characterObject?.Draw("Times Bew Roman", 16);

```

```

var characterObject = characterFactory.GetCharacter(aBunchOfCharacters[3]);
characterObject?.Draw("Comic Sans", 18);

```

```

Console.ReadKey();

```

We can see, that it is indeed the case. We see two character construction statements and two character reuse statements, yet effectively drawing the character is different all four times.

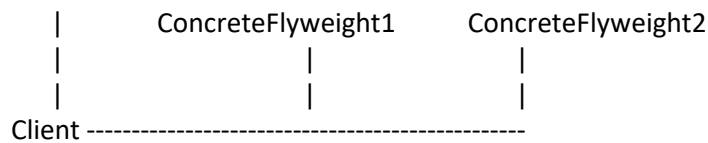
Working with an Unshared Concrete Flyweight

We are looking at the pattern structure we ended up with. A confession, when we looked into the pattern structure, somethings were skipped to make the pattern easier to grasp.

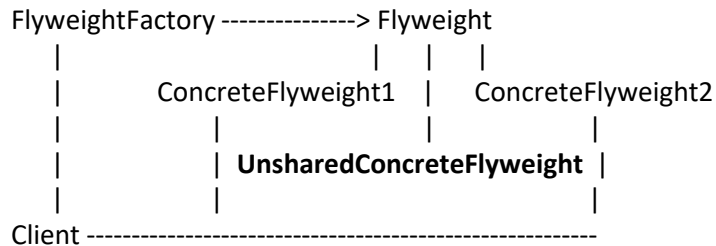
```

FlyweightFactory -----> Flyweight
|                           |
|                           |
|                           |

```



The thing skipped was the UnsharedConcreteFlyweight, which also implements flyweight. This type of flyweight implementation does not need to be shared. The flyweight interface enables sharing, but it does not enforce it. The UnsharedConcreteFlyweight is such an implementation. It enables acting on extrinsic state, while having unshareable intrinsic state.



Let's show this by an example. Say we want to introduce the concept of a paragraph to our document. A paragraph contains a location in the document and it contains characters, in other words, our concrete flyweights. What it doesn't contain is shareable state. The location and the characters it contains are linked to the paragraph. We cannot reuse the paragraph instance. A new paragraph means a new instance. As it does contain concrete flyweights, or characters, it can implement the same flyweight interface and use the method from that interface, Draw, to pass on the passed-in extrinsic state to the flyweights, the characters, it holds.

Now obviously, because the state is not shared, we lose the storage advantage the flyweight pattern is used for, but we do keep the advantage that working with the flyweight remains transparent to the client, whether it is unshared or not.

Demo - Supporting an Unshared concrete Flyweight

As unshared concrete flyweight, we will create a paragraph class. This holds a location, for example 1, 2, or 3. It stands for where in the document the paragraph lives. Next to that, a paragraph also holds a set of characters. We add a constructor that accepts these two, then we implement ICharacter. As it is a flyweight, so it should implement ICharacter. And with ICharacter you have to implement the Draw method. Drawing a paragraph, in our case, means that we want to ask each character to draw itself. So what we do here, is to pass through the extrinsic state to the Draw method on each character, and that is it.

In implementation below characterFactory:

```

public class Paragraph : ICharacter
{
    private int _location;
    private List<ICharacter> _characters = new();

    public Paragraph(List<ICharacter> characters, int location)
    {
        _characters = characters;
        _location = location;
    }

    public void Draw(string fontFamily, int fontSize)
    {

```

```

        Console.WriteLine($"Drawing in paragraph at location {_location}");
        foreach (var character in _characters)
        {
            character.Draw(fontFamily, fontSize);
        }
    }
}

```

On the Factory, we have to tell it how to create this unshared concrete flyweight. For that, we add a new method to the factory, `CreateParagraph`. A paragraph is always a new instance. When we are working with an unshared concrete flyweight, we are always working with new instances. We are not sharing state, in this case. Our `CreateParagraph` method accepts the location and list of characters, and simply creates a new instance of the unshared concrete flyweight, the paragraph, and returns it.

Change to CharacterFactory

```

public class CharacterFactory
{
    private readonly Dictionary<char, ICharacter> _characters = new();

    public ICharacter? GetCharacter(char characterIdentifier)
    {
        // Does the character dictionary contain the one we need?
        if (_characters.ContainsKey(characterIdentifier))
        {
            Console.WriteLine("Character reuse");
            return _characters[characterIdentifier];
        }

        // The character isn't in the dictionary.
        // Create it, store it, return it.
        Console.WriteLine("Character construction");
        switch (characterIdentifier)
        {
            case 'a':
                _characters[characterIdentifier] = new CharacterA();
                return _characters[characterIdentifier];
            case 'b':
                _characters[characterIdentifier] = new CharacterB();
                return _characters[characterIdentifier];
            // and so on...
        }

        return null;
    }

    public ICharacter CreateParagraph(List<ICharacter> characters, int location)
    {
        return new Paragraph(characters, location);
    }
}

```

Let's test this. We first create unshared concrete flyweight. We do that by calling in to `CreateParagraph`, passing through a list of characters and location for the flyweight. The warning given on `characterObject`

means that the character object might be null. If we don't want to allow that, we can add a null check. But for this demo, it isn't required. It doesn't have anything to do with the pattern implementation, as such. And then we can call Draw on the paragraph. As you notice for the client working with the flyweight or an unshared flyweight, it is completely transparent. What is returned from CreateParagraph is an ICharacter, which is the same interface as is returned from the call into GetCharacter. So as far as the client is concerned, this can be treated in a likewise manner.

```
In program.cs above Console.ReadKey();  
//Create unshared concrete flyweight (paragraph)  
var paragraph = characterFactory.CreateParagraph( new List<ICharacter>() { characterObject }, 1);  
  
//draw the paragraph  
paragraph.Draw("Lucinda", 12);
```

Let's build and run this. We see that we are drawing a paragraph, which results in drawing of the first character, as that is the character object we pass through.

Use Cases for the Flyweight pattern

The use cases for this pattern match the reasoning you would use in the first place to decide on whether the case is a good match for this pattern. We already looked into that when we described the pattern.

1. the application uses a large number of objects.
2. storage costs are high because the large amount of objects.
3. most of the object state can be made extrinsic.
4. if you remove extrinsic state, a large group of objects can be replaced by relatively few shared objects.
5. the application does not require object identity.

If these five rules check out, then you have got a good use case. A library system might be a good use case, a product categorization system might be a good use case, but they are not good use cases by default. Always check your specific requirements against these five rules.

Pattern Consequences

A positive consequence is that you may save lots of memory when you use this pattern for an applicable use case.

That being said, it might be that you end up with some more processing costs because of the need to inject or compute extrinsic state if that state was stored as intrinsic state previously. That is a disadvantage, but that is almost always offset by the reduced storage costs, i.e. the reduced memory use.

Another disadvantage is that this is quite a complex pattern, which means that your codebase, as a whole, will become a lot more complicated.

Related patterns

The first related pattern is the state pattern. When you implement it without instance variables in the state objects, you are essentially making these state objects sharable, which makes them flyweights.

The second related pattern is a strategy pattern. A strategy can also be implemented as a flyweight.

- State
State without instance variables makes these objects flyweights.
- Strategy
Strategy can be implemented as a flyweight.

Behavioral Pattern: Template Method

Monday, October 31, 2022 1:45 PM

Coming up

In this module we will dive into the template method pattern. Its intent is to define a skeleton of an algorithm in an operation, deferring some steps to subclasses. This pattern is actually one of the fundamental techniques for code reuse.

We will start by describing the Template Method pattern. For that we will use a real-life example. The real-life will be parsing email for a startup company's software that wants to revolutionize the way we look at email. We will map that to the pattern structure, and we will implement it in a demo. We will continue with learning about use cases for this pattern, and the pattern consequences. We will end this module with related patterns.

Describing the Template Method Pattern

The template method pattern is a behavioral pattern from the well-known Gang of Four library of design patterns.

Its intent is to define the skeleton of an algorithm in an operation, deferring some steps to subclasses. It lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

Let's clarify this with an example.

Imagine that we are writing part of an application that wants to make working with emails easier by accumulating mail across different mail servers and analyzing them. The idea is that the company's proprietary artificial intelligence will ensure that your inbox services the email that are important to you. To be able to do that, the software must connect to different mail servers and parse mails.

So we got a few classes, ExchangeMailParser, ApacheMailParser, EudoraMailParser, and so on. For the sake of the demo, we will assume that the parser parses one HTML mail at a time, by accepting an identifier.

```
public class EudoraMailParser {}  
public class ApacheMailParser {}  
public class ExchangeMailParser {}
```

The thing is, a lot of what has to happen in the algorithm to parse, one mail is the same for these three classes. First, the server needs to authenticate to the server. That is probably a bit different for each implementation. Then the mail body must be parsed, and that is exactly the same for each of these parsers. We don't want that kind of code duplication.

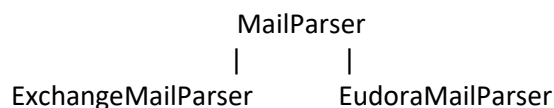
```
public class ExchangeMailServer  
{  
    public void FindServer() { //Some code }  
  
    public void AuthenticateToServer() { //Some code }  
  
    public string ParseHtmlMailBody(string identifier)  
    {  
        //Some code  
        return "The mail body";  
    }  
}
```

```

    }
}

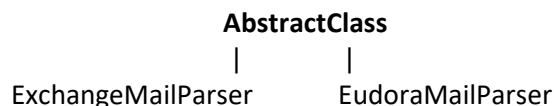
```

What we can do is create an abstract base class with a template method on it. That template method, `ParseMailBody`, accepting an identifier calls the other three methods in a certain order. The other three methods are this also defined on this abstract class. Subclasses like `ExchangeMailParser` or `EudoraMailParser` call this template method on the base class to parse some mail. We allow these subclasses to vary the algorithm by overriding abstracted methods. We could make the `AuthenticateToServer` method abstract as it differs for each class. That means that each subclass must implement this method. We can also allow subclasses to optionally override the other methods as well, in case they would require changes by making them virtual. In Gang of Four lingo, these methods or operations are called hooks, and `FindServer` is one of them in our example. The only thing that absolutely must not be overridden is the template method itself. That is what the template method pattern is, a straightforward, often used pattern in OO design.

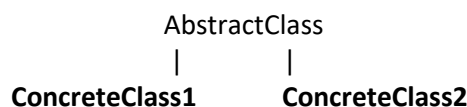


Structure the Template Method Pattern

There is only two players in this pattern structure. The first one is the `AbstractClass`. That is our `MailParser`. This defines abstract primitive operations that concrete subclasses define to implement steps of an algorithm. Our `AuthenticateToServer` method is an example of that. A variation is that not all methods are primitive operations that are abstract and thus must be overridden. Some of these methods can be made virtual so they can be overridden if needed. The other methods, optionally, could be examples of that. The `AbstractClass` also implements a template method, which defines the skeleton of the algorithm. In other words, the order in which the other methods are called. It thus calls primitive methods, as well as other methods defined in an abstract class or on other objects.



The second player is the `ConcreteClass`. This implements the primitive operations to carry out subclass-specific steps of the algorithm. They map to our `ExchangeMailServer`, `ApacheMailServer`, and `EudoraMailServer`, and so on classes.



Demo - Implementing the Template Method pattern

In this demo we will implement the template method pattern. We will start with an abstract class. That is the abstract class component from our diagram, and we name it `MailParser`. On it, we define three different methods to start with. `FindServer` is used to find the mail server. This method tends to be the same across concrete implementations, but we mark it as virtual just in case there is a `MailParser` for which this differs. `AuthenticateToServer` is abstract.

This is a primitive operation, which differs for each subclass. We want subclasses to implement it, and that is why we mark it as abstract. And then we have `ParseHtmlMailBody`. This method remains the same across different implementations, so we don't mark it as abstract nor as virtual. And then we add

one more method. We name the method ParseMailBody, and this is our template method. In it, we define the algorithm by calling the other methods in order.

In Implementation.cs

```
public abstract class MailParser
{
    public virtual void FindServer()
    {
        Console.WriteLine("Finding server ...");
    }

    public abstract void AuthenticateToServer();

    public string ParseHtmlMailBody(string identifier)
    {
        Console.WriteLine("Parsing HTML mail body...");
        return $"This is the body of the mail with id {identifier}";
    }

    public string ParseMailBody(string identifier)
    {
        Console.WriteLine("Parsing mail body in template method....");
        FindServer();
        AuthenticateToServer();
        return ParseHtmlMailBody(identifier);
    }
}
```

Now let's create concrete implementations of it. We start with the exchange implementation. Here we override one method, the abstract one, AuthenticateToServer, to authenticate to the mail server.

Below MailParser{}

```
public class ExchangeMailParser : MailParser
{
    public override void AuthenticateToServer()
    {
        Console.WriteLine("Connecting to Exchange");
    }
}
```

The ApacheMailParser follows likewise principles. Where we only need to override the abstract method.

Below ExchangeMailParser

```
public class ApacheMailParser : MailParser
{
    public override void AuthenticateToServer()
    {
        Console.WriteLine("Connecting to Apache");
    }
}
```

And then we have the EudoraMailParser. This one is slightly different. We are assuming that finding such a server takes a bit more effort. Luckily, we can override the virtual FindServer method from the base

class to achieve this.

Below ApacheMailParser:

```
public class EudoraMailParser : MailParser
{
    public override void FindServer()
    {
        Console.WriteLine("Finding Eudora server through a custom algorithm...");
    }

    public override void AuthenticateToServer()
    {
        Console.WriteLine("Connecting to Eudora");
    }
}
```

Note that the template method, ParseMailBody, is available on all these classes through their base MailParser class. That is the one clients will use. Nothing we have done, changed the order of what is in that method, and we cannot override it nor change it. This one will be written out to the console window.

Let's test this. The first thing we will try, is our ExchangeMailParser. We need to import template method namespace for that. And on it we call the template method, ParseMailBody. We pass through an identifier as well. And now we do the same for the ApacheMailParser, and the EudoraMailParser.

In Program.cs

```
ExchangeMailParser exchangeMailParser = new(); //Quick action to import using statement
Console.WriteLine(exchangeMailParser.ParseMailBody("bf3a298c-9990-4b02-873d-3d3c98ad16d2"));
Console.WriteLine();
```

```
ApacheMailParser apacheMailParser = new();
Console.WriteLine(apacheMailParser.ParseMailBody("07b8a8c7-c598-4b6c-9049-ecce9fe4a56b"));
Console.WriteLine();
```

```
EudoraMailParser eudoraMailParser = new();
Console.WriteLine(eudoraMailParser.ParseMailBody("789fe935-ced2-4fbd-865b-172909560a93"));
```

```
Console.ReadKey();
```

When we run this, we should be able to nicely see that subclassing the base class and overriding an extra method for the EudoraMailParser will have an effect even though we are calling the same template method three times.

When running the test we notice, the AuthenticateToServer method is different for each of these implementations, and we also notice that for the EudoraMailParser, finding the server is different as well.

Use Cases for the Template Method pattern

Use this pattern when you want to implement invariant parts of an algorithm only once and want to leave it to subclasses to implement the rest of the behavior, the parts that can vary like our authenticate to server method.

Also, use it when you want to control which part of an algorithm subclasses can vary. Those are the methods that you make virtual or abstract.

Lastly, use it when you find yourself in a situation where you have a set of algorithms that don't vary much, maybe only in one or two methods. Those are the methods that you make virtual or abstract.

Pattern Consequences

Using this pattern comes with a few consequences. First of all, these template methods are a fundamental technique for code reuse, especially when creating class libraries. That is because template methods are how you factor out common behavior by pulling it out into a parent class, thus removing code duplication.

A potential negative consequence is that template methods cannot be changed. The order of the methods they call, the skeleton, is fixed, and that may be too strict for some clients.

Related patterns

Factory method and template method are related patterns. You can even look at the factory method as a specialization of template method, aimed at object creation. Often template methods use factory methods as part of their skeleton structure.

It is also related to the strategy pattern. Both are used to vary an algorithm. Template methods allows varying part of an algorithm through inheritance. The subclass implements the abstract base methods. It works at class level, so this is static. Strategy is based on composition. By providing different strategies, the object's behavior can be altered. It works at object level, making it dynamic. Behaviors can be switched at run time.

- Factory method
Factory method can be viewed as a specialization of template method. Template methods often use factory methods as part of their skeleton structure.
- Strategy
Template method allows varying part of an algorithm through inheritance: a static approach. Strategy allows behavior to be switched at run time, via composition: a dynamic approach.

Behavioral Pattern: Strategy

Monday, October 31, 2022 1:45 PM

Coming up

In this module we will cover the strategy pattern, also known as the policy pattern. It is one of the most commonly used behavioral patterns. The idea behind it is that it will let an algorithm, the strategy, used by clients to achieve something vary independently from the clients that use it.

We will start by describing the pattern using a real-life example, exporting an order to a certain format. After that we will map that to a structural example, and implement it in a demo. We will also implement an alternative approach to the original pattern template. We will continue to learn about use cases for this pattern and pattern consequences. We will end this module with related patterns.

Describing the Strategy Pattern

The strategy pattern is a behavioral pattern from the Gang of Four library of design patterns, and probably one of the most often used behavioral pattern that exist. Its intent is to define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it. In other words, you define a family of algorithms, put each one in a separate class, and make them interchangeable.

Imagine the following use case. You are working on a part of a system that manages orders. At a certain moment, a business requirement comes in. The order needs to be exportable as a CSV file. You can solve that by adding an Export method on the order that creates this CSV file.

```
public class Order
{
    public void Export()
    {
        //Code to export to CSV...
    }
}
```

A bit later, another requirement comes along. You need to support exporting to JSON as well. This is still manageable. You can accept a parameter in the export method, exportFormat, and depending on the parameter value, execute another piece of code.

```
public enum Format
{
    CSV,
    JSON
}

public void Export(Format exportFormat)
{
    //Code to export to CSV or JSON...
}
```

It appears that two formats are still not good enough. Next you have to add support to XML as well. And now it is starting to be more difficult to manage. You could continue on the same route, adding additional format type and extending the conditional statement in the Export method, but this approach

has some disadvantages. The order class is becoming more complex with all this export logic contained in it. When you want to change export logic or add new ways to export, you always have to change the order class. This definitely means that the order class is starting to have too much responsibilities, and you can partially solve that by adhering to the single responsibility principle.

```
public enum Format
{
    CSV,
    JSON,
    XML
}

public void Export(Format exportFormat)
{
    //Code to export to CSV or JSON or XML...
}
```

You can put the different types of export functionalities in classes of their own, CSVExportService, JSONExportService, XMLExportService, and so on. And store instances of those on the Order class. In the export method, the correct instance is then used, depending on the inputted exportFormat parameter. That also means that the Order class is now responsible for the lifetime management of the different export services. And you have tightly coupled the Order class to these implementations. That still makes it hard to add export functionality for new formats.

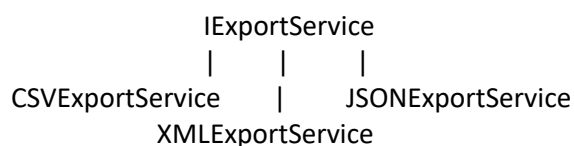
```
public class Order
{
    private JSONExportService _jsonExportService = new();
    private XMLExportService _xmlExportService = new();
    private CSVExportService _csvExportService = new();

    public void Export(Format exportFormat)
    {
        //Export using service instances, depending on exportFormat
    }
}
```

Enter the strategy pattern, which solves these issues. The first thing we will do is declare an interface, IExportService. This declares one method, Export, which accepts the thing to export, the Order, in our case.

IExportService

Our three or potential more export service now become implementations of this interface, CSVExportService, JSONExportService, and XMLExportService. In the Export method implementation, the concrete export is handled.



The order class then has a property of IExportService, which is set to one of the concrete implementations. Whenever Export is called on Order, the currently set IExportService implementation

is used to handle the export.



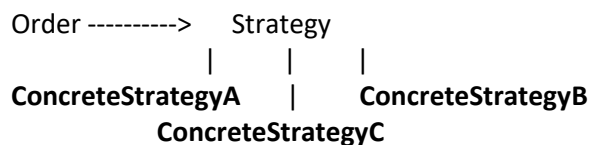
This means that the order object isn't responsible for selecting an appropriate algorithm, in other words, an interface implementation, for the job. It is the client that passes this implementation to the Order class. The Order class itself doesn't need to know about this. All it knows about is the interface and not the implementation. We have thus achieved loose coupling, making the order independent of the implementation, and that makes it easier to add new implementations or modify existing ones. There is no need to change the Order class to do so.

Strategy Pattern Structure

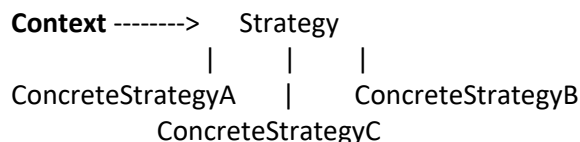
Let's start with our IExportService. That, in pattern lingo, is named a strategy. Such a strategy declares an interface common to all supported algorithms. The context, which is our Order class, we will get there soon enough, uses this interface to execute a concrete algorithm.



That brings us to the ConcreteStrategy. These are implementation of the Strategy interface. In other words, they implement the algorithm using the strategy interface. This maps to our different service implementations.



The last part of the story is the Context, which maps to our order class. This is configured with a concrete strategy object and maintains a reference to a strategy object. That is what the arrow from Context to strategy means. It often contains a way for the strategy to access its data. In our example, that was the export method. That method would require the data that needs to be written to the file, for example, by passing through the full object that needs to be exported, the order in our example.



Demo - Implementing the Strategy pattern

Let's start with creating the strategy, that is our IExportService interface. We are using an interface here, but we can also use an abstract base class in case you want to provide some out-of-the-box implementation of parts of its functionality. In this class it is defined one method, Export, which accepts an Order object as it needs its data to be able to export it. This is just one way of providing an interface to the order data. Another way could be to accept just property values that need to be exported. But in this demo, we are mimicking that we need access to all properties of the order object.


```
In Implementation.cs
public interface IExportService
{
    void Export(Order order);
}
```

Let's make the order class. In our real life example, we had three different formats to export to, so let's create implementations of the strategy for those. These are concrete strategies. One concrete strategy is `JsonExportService`, which implements the `IExportService` interface. That means we need to implement the `Export` method, which we do.

Our second concrete strategy is the `XMLExportService`, which follows a likewise principles, as the `jsonExportService`, and the last one is our `CSVExportService`, which also follows the same principles. We implement one method on it, `export`.

Below `IExport {}`:

```
public class JSONExportService : IExportService
{
    public void Export(Order order)
    {
        Console.WriteLine($"Exporting {order.Name} to JSON.");
    }
}

public class XMLExportService : IExportService
{
    public void Export(Order order)
    {
        Console.WriteLine($"Exporting {order.Name} to XML.");
    }
}

public class CSVExportService : IExportService
{
    public void Export(Order order)
    {
        Console.WriteLine($"Exporting {order.Name} to CSV.");
    }
}
```

Maybe it is time to get that order to resolve, so let's add an order class, that `Order` class that maps to the context component of our pattern structure. It is a straightforward class. We add a few common order-related fields and a constructor.

The important part is what is coming now. We need to add one more property to our `Order` class, and this one is of type `IExportService`, our strategy. It will be set from outside of our `Order` class so it is nullable. And then we add an `export` method on the order class. It is in this `export` method that we use the strategy to export the order.

Important to know is that the `Order` class doesn't know about the concrete implementation of the strategy that is going to be used. All it knows is that something of type `IExportService` will be used to export itself. And with that, we have created all the different moving parts of our pattern.

Below CSVExportService:

```
public class Order
{
    public string Customer { get; set; }
    public int Amount { get; set; }
    public string Name { get; set; }
    public string? Description { get; set; }
    public IExportService? ExportService { get; set; }

    public Order(string customer, int amount, string name)
    {
        Customer = customer;
        Amount = amount;
        Name = name;
    }

    public void Export()
    {
        ExportService?.Export(this);
    }
}
```

Now let's test this! Let's make an instance of an order, and let's try exporting it. It is up to the client to provide the concrete implementation of the strategy, and we do that by setting the ExportService property on our order. We set it to an instance of CSVExportService, and we try exporting. Afterwards, we set the ExportService to a new instance of JSONExportService, and we try export again. Then you add a ReadKey statement, and then run the test.

In Program.cs

```
var order = new Order("Marvin Software", 5, "Visual Studio License"); //Quick action to insert using statement.
```

```
order.ExportService = new CSVExportService();
order.Export();
```

```
order.ExportService = new JSONExportService()
order.Export();
```

```
order.ExportService = new XMLExportService()
order.Export();
```

```
Console.ReadKey();
```

This works as expected. Our different strategy implementations made sure that the correct export service was used.

To add a new export service, the nice thing is that we no longer need to make changes to the Order class, as it is not tightly coupled to an implementation of IExportService. Note that this implementation does mean that Export can be called without an ExportService being set. To avoid that, we could provide a default value. For example, in the order constructor, we can set the ExportService to a new instance of CSVExportService stating that it will be our default value

In implementation change Order class:

```

public class Order
{
    public string Customer { get; set; }
    public int Amount { get; set; }
    public string Name { get; set; }
    public string? Description { get; set; }
    public IExportService? ExportService { get; set; }

    public Order(string customer, int amount, string name)
    {
        Customer = customer;
        Amount = amount;
        Name = name;
        ExportService = new CSVExportService();
    }

    public void Export()
    {
        ExportService?.Export(this);
    }
}

```

Another approach would be to not allow the order to be constructed without providing an IExportService implementing object. When we implement it like this, it is still the caller that provides the default and not the Order class itself. That being said, dependencies like this are often injected via an IOC container, which tend to have their own null checks in place. It still remains a good practice to check for null reference issues.

In implementation change Order class:

```

public class Order
{
    public string Customer { get; set; }
    public int Amount { get; set; }
    public string Name { get; set; }
    public string? Description { get; set; }
    public IExportService? ExportService { get; set; }

    public Order(string customer, int amount, string name, IExportService exportService)
    {
        Customer = customer;
        Amount = amount;
        Name = name;
        ExportService = exportService;
    }

    public void Export()
    {
        ExportService?.Export(this);
    }
}

```

But for this demo we won't have IExportService in the method.

In implementation change Order class:

```
public class Order
{
    public string Customer { get; set; }
    public int Amount { get; set; }
    public string Name { get; set; }
    public string? Description { get; set; }
    public IExportService? ExportService { get; set; }

    public Order(string customer, int amount, string name)
    {
        Customer = customer;
        Amount = amount;
        Name = name;
    }

    public void Export()
    {
        ExportService?.Export(this);
    }
}
```

Demo - Implementing a Strategy

In this demo we will look into a variation of the pattern template, which is still a valid strategy pattern implementation. What we can do is have the export method accept an IExportService object instead of storing it on the Order class. We add a null check, and instead of using the ExportService property on the order class, we use the passed-through instance. It means we can get rid of this property. This is a variation of the template, and it just looks a little bit different from the structural design pattern. But in essence, we are still leveraging the strategy pattern here. The intent is still the same.

In implementation change Order class:

```
public class Order
{
    public string Customer { get; set; }
    public int Amount { get; set; }
    public string Name { get; set; }
    public string? Description { get; set; }

    public Order(string customer, int amount, string name)
    {
        Customer = customer;
        Amount = amount;
        Name = name;
    }

    public void Export(IExportService exportService)
    {
        if (exportService is null)
        {
            throw new ArgumentNullException(nameof(exportService));
        }
    }
}
```

```

        exportService.Export(this);
    }
}

```

Let's change our test code to accommodate for this adjustment. We don't have an export service property to set anymore. We simply pass through an instance to the Export method. And let's run this test.

Changes in Program.cs

```
var order = new Order("Marvin Software", 5, "Visual Studio License"); //Quick action to insert using statement.
```

```
order.Export(new CSVExportService());
```

```
order.Export(new JSONExportService());
```

```
order.Export(new XMLExportService());
```

```
Console.ReadKey();
```

It still works as expected. It is a small change, but it is one that you will often encounter in today's code bases. In essence, all these methods that accept an interface instead of an implementation to achieve a task are thus implementations of the strategy pattern, and that is why this pattern is so commonly used.

Use Cases for the Strategy pattern

You can use this pattern when many related classes differ only in their behavior, not in their interface. Strategies provide a way to configure a class with one of many behaviors, as we have seen with our three different export service implementations.

You can also use it when you need different variants of an algorithm, which you want to be able to switch at runtime, an export algorithm to three different forms, for example.

Another good use case is when your algorithm uses data, code or dependencies that the clients shouldn't know about. Imagine that exporting to a proprietary format needs some additional configuration or properties. The clients don't need to know about this or be allowed to change it. When they are working on an interface, they don't know about these, they only know about the export method existing.

Lastly, use it when a class defines many different behaviors which appear as a bunch of conditional statements in its methods. In other words, when you are dealing with large if or switch statements.

Pattern Consequences

The strategy pattern offers an alternative to subclassing your context. Continuing with our example, we could have created an XML order subclass, a CSV order subclass, and so on. But that would have hardwired the behavior into the context, the order. That makes it harder to understand and maintain the context. The strategy pattern avoids that. In other words, you are replacing inheritance with composition, and that is one of the things the Gang of Four favors in OO design.

Important as well is that new strategies can be introduced without having to change the context. That is the open/closed principle from the SOLID coding principles at work.

Strategies also eliminate conditional statements. You don't need that anymore to select the required

behavior because it's the client that injects it.

Strategies can provide a choice of implementations with the same behavior if needed, and the client can choose which one fits best at runtime.

This brings us to a drawback. If it is the client that injects the strategy to use, the client must be aware of how strategies differ, which means that they can be exposed to implementation issues. Therefore, only use this pattern when the variation in behavior is relevant to clients.

There is also a bit of overhead in communication between the strategy and the context. For example, when creating export services, some of them might not export the full order, even though we do pass it through.

And lastly, this pattern does introduce an additional number of objects. If you only have a few algorithms, it might not be worth the additional complexity.

Related patterns

The Gang of Four only defines one related pattern, the flyweight. That is because strategy objects often make good flyweights. But there is more than just this related pattern.

Strategy, Bridge, and State are all related, and they look similar because they are all based on composition, but they solve different problems. The strategy pattern is also related to the template method pattern. Both are used to vary an algorithm.

Template method allows varying part of an algorithm through inheritance, the subclass that implements the abstract base methods. It works at class level so this is static. Strategy, is based on composition. By providing different strategies, the objects behavior can be altered. It works at object level, so it is dynamic. Behaviors can be switched at runtime.

- Flyweight
Strategy objects make good flyweights.
- Bridge
Also based on composition, but solves a different problem.
- State
Also based on composition, but solves a different problem.
- Template
Template method allows varying part of an algorithm through inheritance: a static approach.
Strategy allows behavior to be switched at runtime, via composition: a dynamic approach.

Behavioral Pattern: Command

Monday, October 31, 2022 1:45 PM

Coming up

In this module we will cover the command pattern, also called the action or transaction pattern. It allows us to encapsulate requests as objects.

We will start by describing the pattern using a real life example. As real-life example, we will use UI interaction, clicking a button to add an employee to a list. After that we will map that example to the structural pattern. We will also implement our real-life example in a demo, and we will look into a variation which will allow us to support undo by using a command manager. We will continue with learning about use cases for this pattern, and pattern consequences. We will end the module with related patterns.

Describing the Command Pattern

The Command pattern is a behavioral pattern from the Gang of Four library of design patterns. Behavioral patterns define ways to communicate between classes or objects, letting you concentrate on the way objects are interconnected. The command pattern is a very nice illustration of that. Its intent is to encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations. It just turns the request into a standalone object, and once we got an object, we have got something to work with and/or pass around.

To clarify this. A very typical example of a command is clicking a button in the user interface, for example, to open a file, add a product to a shopping cart, add a new employee to a list, and so on. In a naive implementation we could just write the code for that in the button click event handler, but that comes with some potential issues. You might want to separate concerns by not having the code for handling the request, the button click, in our example, at the same location as the button itself. You might want to be able to reuse the actual code that needs to be executed, and maybe your application architecture doesn't even technically allow you to implement the code for the button click in the same place as where the button resides. For example, when you have separated out the button or menu in a separate project that doesn't know about the receiver of the actual request.

```
void SomeButton_Click(Object sender, EventArgs e)
{
    //open file...
    //add product...
    //add employee to list...
}
```

This is where the command pattern comes in. It allows you to decouple the requester of an action from the receiver. This pattern is very common in mobile or rich UI development, where you would typically use the Model-View-ViewModel pattern combined with the command pattern for transferring UI interactions to your view model. At that level, the actual command would be implemented.

A command would be bound to some UI XAML code or, if that is not possible, could manually be executed via the button click. You can see examples of that below.

```
//Execute a command on click via binding
<Button Command="{Binding SomeCommand}" Content="A button" />
```

```
//manually execute a command on click
void SomeButton_Click(Object sender, EventArgs e)
{
    var viewModel = (AViewModelClass)DataContext;
    if (viewModel.SomeCommand.CanExecute())
    {
        viewModel.SomeCommand.Execute();
    }
}
```

Let's learn how we can create these commands. Let's say we are building an employee-manager application. In it we want to add an employee to a list of employees a certain manager manages when a user clicks a button. The object that invokes the command could be a button only interface to which the command is bound, or on which the command property is set.

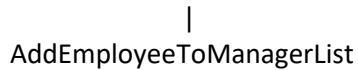
Button

Adding that employee to that list, that is executing the command. So we could have an interface, ICommand, containing one method, Execute, to execute the command. The interface could also be implemented as an abstract class.

Button -----> ICommand

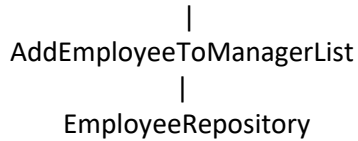
A good name for the command, in our case, would be `AddEmployeeToManagerList`. This class implements the `ICommand` interface and executes the action, actually adding the employee to a manager's employee list. For that, the command needs something, it needs to know who the employee is. It also needs to know who the manager is. So that is additional state the command must hold.

Button -----> ICommand



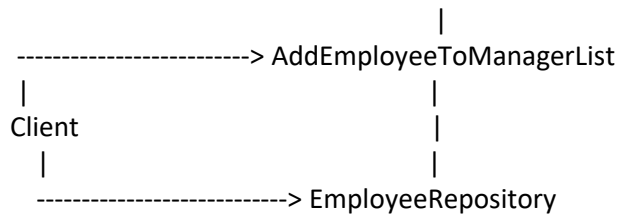
Once our command is being executed, something needs to happen. The employee needs to effectively be added to the manager's list. We could use a repository for that on which we can call a method to add the employee.

Button -----> ICommand



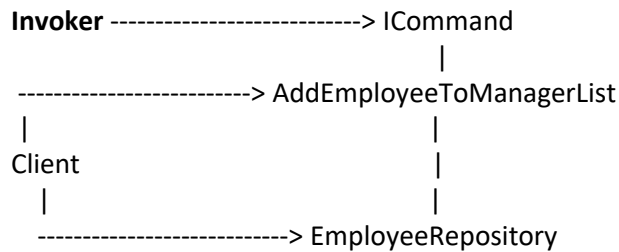
And that brings us to the last part, the client application, which creates the commands. In our case, it would pass the repository through to the command when it creates the command, typically via the command constructor.

Button -----> ICommand

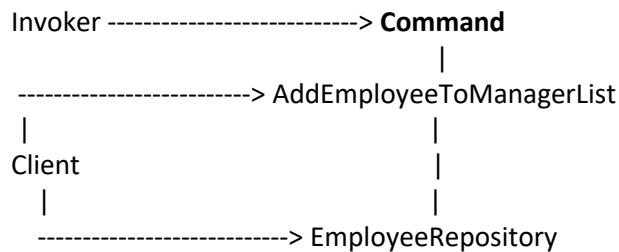


Structure of the Command Pattern

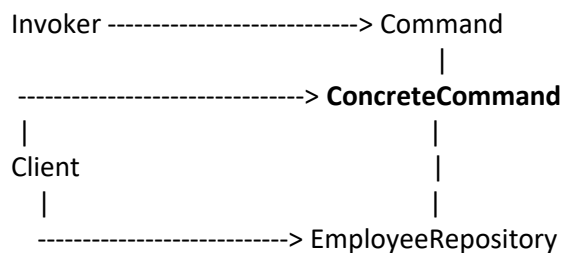
We will start with the object that evokes the command. This is the Invoker, and it is responsible for asking the command to carry out a request. It maps to the menu button in our example.



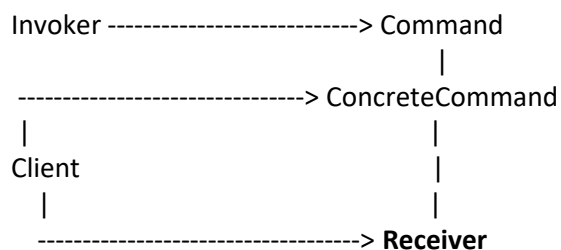
ICommand, our command interface, maps to Command. It declares an interface for executing an operation.



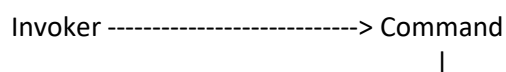
The implementation of that interface, then is the ConcreteCommand that maps to our AddEmployeeToManagerList command. It defines a binding between the receiver and the action. It implements Execute, and it does that by invoking the corresponding operations on the receiver.

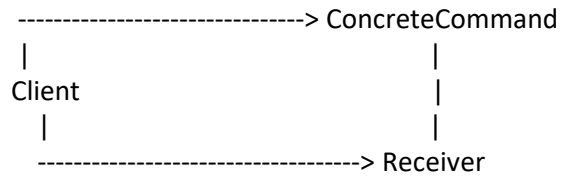


And that brings us to that receiver. The receiver is an object that knows how to perform the operations associated with carrying out a request. In our case, that is our repository, but any class may serve as a receiver.



And lastly, we got the client. The client creates the ConcreteCommand and sets its receiver.





Demo - Implementing the Command pattern

In this demo, we will implement the use case we covered when describing the command pattern. The first thing we will need for that is a bit of plumbing code. We need classes for our employee, for our manager, and we also need a repository. Let's start with the employee. This is a pretty straightforward class. It mimics an employee. Every employee has an ID and a name, and we use the constructor to convey meaning that we want this employee to be initialized with an ID and a name.

In Implementations.cs

```

public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }

    public Employee(int id, string name)
    {
        Id = id;
        Name = name;
    }
}
  
```

Now we need a manager. A manager is also an employee, so we let it derive from the Employee class. But next to being an employee, the manager also manages a bunch of employees. We mimic that by adding an employee list.

```

Below Employee{}
public class Manager : Employee
{
    public List<Employee> Employees = new();
    public Manager(int id, string name) : base(id, name)
    {
    }

}
  
```

Now we need something to manage these classes, or rather, we need a way to add employees to a manager's list, remove employees from it, and check whether or not an employee is already in the manager's list. We can do this via a repository. First, we add an interface that defines the contract of our repository. This acts as the interface for the receiver, which is one of the components from the structure of the pattern. As you can see it has the three methods we already talked about and one additional method, WriteDataStore. That is simply a helper method we are going to use to write out some info to the console output window.

```

Below Manager {}
public interface IEmployeeManagerRepository
{
    void AddEmployee(int managerId, Employee employee);
  }
  
```

```

    void RemoveEmployee(int managerId, Employee employee);
    bool HasEmployee(int managerId, int employeeId);
    void WriteDataStore();
}

```

Now we need to implement this repository. So as far as the data store is concerned, we are going to use an in-memory list. In a real-life scenario you would probably want to persist this to a database somewhere. We use the list of managers as a data store to mimic a database. And then we have methods to add employees to these managers from the data store. So we have one method that accepts a manager ID and the employee to add, to add it, and another to remove it, and then another one to check whether a manager already is managing an employee. We can use this to check whether or not adding the employee can go through. Lastly, we have WriteDataStore method, that is just a helper method which will write out the data store to the console output window.

```

Below IEmployeeManagerRepository{
public class EmployeeManagerRepository : IEmployeeManagerRepository
{
    //for demo purposes, use an in-memory datastore as a fake "manager list"
    private List<Manager> _managers = new List<Manager>()
    {
        new Manager(1, "Katie"),
        new Manager(2, "George")
    };

    public void AddEmployee(int managerId, Employee employee)
    {
        //in real-life, add additional input and error checks
        _managers.First(m => m.Id == managerId).Employees.Add(employee);
    }

    public void RemoveEmployee(int managerId, Employee employee)
    {
        //in real-life, add additional input and error checks
        _managers.First(m => m.Id == managerId).Employees.Remove(employee);
    }

    public bool HasEmployee(int managerId, int employeeId)
    {
        //in real-life, add additional input and error checks
        return _managers.First(m => m.Id == managerId).Employees.Any(e => e.Id == employeeId);
    }

    public void WriteDataStore()
    {
        foreach (var manager in _managers)
        {
            Console.WriteLine($"Manager {manager.Id}, {manager.Name}");
            if (manager.Employees.Any())
            {
                foreach (var employee in manager.Employees)
                {
                    Console.WriteLine($"  \t Employee {employee.Id}, {employee.Name}");
                }
            }
        }
    }
}

```

```

        }
        else
        {
            Console.WriteLine($"{t} No employees.");
        }
    }
}

```

Now we got our plumbing code, and we also have the receiver. This receiver is the action, that will eventually be executed when the command gets executed. The first thing we need for that is an ICommand interface. That is the command from the pattern structure. In the pattern structure, it contained just one method, Execute. But these pattern should be seen as templates. In real-life implementations, additional methods or variations often exist, and this is such a case. We will immediately extend the pattern with an additional action, CanExecute. You can use this to check business rules that might block the command from executing. Executing the command and knowing whether it can be executed are both contained within the command itself.

```

Below EmployeeManagerRepository {}
public interface ICommand
{
    void Execute();
    bool CanExecute();
}

```

The next class is the concrete implementation of a command. We name that AddEmployeeToManagerList. We want it to implement the ICommand interface, and when we do that, we see that we have to implement CanExecute and Execute. Our command needs a few things to be able to correctly function. It needs access to our repository, but it also needs to know the manager ID and the employee. That is the state of the command. As we need all these for the command to correctly function, we can convey that by adding a constructor that accepts these. Like this, we state that the command shouldn't be constructed without them, and we store these in private readonly fields.

The CanExecute method, we will ensure that the command can only be executed if the employee isn't already an employee on the manager's employee list. We also add a null check to ensure that the employee we want to add isn't null. Additional rules can exist as well, like ensuring that an employee can only be managed by one manager at the time.

In the Execute method, we add the employee to the manager's employee list. But we get an warning, that states that employee may be null, and that is true. So we should add a null check in the execute method as well.

```

Below ICommand{}
public class AddEmployeeToManagerList : ICommand
{
    private readonly IEmployeeManagerRepository _employeeManagerRepository;
    private readonly int _managerId;
    private readonly Employee? _employee;

    public AddEmployeeToManagerList(IEmployeeManagerRepository employeeManagerRepository,
    int managerId, Employee? employee)
    {
        _employeeManagerRepository = employeeManagerRepository;
        _managerId = managerId;
    }
}

```

```

        _employee = employee;
    }

    public bool CanExecute()
    {
        //employee shouldn't be null
        if (_employee == null)
        {
            return false;
        }

        //employee shouldn't be on the manager's list already
        if (_employeeManagerRepository.HasEmployee(_managerId, _employee.Id))
        {
            return false;
        }
        return true;
    }

    public void Execute()
    {
        if (_employee == null)
        {
            return;
        }

        _employeeManagerRepository.AddEmployee(_managerId, _employee);
    }
}

```

Now we need something to revoke this request. In our example the command is invoked by a user clicking a button in the user interface. So the invoker could be that button, either via binding or by setting the command in some other way. But in this scenario we don't have an user interface, as we are building a console app. We can instead add a small manager class in between, a `CommandManager`, which is responsible for effectively invoking commands. Not just one command, but multiple commands throughout your application. In this demo, we can use that `CommandManager` as an invoker class. The `commandManager` class is straightforward. In real life the interface would call into the `commandmanager` class to execute commands. This way makes it easy to add in additional functionality, like undoing commands. In this iteration, the `CommandManager` simply acts as a passed-through class that doesn't really do anything, but we will build upon this.

Below `AddEmployeeToManagerList{}`

```

public class CommandManager
{
    public void Invoke(ICommand command)
    {
        if (command.CanExecute())
        {
            command.Execute();
        }
    }
}

```

The last part, the client. That is our Program class. Let's make a new instance of our command manager and our repository. For that we need to import the command namespace. And now let's create a command and invoke it. For that we use the commandManager. We call invoke on it, and we pass through a new command. To make a new instance, we need to pass through the repository, an id for the manager and an employee. Once the command has been invoked, we write out to data store so we can see what just happened. Let's give this a try!

In Program.cs

```
CommandManager commandManager = new(); //Quick Action to add using statement.  
IEmployeeManagerRepository repository = new EmployeeManagerRepository();
```

```
commandManager.Invoke(new AddEmployeeToManagerList(repository, 1, new Employee(111,  
"Kevin")));  
repository.WriteDataStore();
```

```
commandManager.Invoke(new AddEmployeeToManagerList(repository, 1, new Employee(222,  
"Clara")));  
repository.WriteDataStore();
```

```
commandManager.Invoke(new AddEmployeeToManagerList(repository, 2, new Employee(333,  
"Tom")));  
repository.WriteDataStore();
```

```
Console.ReadKey();
```

We have two managers, Katie and George. They start with no employees. First we add kevin, and katie has 1 employee while George still has 0. After our second command. Katie has 2 employees, while George still has 0. After the third command Katie has still 2 employees, and George has one. If we want to add the same employee again, the command will not be executed.

Demo - Supporting Undo with

Time to extend our example with undo functionality, and this is where our command manager will shine. To enable undo functionality, we need to add two things. Let's think about what undoing a command would mean. Seeing executing a command, in our case, means that we add an employee for a manager. Undoing this command will mean that we want to remove the employee from the manager. The undo action is straightforward. We call into RemoveEmployee under repository after a null check.

In implementation make change to AddEmployee...

```
public class AddEmployeeToManagerList : ICommand  
{  
    private readonly IEmployeeManagerRepository _employeeManagerRepository;  
    private readonly int _managerId;  
    private readonly Employee? _employee;  
  
    public AddEmployeeToManagerList(  
        IEmployeeManagerRepository employeeManagerRepository,  
        int managerId,  
        Employee? employee)  
    {  
        _employeeManagerRepository = employeeManagerRepository;  
        _managerId = managerId;  
        _employee = employee;  
    }  
}
```

```

    }

    public bool CanExecute()
    {
        // employee shouldn't be null
        if (_employee == null)
        {
            return false;
        }

        // employee shouldn't be on the manager's list already
        if (_employeeManagerRepository.HasEmployee(_managerId, _employee.Id))
        {
            return false;
        }

        // other potential rule(s): ensure that an employee can only be added to
        // one manager's list at the same time, etc.
        return true;
    }

    public void Execute()
    {
        if (_employee == null)
        {
            return;
        }
        _employeeManagerRepository.AddEmployee(_managerId, _employee);
    }

    public void Undo()
    {
        if (_employee == null)
        {
            return;
        }

        _employeeManagerRepository.RemoveEmployee(_managerId, _employee);
    }
}

```

On the command manager, we can then keep a list of commands. Seeing undoing means executing the undo action on each command in reverse order. A stack is a good structure to store these commands in due to the fact that it is a first in last out structure. So when a command has been executed, we push the command onto the stack, and then we can add one or more undo actions in our command manager. Say that we have an undo action that will reverse the last command. The first thing we do is check whether there is still something on the stack, and if there is, we call pop. That results in the last item on the stack being returned. If it is not null, we call undo on it.

Make changes to CommandManager:

```

public class CommandManager
{
    private readonly Stack<ICommand> _commands = new Stack<ICommand>();

```

```

    public void Invoke(ICommand command)
    {
        if (command.CanExecute())
        {
            command.Execute();
            _commands.Push(command);
        }
    }

    public void Undo()
    {
        if (_commands.Any())
        {
            _commands.Pop?.Undo();
        }
    }
}

```

Undo doesn't resolve, that is because we are working on ICommand, so the interface, and we only added the Undo action on command, but not on the ICommand interface. Add the undo command here as well. That will resolve it.

Changes to ICommand:

```

public interface ICommand
{
    void Execute();
    bool CanExecute();
    void Undo();
}

```

What we also should do is to add an UndoAll action. The idea is that this one will reverse all commands. It is a bit easier for the client to interact with the command manager like that.

Make changes to CommandManager:

```

public class CommandManager
{
    private readonly Stack<ICommand> _commands = new Stack<ICommand>();

    public void Invoke(ICommand command)
    {
        if (command.CanExecute())
        {
            command.Execute();
            _commands.Push(command);
        }
    }

    public void Undo()
    {
        if (_commands.Any())
        {

```



```

        _commands.Pop()?.Undo();
    }
}

public void UndoAll()
{
    while (_commands.Any())
    {
        _commands.Pop()?.Undo();
    }
}
}

```

Let's test this. After we add Kevin, we undo that, and we write out the data store again. And in the end, we undo all the actions we previously executed.

Changes to Program.cs

```
CommandManager commandManager = new(); //Quick Action to add using statement.
```

```
IEmployeeManagerRepository repository = new EmployeeManagerRepository();
```

```
commandManager.Invoke(new AddEmployeeToManagerList(repository, 1, new Employee(111,
"Kevin")));
repository.WriteDataStore();
```

```
commandManager.Undo();
repository.WriteDataStore();
```

```
commandManager.Invoke(new AddEmployeeToManagerList(repository, 1, new Employee(222,
"Clara")));
repository.WriteDataStore();
```

```
commandManager.Invoke(new AddEmployeeToManagerList(repository, 2, new Employee(333,
"Tom")));
repository.WriteDataStore();
```

```
commandManager.UndoAll();
repository.WriteDataStore();
```

```
Console.ReadKey();
```

When testing this. We see after the second command is executed, both managers has 0 employees. And after the final command both managers has again 0 employees.

Use Cases for the Command pattern

A very obvious use case for this pattern is when you want to parameterize objects with an action to perform. That is exactly what we did in our example. We turned an action, a button click, into a standalone object, which we can pass around.

But there is more potential use cases. Supporting undo is one of them, as we have covered in our second demo.

Also, consider this pattern when you want to specify, queue, and execute requests at different times. The lifetime of the command object can be independent of that of the original request. You can transfer the command object to another process and fulfill it there.

And lastly, you can also easily augment this pattern to store a list of executed changes so they can be reapplied in case of, for example, a system crash. To implement something like that, store or log the command object and re-execute it in case an issue happens.

Pattern Consequences

The main consequence for this pattern is that it decouples the class that invokes the operation from the one that knows how to perform it. This is the single responsibility principle from the SOLID coding principle at work.

Interesting to know is that commands can be manipulated and extended like any other object. Commands can also be assembled into a composite command. For example, a button which restarts creating a list of employees from the context of an employee might first execute a clear employee list command followed by an add employee to list command. In general these types of commands are an instance of the composite pattern.

Lastly, as far as the positive consequences go, it is easy to add new commands because you don't have to change existing classes. You simply add a new class that implements the required ICommand interface. That is the open/closed principle at work.

A potential negative consequence is complexity. By transferring requests into objects, we are adding an additional layer.

Related patterns

The first related pattern is a structural one, the composite pattern. This can be used to implement commands composed of other commands.

Memento, a behavioral pattern, can be used to store the state a command requires to undo its effect.

This pattern is also related to a creational pattern, the prototype pattern. That is because a command that must be copied before being placed on the history list, which is what you need in case you want to support undo, acts as a prototype.

There is also a relation between the chain of Responsibility and the command pattern. Handlers in the chain of responsibility can be implemented as commands.

- Composite
Can be used to implement commands composed of other commands.
- Memento
Can be used to store the state a command requires to undo its effect.
- Prototype
In case of supporting undo, a command that must be copied acts as a prototype.
- Chain of Responsibility
Handlers can be implemented as commands.

When we look at some of the other patterns in this course, we find others that connects senders and receivers of requests. In that regard, chain of responsibility, command, mediator, and observer are all related.

Chain of responsibility passes a request along a chain of receivers that can be defined at runtime until at least one handles it.

Command connects senders with receivers unidirectionally, for example, from a button that is pushed in the UI to the action that finally gets executed.

The mediator pattern eliminates direct connections altogether. Communication runs through the mediator.

And lastly, the observer allows receivers of requests to subscribe and unsubscribe to them at runtime. So all of these patterns connects senders and receivers, but they all do it in different ways solving different problems.

Chain of Responsibility	Command	Mediator	Observer
Passes a request along a chain of receivers	Connects senders with receivers unidirectionally	Eliminates direct connections altogether	Allows receivers of requests to (un)subscribe at runtime

Behavioral Pattern: Memento

Monday, October 31, 2022 1:45 PM

Coming up

In this module we will cover the memento pattern, sometimes called the token pattern. It is all about capturing and externalizing an object's internal state.

We will start by describing the pattern using a real-life example. As a real-life example, for once we will build upon something from a previous module. We will use the memento pattern to store a command's internal state to support undo actions. We will map that to the structural design pattern, and we will implement this real-life example in a demo. After that we will continue with learning about use cases, pattern consequences, and related patterns.

Describing the Memento Pattern

The memento pattern is a behavioral pattern from the Gang of Four library of design patterns. The intent of the memento is to capture and externalize an object's internal state so the object can be restored to that state later without violating encapsulation. In other words this pattern will allow us to store and restore private field and property values. Let's clarify that with a real-life example. We are going to use undo functionality of commands. This pattern is not used that much these days. You can look at command as a class that has some internal state, so private state, that needs to be stored to be able to undo the command. It is easy enough to store public state, but private state is different.

An example we previously used was a command that added an employee to a manager's list of employees. That command needed to know about the employee to add, about the manager, and it also needed a repository through which interaction with the underlying data store could happen.

```
public class AddEmployeeToManagerList : ICommand
{
    private readonly IEmployeeManagerRepository _employeeManagerRepository;
    private readonly int _managerId;
    private readonly Employee? _employee;

    //Implementation
}
```

The manager ID and the employee, that is internal state. We used the CommandManager that was responsible for executing the command and storing necessary information about the command to undo it. We did that by storing the command object itself in memory. And while that works, this could mean we would be storing too much information. We don't necessarily need to store the full command object.

```
public class CommandManager
{
    private readonly Stack<ICommand> _commands = new Stack<ICommand>();

    public void Invoke(ICommand command)
    {
        //Implementation
    }

    public void Undo()
```

```

    {
        //Implementation
    }
}

```

We simply need to store its state or part of its state. Let's switch back to that command implementation. If that is what we want, we could make the state public and access it as such to store it somewhere. Yet, this breaks the command's encapsulation, and that is what the memento solves. It allows saving and restoring internal state without breaking encapsulation.

```

public class AddEmployeeToManagerList : ICommand
{
    private readonly IEmployeeManagerRepository _employeeManagerRepository;
    public readonly int _managerId;
    public readonly Employee? _employee;

    //Implementation
}

```

Let's look at our command again. The internal state we want to store consists of the employee and the manager ID. We don't need to store a reference to the repository or anything. That remains the same for all AddEmployeeToManagerList commands. The first thing that we will need is a memento class that can contain this state. A good name would be AddEmployeeToManagerListMemento, containing the state we want to store via properties.

AddEmployeeToManagerListMemento

On the command itself, AddEmployeeToManagerList, we would then add two methods, one to create a memento and return it, and one to restore a memento. That is a method that accepts the memento and is responsible for restoring the command's state.

AddEmployeeToManagerList -----> AddEmployeeToManagerListMemento

And now we need something that manages this. That is our CommandManager. In an implementation we ended up with in the previous module, we stored commands on a stack. We no longer have to do that. This time, we can store the mementos on the stack, and doing a command then means that we take one existing AddEmployeeToManagerList command, restore the memento on it, and call the undo action.

AddEmployeeToManagerList -----> AddEmployeeToManagerListMemento

```

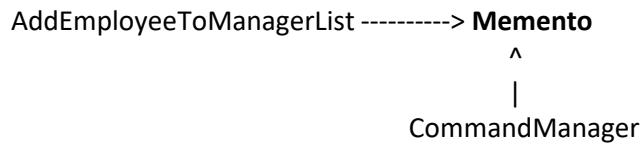
      ^
      |
CommandManager

```

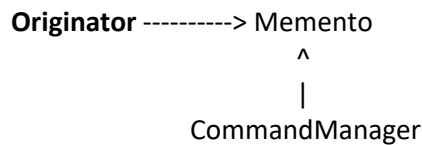
Important here is that we want to provide two different interfaces to this memento, one wider interface that is used by the command to create a memento. This passes through state and has access to set the state. The other interface to the memento is used by the CommandManager. That one stores these mementos and passes them through to the command. But it cannot change the memento itself anymore after it has been created. It has access to the getters but not to the setters, so that is a narrow interface. We can achieve that with private setters, as we will see in the implementation.

Structure of the Memento Pattern

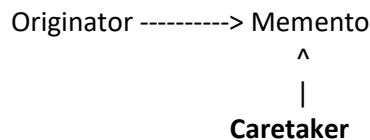
We got three players in this pattern. The first one is the memento itself, which maps to our `AddEmployeeToManagerListMemento` class. It is responsible for storing the internal state of the originator object. It may store as much or as little of the originator state as is necessary. Important here is that we try to protect the state against access by other objects as much as possible. That is that wide versus narrow interface explanation we just covered.



And then we have the originator that maps to our `AddEmployeeToManagerList` command. It is responsible for creating a memento with a snapshot of its current internal state. It also uses the memento to restore its internal state. For example, via a `restoreMemento` method.



Lastly, we have the caretaker. This object is responsible for the memento's safekeeping. It shouldn't operate on or examine the memento's content. It simply passes it through to the originator. That is what the command manager does. Such a caretaker can work on one or more mementos. In our use case, the `CommandManager` is a caretaker that works on multiple mementos.



Demo - Implementing the Memento pattern

We will start this demo with copying over the `Implementation` and `Program` classes from the previous module.

In `Implementation.cs` - Copied from `Command` implementation

```

public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }

    public Employee(int id, string name)
    {
        Id = id;
        Name = name;
    }
}

public class Manager : Employee
{
    public List<Employee> Employees = new();
    public Manager(int id, string name) : base(id, name)
    {

```

```

    }
}

public interface ICommand
{
    void Execute();
    bool CanExecute();
    void Undo();
}

public class AddEmployeeToManagerList : ICommand
{
    private readonly IEmployeeManagerRepository _employeeManagerRepository;
    private readonly int _managerId;
    private readonly Employee? _employee;

    public AddEmployeeToManagerList(IEmployeeManagerRepository employeeManagerRepository,
    int managerId, Employee? employee)
    {
        _employeeManagerRepository = employeeManagerRepository;
        _managerId = managerId;
        _employee = employee;
    }

    public bool CanExecute()
    {
        //employee shouldn't be null
        if (_employee == null)
        {
            return false;
        }

        //employee shouldn't be on the manager's list already
        if (_employeeManagerRepository.HasEmployee(_managerId, _employee.Id))
        {
            return false;
        }
        return true;
    }

    public void Execute()
    {
        if (_employee == null)
        {
            return;
        }

        _employeeManagerRepository.AddEmployee(_managerId, _employee);
    }

    public void Undo()
    {

```

```

        if (_employee == null)
        {
            return;
        }

        _employeeManagerRepository.RemoveEmployee(_managerId, _employee);
    }
}

```

```

public class CommandManager
{
    private readonly Stack<ICommand> _commands = new Stack<ICommand>();

    public void Invoke(ICommand command)
    {
        if (command.CanExecute())
        {
            command.Execute();
            _commands.Push(command);
        }
    }

    public void Undo()
    {
        if (_commands.Any())
        {
            _commands.Pop()?.Undo();
        }
    }

    public void UndoAll()
    {
        while (_commands.Any())
        {
            _commands.Pop()?.Undo();
        }
    }
}

```

```

public interface IEmployeeManagerRepository
{
    void AddEmployee(int managerId, Employee employee);
    void RemoveEmployee(int managerId, Employee employee);
    bool HasEmployee(int managerId, int employeeId);
    void WriteDataStore();
}

```

```

public class EmployeeManagerRepository : IEmployeeManagerRepository
{
    //for demo purposes, use an in-memory datastore as a fake "manager list"
    private List<Manager> _managers = new List<Manager>()
    {
        new Manager(1, "Katie"),
    }
}

```



```

        new Manager(2, "George")
    };

    public void AddEmployee(int managerId, Employee employee)
    {
        //in real-life, add additional input and error checks
        _managers.First(m => m.Id == managerId).Employees.Add(employee);
    }

    public void RemoveEmployee(int managerId, Employee employee)
    {
        //in real-life, add additional input and error checks
        _managers.First(m => m.Id == managerId).Employees.Remove(employee);
    }

    public bool HasEmployee(int managerId, int employeeId)
    {
        //in real-life, add additional input and error checks
        return _managers.First(m => m.Id == managerId).Employees.Any(e => e.Id == employeeId);
    }

    public void WriteDataStore()
    {
        foreach (var manager in _managers)
        {
            Console.WriteLine($"Manager {manager.Id}, {manager.Name}");
            if (manager.Employees.Any())
            {
                foreach (var employee in manager.Employees)
                {
                    Console.WriteLine($"    \t Employee {employee.Id}, {employee.Name}");
                }
            }
            else
            {
                Console.WriteLine($"    \t No employees.");
            }
        }
    }
}

```

In Program.cs - Copied from Command implementation

CommandManager commandManager = new(); //Quick Action to add using statement.

IEmployeeManagerRepository repository = new EmployeeManagerRepository();

```

commandManager.Invoke(new AddEmployeeToManagerList(repository, 1, new Employee(111,
"Kevin")));
repository.WriteDataStore();

```

```

commandManager.Undo();
repository.WriteDataStore();

```

```

commandManager.Invoke(new AddEmployeeToManagerList(repository, 1, new Employee(222,

```

```
"Clara"))));  
repository.WriteDataStore();
```

```
commandManager.Invoke(new AddEmployeeToManagerList(repository, 2, new Employee(333,  
"Tom"))));  
repository.WriteDataStore();
```

```
commandManager.UndoAll();  
repository.WriteDataStore();
```

```
Console.ReadKey();
```

Now let's add a Memento class to hold the internal state of our AddEmployeeToManagerList command. We name it AddEmployeeToManagerListMemento. Seeing we want to hold the manager ID and employee, we give it two properties, a ManagerId and an Employee. We also give these properties private setters. Like that, the caretaker will not be able to modify the memento. Seeing our properties have private setters, that means we can only set them via the constructor, so we do have to add a constructor which accepts the manager ID and the employee, but that makes sense because a memento without these fields doesn't make sense. So, it is a good idea to convey that meaning via the constructor.

```
In implementation.cs above AddEmployeeToManagerList : ICommand {}  
public class AddEmployeeToManagerListMemento  
{  
    public int ManagerId { get; private set; }  
    public Employee? Employee { get; private set; }  
  
    public AddEmployeeToManagerListMemento(int managerId, Employee? employee)  
    {  
        ManagerId = managerId;  
        Employee = employee;  
    }  
}
```

And now we need to extend our command a little bit. The first thing we need is a method that will create a memento, so we name that CreateMemento in which we create an instance of our AddEmployeeToManagerListMemento. And now we need a method to restore the state of the command. We name that RestoreMemento. This is used to restore our command's state and it accepts the memento. This does mean that we no longer keep the manager ID and employee fields at the level of our command readonly, as we now have to be able to change them outside of the constructor. And that is it for our command class, in other words, the originator.

```
Make changes to AddEmployeeToManagerList : ICommand {}  
public class AddEmployeeToManagerList : ICommand  
{  
    private readonly IEmployeeManagerRepository _employeeManagerRepository;  
    private int _managerId;  
    private Employee? _employee;  
  
    public AddEmployeeToManagerList(IEmployeeManagerRepository employeeManagerRepository,  
int managerId, Employee? employee)  
    {  
        _employeeManagerRepository = employeeManagerRepository;
```

```

        _managerId = managerId;
        _employee = employee;
    }

    public AddEmployeeToManagerListMemento CreateMemento()
    {
        return new AddEmployeeToManagerListMemento(_managerId, _employee);
    }

    public void RestoreMemento(AddEmployeeToManagerListMemento memento)
    {
        _managerId = memento.ManagerId;
        _employee = memento.Employee;
    }

    public bool CanExecute()
    {
        //employee shouldn't be null
        if (_employee == null)
        {
            return false;
        }

        //employee shouldn't be on the manager's list already
        if (_employeeManagerRepository.HasEmployee(_managerId, _employee.Id))
        {
            return false;
        }
        return true;
    }

    public void Execute()
    {
        if (_employee == null)
        {
            return;
        }

        _employeeManagerRepository.AddEmployee(_managerId, _employee);
    }

    public void Undo()
    {
        if (_employee == null)
        {
            return;
        }

        _employeeManagerRepository.RemoveEmployee(_managerId, _employee);
    }
}

```

What is left is the caretaker, that is our command manager. As you notice, once you start combining

different patterns, which is often the case, the same class will often be used by the different patterns, thus mapping two different component names from the pattern structure. The first thing we need to do here is change the stack. We no longer want to store commands on it, we want to store mementos on it. So, it is not a `Stack<ICommand>` anymore, it is a `Stack<AddEmployeeToManagerListMemento>`. And make a `new()` of it. We also want to add one command field. We need that to hold our `AddEmployeeToManagerList` command. This command will be reused. Also we don't instantiate it yet. We didn't store a reference to the repository in our memento as that remains the same overall command instances in case you need more.

Therefore, we need to set this field to the first instance of the command that is created by the client so we can get ahold of the correct repository reference. That is what we do in the `invoke` method. Also note that we do have to cast the incoming `ICommand` to an `AddEmployeeToManagerList` command to be able to store it. We are now, tying this to concrete implementation. In a real-life scenario this might become a bit more difficult to manage, but we will get to this and other disadvantages right after the implementation.

Next to storing our command, we also push a memento on the stack by calling `_mementos.Push`. Now, as you notice calling `CreateMemento` on command doesn't resolve. That is because `CreateMemento` is implemented on the concrete command class, while command is of type `ICommand`, the underlying contract or interface. That means that we need to cast here as well. And there we go, now the code does resolve. But as you have noticed, we are now tying the `invoke` method to one specific type. That is a bit of a disadvantage.

When we want to undo the last one, we first check whether mementos exist in the list, and if one does, we restore that memento on the command by calling into `RestoreMemento`, passing through the memento we popped from the stack. Now, the state of the command is that of the memento, and when we call `Undo`, we effectively undo the correct state change. For undoing all commands we do the same, but in a loop that loops through all mementos.

Make changes `CommandManager`{

```
public class CommandManager
```

```
{
```

```
    private readonly Stack<AddEmployeeToManagerListMemento> _mementos = new();
```

```
    private AddEmployeeToManagerList? _command;
```

```
    public void Invoke(ICommand command)
```

```
    {
```

```
        //If the command has not been stored yet, store it - we will reuse it instead of storing
        different instances
```

```
        if (_command == null)
```

```
        {
```

```
            _command = (AddEmployeeToManagerList)command;
```

```
        }
```

```
        if (command.CanExecute())
```

```
        {
```

```
            command.Execute();
```

```
            _mementos.Push(((AddEmployeeToManagerList)command).CreateMemento());
```

```
        }
```

```
    }
```

```
    public void Undo()
```

```
    {
```

```

        if (_mementos.Any())
        {
            _command?.RestoreMemento(_mementos.Pop());
            _command?.Undo();
        }
    }

    public void UndoAll()
    {
        while (_mementos.Any())
        {
            _command?.RestoreMemento(_mementos.Pop());
            _command?.Undo();
        }
    }
}

```

Let's try this out. We do not need to change anything on the client code. As far as that is concerned, all of this is of no importance. The Program.cs code we copied from command module remains the same. The results are the same as in the last demo in Command.

So we are now using mementos, but you can already see a negative consequence here. The command that is reused is always the same command. If you want this to work with different commands, which you typically want the command manager to do, you will have to implement support for that. Moreover, you will also need memento classes for each command, and it would also be nice to find a solution for all this casting. So this has a tendency to become a hassle pretty fast.

Use Cases for the Memento pattern

There is just one main use case for this pattern consisting of two parts. Use it when an object's state or part of it must be saved so that it can be restored to that state later, and a direct interface to obtaining the state would expose implementation details and break the object encapsulation. We had such a use case. We avoid exposing implementation details of our command through a memento, or at least we avoid changing the internal values.

Pattern Consequences

A major consequence of this pattern is that it preserves encapsulation boundaries. Information that only an originator should manage, but must be stored outside of the originator, is no longer exposed.

It also simplifies the originator. It no longer needs to concern itself with storage management.

But using mementos might be expensive if the originator must copy over large amounts of information to store in the memento or if clients create and return mementos to the originator often. So make sure that encapsulating and restoring originator state is cheap because otherwise the pattern might not be appropriate.

And last, but not least, it also tends to complicate things. We noticed that with the command manager in our example. We are now just using one command on which we change state for undo functionality, but code tends to become cluttered and/or complex when you need to work with many different types of state that you need to store and restore.

- It preserves encapsulation boundaries
- It simplifies the originator
- Using mementos might be expensive
- It can introduce complexity to your code base

Related patterns

The first related pattern is the command pattern as it can use a memento to store and restore its state.

Next to that mementos can also be used for iteration. In that case, a memento can be used to capture current iteration state and potentially roll it back.

- Command
Can use a memento to store and restore its state
- Iterator
Memento can be used to capture the current iteration state and potentially roll it back.

Behavioral Pattern: Mediator

Monday, October 31, 2022 1:46 PM

Coming up

In this module we will cover the mediator pattern. Its intent is to define an object, the mediator, that encapsulates how a set of objects interact.

We will start by describing the pattern using a real-life example. The archetypical example for a mediator is a chatroom, a common use case which everyone can easily understand. And chances are, you will have to implement something along these lines during your career. We will map that to the design pattern structure, and implement it in a demo. We will continue to learn about use cases for this pattern and consequences. We will finish the module with related patterns.

Describing the Mediator Pattern

The mediator pattern is a behavioral pattern from the Gang of Four library of design patterns.

The intent of the mediator pattern is to define an object, which is the mediator, that encapsulates how a set of objects interact. It does that by forcing objects to communicate via that mediator. By doing so, it promotes loose coupling by keeping objects from referring to each other explicitly, and it also lets you vary their interaction independently.

For example, a set of objects that need to interact with each other. To be able to do that, these objects need to hold references to each other. If we are talking about just a few objects, that is still okay. But imagine that these are not a few objects, but tens, hundreds, or thousands of objects. Holding on to all of these references and managing them and keeping communication in sync becomes a real issue, really fast. Then we enter the mediator pattern. What this pattern does is provide a central object, which is the mediator. It maintains references of objects that want to communicate with each other, and it handles communication between them. By using the mediator pattern all this becomes more manageable.

Let's map this to a real-life example. Imagine we are writing software for a team of people working at a bank, amongst them, account managers that sell banking products and lawyers that check these products for compliance. A part of the software we need to write is a chat application which needs to be custom built due to high security and confidentiality requirements. We could now create TeamMember objects that hold references to all other TeamMember objects that need to communicate with each other, but that is exactly what we don't want to do.

```
public class Lawyer : TeamMember
{
    private List<TeamMember> _teamMembersInChat = new();
}

public class AccountManager : TeamMember
{
    private List<TeamMember> _teamMembersInChat = new();
}
```

- We want to avoid holding references between team member objects.

Instead we will use the mediator pattern. We start with one central object, the ChatRoom. This is

abstract. It defines the interface for communicating with team members. Remember, as usual with these patterns and as detailed in the introduction module, these patterns want to encourage working on interfaces and not on implementations. Technically, that means either an abstract class or an interface.

IChatRoom

A concrete implementation of this would then be named TeamChatRoom.

```

IChatRoom
|
|
TeamChatRoom

```

And now we have a TeamMember, again, an abstract class which holds a reference to the abstract ChatRoom interface. It needs this to be able to communicate with it. So instead of communicating directly with other team members, it communicates with the ChatRoom.

```

IChatRoom -----> TeamMember
|
|
TeamChatRoom

```

Our lawyers and account managers are then concrete implementations of the TeamMember abstract base class. These are the objects that effectively receive and send messages through the ChatRoom. This TeamChatRoom thus holds the actual references to the team members.

```

IChatRoom -----> TeamMember
|               |               |
|               |               |
|               |-----> AccountManager
TeamChatRoom -----> Lawyer

```

Structure of the Mediator Pattern

The abstract chatroom class, that is our mediator. This defines an interface for communicating with colleague objects.

```

Mediator -----> TeamMember
|               |               |
|               |               |
|               |-----> AccountManager
TeamChatRoom -----> Lawyer

```

Such a colleague maps to our team member abstract class.

```

Mediator -----> Colleague
|               |               |
|               |               |
|               |-----> AccountManager
TeamChatRoom -----> Lawyer

```

Whereas account manager and lawyer are ConcreteColleague classes. They know their mediator, and they communicate with it when they would have otherwise communicated with another colleague.

```

Mediator -----> Colleague
|               |               |

```



```

    |           |-----> ConcreteColleague1   |
TeamChatRoom -----> ConcreteColleague2

```

Lastly, we have the concrete mediator, our TeamChatRoom. This derives from mediator. It knows and maintains its colleagues, and it implements cooperative behavior by coordinating colleague objects. It this, routes messages between them.

```

Mediator -----> Colleague
    |           |           |           |
    |           |-----> ConcreteColleague1   |
ConcreteMediator -----> ConcreteColleague2

```

Demo - Implementing the Mediator pattern

We will start this demo with the mediator object itself, the ChatRoom, an abstract base class of which we will require a concrete implementation later on. The functionality required is sending a message, so we add an abstract method for that. The parameters we want to accept here really depend on your use case. What is accepted here is what the mediator has access to and can thus communicate to the team members or use for additional logic. A from parameter and a message parameter seemed like a good idea for our demo. We currently don't support sending something to a specific member or set of members. The other type of functionality we need is registering users for our chatroom, so we need a register method. As a parameter, it accepts a TeamMember that needs to be registered.

```

In Implementation.cs
public abstract class ChatRoom
{
    public abstract void Register(TeamMember teamMember);

    public abstract void Send(string from, string message);
}

```

We have used an abstract base class here, but as we didn't provide default implementations, you could technically implement this as an interface as well. So we will switch this around a bit. and make ChatRoom to an interface instead of an abstract class. As when you don't have default implementations, there really is no need to work with abstract classes.

```

public interface IChatRoom
{
    void Register(TeamMember teamMember);

    void Send(string from, string message);
}

```

Let's add a TeamMember class. That is the colleague from the pattern structure. It needs to hold a reference to our mediator as it is through this mediator that it communicates with other team members. In principle this mediator doesn't need to be accessible from outside of this class. We can have this class send the message through the mediator and have subclasses simply use the base method we define. We also add a name property to hold the member's name. To convey that we don't want these to be null, we add a constructor which accepts both the name and the IChatRoom implementing instance.

```

public abstract class TeamMember
{

```

```

private IChatRoom _chatroom;

public string Name { get; set; }

public TeamMember(string name, IChatRoom chatroom)
{
    Name = name;
    _chatroom = chatroom;
}
}

```

But there is a potential issue with this. It is the IChatRoom implementation that sets the mediator for each team member when team members are registered on it. So, at that time, the team members are already constructed, so we need to change this a bit. We make IChatRoom nullable, and we add an internal method to set the chatroom instance to avoid this being possible from outside of our assembly. We no longer accept a chatroom via our constructor.

And then we need it to implement the method to send messages. So, we add a send method, and in it we accept a message and use the mediator to send it. The from parameter is the name of the team member. Team members don't only send messages. They also need to be able to receive them. So that is the last method we want to add here.

A concrete mediator will call this method on each team member it manages to communicate messages to them. For our demo, we will write out the message we receive to the console output window. What we also want to do is mark this method as virtual. By doing so, we allow concrete implementations to override it and provide their own specific implementation if needed. So we are working on an abstract class here and this time, it does contain implementation details. We cannot switch this out for an interface.

```

public abstract class TeamMember
{
    private IChatRoom? _chatroom;

    public string Name { get; set; }

    public TeamMember(string name)
    {
        Name = name;
    }

    internal void SetChatroom(IChatRoom chatroom)
    {
        _chatroom = chatroom;
    }

    public void Send(string message)
    {
        _chatroom?.Send(Name, message);
    }

    public virtual void Receiver(string from, string message)
    {
        Console.WriteLine($"Message {from} to {Name}: {message}");
    }
}

```

```

    }
}

```

Then let's add a concrete implementation of this TeamMember abstract base class, the Lawyer. We call into the base constructor, and we override the receive method. We marked this virtual so we are allowed to do that. Like that, we can write out which concrete class received the message. Next to that, we also call into base.Receive to write out a new message itself.

```

public class Lawyer : TeamMember
{
    public Lawyer(string name) : base(name){}

    public override void Receiver(string from, string message)
    {
        Console.WriteLine($"{nameof(Lawyer)} {Name} received: ");
        base.Receive(from, message);
    }
}

```

And then we do something likewise for the AccountManager class. And with, that we have got our two team members.

```

public class AccountManager : TeamMember
{
    public AccountManager(string name) : base(name) {}

    public override void Receiver(string from, string message)
    {
        Console.WriteLine($"{nameof(AccountManager)} {Name} received: ");
        base.Receive(from, message);
    }
}

```

Now we can continue with the concrete mediator implementation. That is our team chatroom, which implements the IChatRoom interface. So we chose to implement it, and that means we need to provide implementations for two methods, Register and Send.

Team members are registered via the Register method. Seeing our team chatroom needs to know which members to send messages to, it needs to store them. There is multiple ways of doing that, collections, lists, whichever construction you want to use. As I happen to know that I am going to want to search for people by name in the next demo, I am going to use a dictionary.

In the Register method, we then link the mediator to the team member by calling into SetChatroom. When we call into it, we pass through the mediator itself, this.SetChatroom expects an IChatRoom implementing object, which in this case, is our TeamChatRoom instance. After that we add it to the dictionary. This does assume that the name must be unique as we are using it as key. If that is not something you desire, use another key or another collection construct.

And then it is the send method. Sending a message through the mediator assumes that all members must receive it in this scenario. So we run through all of them and call receive on them.

```

public class TeamChatRoom : IChatRoom
{

```

```

private readonly Dictionary<string, TeamMember> teamMembers = new();
public void Register(TeamMember teamMember)
{
    teamMember.SetChatroom(this);
    if (!teamMembers.ContainsKey(teamMember.Name))
    {
        teamMembers.Add(teamMember.Name, teamMember);
    }
}

public void Send(string from, string message)
{
    foreach (var teamMember in teamMembers.Values)
    {
        teamMember.Receiver(from, message);
    }
}
}

```

Let's test this! First we create a new instance of the TeamChatRoom. For that, we need to import the Mediator namespace. Then we create a few team members so they can chat with each other. We will add two lawyers, Sven and Kenneth, and three account managers Ann, John and Kylie. These are the five team members that will chat with each other, so we need to register them on our chatroom. For that, we call into register on our TeamChatRoom instance for each of them. And then let's see what happens when we send a message. Let's say that Ann is going to send a message because she needs a compliance check on a file. Sven sees the message and says he is on it. Finish the code with a ReadKey statement. Then run the test.

In Program.cs

```
TeamChatRoom teamChatroom = new();
```

```

var sven = new Lawyer("Sven");
var kenneth = new Lawyer("Kenneth");
var ann = new AccountManager("Ann");
var john = new AccountManager("John");
var kylie = new AccountManager("Kylie");

```

```

teamChatroom.Register(sven);
teamChatroom.Register(kenneth);
teamChatroom.Register(ann);
teamChatroom.Register(john);
teamChatroom.Register(kylie);

```

```

ann.Send("Hi everyone, can someone have a look at file ABC123? I need a compliance check.");
sven.Send("On it!");

```

```
Console.ReadKey();
```

As we see all team members got both the messages. So our mediator works as expected. But maybe not everyone should receive every message all the time.

Demo - Supporting Communication between specific objects

A common requirement when creating a system that requires communication between lots of different objects is limiting communication to specific objects only. Mapped to our system, we could say we want to support functionality in which a team member can choose to only communicate with a specific person by name or another member from the same type, for example lawyers that only want to chat with lawyers and don't want their messages to be delivered to account managers and vice versa.

We will start with the first scenario, sending messages to a specific person. In the previous demo we would use the name as key for the dictionary, making it unique. We can use that same name to select the person to send the message to. So the first thing we need is a method that accepts this parameter on the mediator.

In implementations.cs

Change IChatRoom:

```
public interface IChatRoom
{
    void Register(TeamMember teamMember);

    void Send(string from, string message);

    void Send(string from, string to, string message);
}
```

And then we need to implement this on the concrete mediator. That is our TeamChatRoom. Let's implement the interface, and here is the method we need to implement. We want to send the message to a specific team member. And seeing that TeamMembers is a dictionary, which stores team members with name as key, we use the dictionary indexer or to get the correct team member. If it is not null, we call Receive on it. We have now ensured that the correct team member will receive a message.

Change to TeamChatRoom

```
public class TeamChatRoom : IChatRoom
{
    private readonly Dictionary<string, TeamMember> teamMembers = new();

    public void Register(TeamMember teamMember)
    {
        teamMember.SetChatroom(this);
        if (!teamMembers.ContainsKey(teamMember.Name))
        {
            teamMembers.Add(teamMember.Name, teamMember);
        }
    }

    public void Send(string from, string message)
    {
        foreach (var teamMember in teamMembers.Values)
        {
            teamMember.Receive(from, message);
        }
    }

    public void Send(string from, string to, string message)
    {
        var teamMember = teamMembers[to];
        teamMember?.Receive(from, message);
    }
}
```

```

    }
}

```

What is left now is ensuring that a team member can send such a message. In the TeamMember abstract base class and add an additional method on it, a Send method that accepts a to string. The only thing that this method does is pass it through to the sentiment that we just implemented on our chat room, and that is it already.

Change to TeamMember:

```

public abstract class TeamMember
{
    private IChatRoom? _chatroom;

    public string Name { get; set; }

    public TeamMember(string name)
    {
        Name = name;
    }

    internal void SetChatroom(IChatRoom chatroom)
    {
        _chatroom = chatroom;
    }

    public void Send(string message)
    {
        _chatroom?.Send(Name, message);
    }

    public void Send(string to, string message)
    {
        _chatroom?.Send(Name, to, message);
    }

    public virtual void Receiver(string from, string message)
    {
        Console.WriteLine($"Message {from} to {Name}: {message}");
    }
}

```

Let's test this. We want to have Sven send a message to Ann to jump on a Teams call to talk about the file he is checking. This message we sent was only delivered to Ann.

In program above ReadKey()
sven.Send("Ann", "Could you join me in a Teams call?");

Another useful scenario would be sending a message to all team members of a specific type. The implementation is not that different from what we just did, save for the fact that we can use some generics for that, which is why we will do it.

The first thing we need is another method on our chat room. We want to pass through a type, lawyer or account manager. We can use generics for that. The type must derive from the TeamMember class, so

we pass that as a restriction.

Change to IChatRoom

```
public interface IChatRoom
{
    void Register(TeamMember teamMember);

    void Send(string from, string message);

    void Send(string from, string to, string message);

    void SendTo<T>(string from, string message) where T : TeamMember;
}
```

Let's implement the interface. We see that we need to implement one additional method. In this method, we want to run through all team members of a specific type, and the type they are of, that is T, the type that is passed through. So we will do that. We get all Values.Of<T> from our dictionary, and we let them receive the message.

Change to TeamChatRoom

```
public class TeamChatRoom : IChatRoom
{
    private readonly Dictionary<string, TeamMember> teamMembers = new();

    public void Register(TeamMember teamMember)
    {
        teamMember.SetChatroom(this);
        if (!teamMembers.ContainsKey(teamMember.Name))
        {
            teamMembers.Add(teamMember.Name, teamMember);
        }
    }

    public void Send(string from, string message)
    {
        foreach (var teamMember in teamMembers.Values)
        {
            teamMember.Receive(from, message);
        }
    }

    public void Send(string from, string to, string message)
    {
        var teamMember = teamMembers[to];
        teamMember?.Receive(from, message);
    }

    public void SendTo<T>(string from, string message) where T : TeamMember
    {
        foreach (var teamMember in teamMembers.Values.Of<T>())
        {
            teamMember.Receive(from, message);
        }
    }
}
```

```

    }
}

```

All that is left is one more method on our abstract TeamMember base class. Just like we added a method to send a message to a specific person, we now add a method to send a message to a specific type, which calls into the Send method on the underlying chat room.

Changes to TeamMember

```

public abstract class TeamMember
{
    private IChatRoom? _chatroom;

    public string Name { get; set; }

    public TeamMember(string name)
    {
        Name = name;
    }

    internal void SetChatroom(IChatRoom chatroom)
    {
        _chatroom = chatroom;
    }

    public void Send(string message)
    {
        _chatroom?.Send(Name, message);
    }

    public void Send(string to, string message)
    {
        _chatroom?.Send(Name, to, message);
    }

    public void SendTo<T>(string message) where T : TeamMember
    {
        _chatroom?.SendTo<T>(Name, message);
    }

    public virtual void Receiver(string from, string message)
    {
        Console.WriteLine($"Message {from} to {Name}: {message}");
    }
}

```

Let's test this! Once Sven lets Ann know that the file is good to go, Ann wants to communicate this to all other account managers. She can use the SendTo method, passing through AccountManager as type for that.

```

In Program.cs above ReadKey()
sven.Send("Ann", "All good :)");
ann.SendTo<AccountManager>("The file was approved!");

```


The test shows that Ann's message went to all other account managers.

Alternative: MediatR

What we are looking at now is MediatR, a very popular .NET implementation of the mediator pattern made by Jimmy Bogard, the guy who also made automapper.

This project is on NuGet ([gothub.com/jbogard/MediatR](https://github.com/jbogard/MediatR)), and it supports request/response, commands, queries, notifications, and events, synchronous and asynchronous with intelligent dispatching via C# generic variance. The way Jimmy implemented it does differ from our pattern implementation. Ours is closer to the original design, but that is okay.

As mentioned a few times, pattern implementations can and will differ. As long as the principle and the intent remain the same, that's okay, and that is the case with MediatR.

Use Cases for the Mediator pattern

There are three good use cases when applying this pattern is something to consider.

First, when a set of objects communicate in well-defined but complex ways, which results in interdependencies that are unstructured and difficult to understand.

Two, when, because an object refers to and communicates with many other objects, it is difficult to reuse, in other words, when you got tight coupling between a set of likewise classes.

Both the first and second apply to our real-life chatroom example.

And lastly, when behavior that is distributed between several classes should be customizable without a lot of subclassing. In that case the mediator pattern can avoid a lot of that subclassing. What this means is that the mediator localizes behavior that would otherwise have to be distributed amongst different classes, and it does that because all relations between components are contained within the mediator. From that follows that when using the mediator, the colleague classes can be reused as is when the behavior has to be customized. It is contained within the mediator without requiring changes to or subclassing of the colleague classes.

Pattern Consequences

There are five important potential consequences to this pattern.

First of all, as we just detailed as last use case in the previous clip, it limits subclassing.

A second consequence is that it decouples colleagues. Thanks to the mediator, they are now loosely coupled, instead of requiring the colleagues to know about each other.

It also simplifies object protocols. What is meant by that is that interactions between objects that would normally be many-to-many interactions are replaced with one-to-many interactions between the mediator and its colleagues. These are simpler to maintain, extend, or understand. The open/closed principle also applies.

New mediators can be introduced without having to change the components. And lastly, a disadvantage, mediator centralizes control, which, as a consequence, can make the mediator complex. That is a disadvantage, because it could turn your mediator in a monolith that is hard to maintain.

- Limits subclassing
- Decouples colleagues
- Simplifies object protocols

- New mediators can be introduced without having to change the components.
- It centralizes control, which can make the mediator turn into a monolith.

Related patterns

The first related pattern is a structural one, the facade pattern. Both mediator and facade organize collaboration between a lot of normally tightly coupled classes. But there are differences. Mediator, like facade, abstracts functionality of existing classes, but it is not the same. The purpose of the mediator is abstracting communication between objects, often centralizing functionality that doesn't belong in those objects. Facade, on the other hand, simply abstracts the interface to the subsystem objects to promote ease of use.

- Facade
Mediator abstracts communication between objects. Facade abstracts the interface to the subsystem objects to promote ease of use.

When we look at some of the other patterns in this course, we find others that connect senders and receivers of requests.

In that regard, chain of responsibility, command, mediator, and observer are all related. If you watched the module in which we tackled the command pattern, this would be familiar.

Chain of responsibility passes the request along a chain of receivers that can be defined at runtime, until at least one handles it.

Command connects senders which receivers unidirectionally, for example from a button that is pushed in a UI, to the action that finally gets executed.

The mediator pattern, as we have just learned, eliminates direct connections altogether. Communication runs through that mediator.

And lastly, the observer allows receivers of requests to subscribe and unsubscribe them at run time.

Behavioral Pattern: Chain of Responsibility

Monday, October 31, 2022 1:46 PM

Coming up

In this module, we will cover the chain of responsibility pattern. Its intent is to avoid coupling the sender of request to its receiver by giving more than one object a chance to handle the request.

We will start by describing the pattern. As a real-life example, we will use a document validation and approval chain. After that, we will map it to a structural example and implement it. We will continue with learning about use cases for this pattern and the pattern consequences. We will end the module with related patterns.

Describing the Chain of Responsibility Pattern

The chain of responsibility pattern is a behavioral pattern from the Gang of Four library of design patterns. Its intent is to avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. It does that by chaining the receiving objects and passing the request along the chain until an object handles it. To show this we will show an example, an application, which validates a document. The first thing that needs to happen is ensure that the document's title is filled out. Up next, we have got a requirement that the document's last modified date must be less than 30 days old. If those check out, the document must be read and approved by the litigation department, after which it can be sent to the manager and approved there. Only after the manager has approved the document, it is considered valid and can be sent to the customer. As you gather from our code example, that is a lot of conditional statements that need to evaluate to true before the document is considered approved and ready to send to a customer. The method that checks the document becomes bloated and hard to manage, and there is no easy way to reuse these checks in other parts of our code. Enter the chain of responsibility pattern.

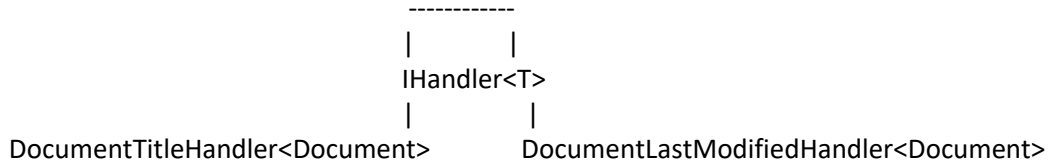
```
public bool Validate()
{
    if (document.Title == string.Empty)
        return false;
    if (document.LastModified < DateTime.UtcNow.AddDays(-30))
        return false;
    if (!document.ApprovedByLitigation)
        return false;
    if (!document.ApprovedByManagement)
        return false;
    return true;
}
```

The idea is that we are going to put all these validations and approvals in objects of their own, handlers. So we end up with an interface, say `IHandler`, which states that there should be a method to handle the incoming request. We could scope this to a document class, as that is what we are using in the example, but using generics make this more reusable, so that is what he suggest. As each handler can pass the request onto the next one, we also need to provide a way to set the next handler. That is the potential successor.

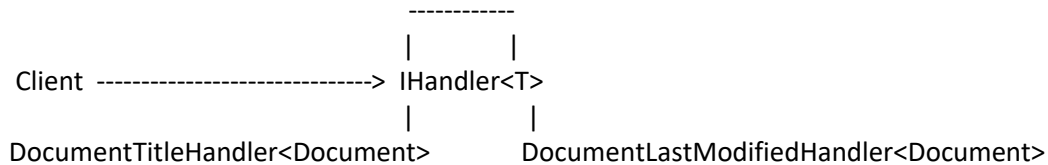
```
-----
|       |
IHandler<T>
```

For each of our validations, we have an implementation of this interface, a handler. So, we got a `DocumentTitleHandler`, a `DocumentLastModifiedHandler`, a `documentApprovedByLitigationHandler`, and the `documentApprovedByManager`. For readability purposes, it is only used two in the illustration. What these handlers do is get the document passed into

their Handle method and can then execute their part of the document validation. If the validation don't check out, we don't continue, but if it does, we can continue to the successor. Like that the document flows through the chain.

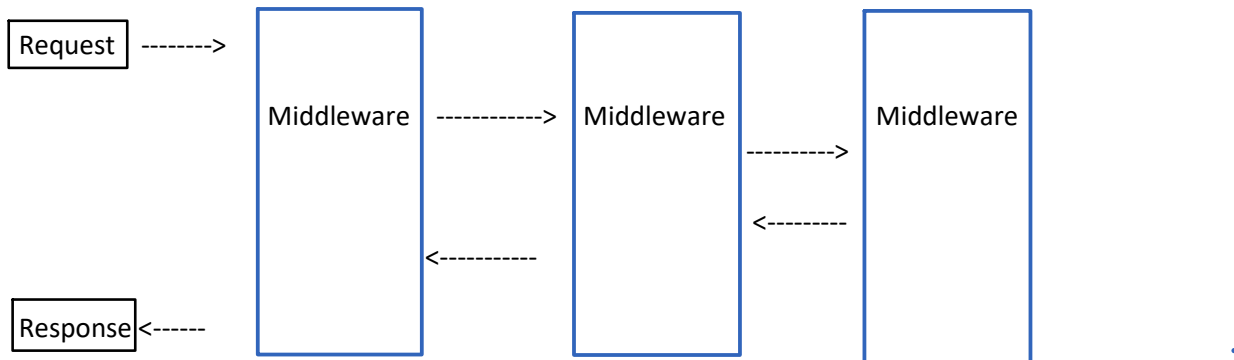


And of course, we have a client that initiates the request. The client chains the handlers and provides it with a document to validate.



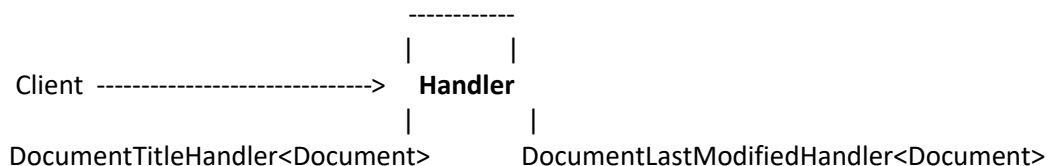
Before we continue with mapping this to the pattern structure, there is something to point out. The example used in this , is a real-life example that is in line with today's interpretation of the chain of responsibility pattern. However, if you look up the original implementation from the Gang of Four book, you would notice it is a bit more strict. In that interpretation, a request moves up the chain of handlers, and each handler simply checks whether it can handle that request or not. If it can't, nothing happens to the request except passing it on. And if it can handle it, it is handled and not passed on anymore. Both are valid implementations.

As a side note, if you have worked with ASP.NET Core, this should feel quite familiar. In how ASP.NET Core request pipeline works. Essentially, you are chaining pieces of middleware that can handle and/or alter a request and potentially pass it on to another piece of middleware. That is a prime example of chain of responsibility pattern at work.

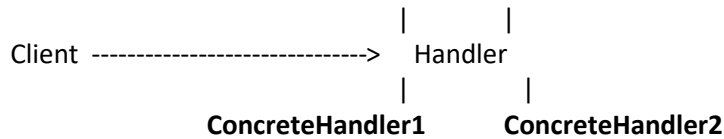


Structure of the Chain of Responsibility Pattern

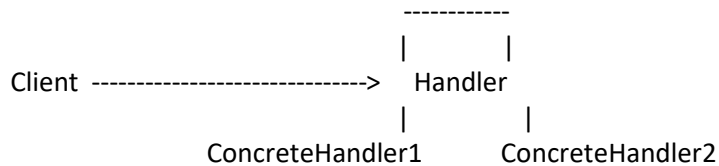
The first player in our pattern structure is the handler, which maps to IHandler interface. This defines an interface for handling requests and optionally implements the successor link.



Then we have got our concrete implementations, which are named ConcreteHandler. These handle requests. They can access the successor and potentially pass on the request to it.



Lastly, we have the client. The client is responsible for initiating the request to ConcreteHandler object in the chain.



Demo - Implementing the Chain of Responsibility pattern

In this demo we will implement the chain of responsibility pattern using our document validation example. So let's start with a Document class so we have something to validate. This is a pretty straightforward class. The document consist of a title, a last modified date, and two Booleans stating whether or not it has been approved by litigation and/or management. In the constructor, we accept these four values and set the properties.

In implementation.cs

```

public class Document
{
    public string Title { get; set; }
    public DateTimeOffset LastModified { get; set; }
    public bool ApprovedByLitigation { get; set; }
    public bool ApprovedByManagement { get; set; }

    public Document(
        string title,
        DateTimeOffset lastModified,
        bool approvedByLitigation,
        bool approvedByManagement)
    {
        Title = title;
        LastModified = lastModified;
        ApprovedByLitigation = approvedByLitigation;
        ApprovedByManagement = approvedByManagement;
    }
}
  
```

Then we need to create an interface for our handler. This is the handler component from the pattern structure. You could potentially use an abstract class for this as well if you want to provide some basic implementation logic. But in this case we can use an interface, that we name IHandler. We also make it generic so we can use it on other classes as well and not just on the Document class. It has a method, Handle, which will handle the incoming request. The request that comes in is of type T. That is our document. We also need a way to set the successor, the next handler in the chain. For that, we define a SetSuccessor method, which accepts that next handler. We conveniently have this method return an IHandler of T. That is the successor. This creates a fluent interface, allowing us to easily chain these handlers.

Below Document{}

```

public interface IHandler<T> where T : class
{
    IHandler<T> SetSuccessor(IHandler<T> successor);
}
  
```

```

        void Handle(T request);
    }

```

Then let's implement a ConcreteHandler for checking the title. We will name it DocumentTitleHandler, and we will have it implement our IHandle interface. The type we pass though is Document because that is what this handler works on. Then we implement the interface.

In the Handle methods, we do our validation. Let's rename the parameter that comes in to document instead of request. We check whether the title is empty or not. If it is, validation fails, and we throw a new validation exception. That is defined in System.ComponentModel.DataAnnotations. It is built in to the framework, so let's import that namespace (using statement). If title is not empty, we can go to the next handler. But we haven't got a field for that yet. So let's add a new field, successor, which we will use to store a reference to the next handler in line. Now we can call Handle on the successor, passing through the document. Note that successor is nullable. The handler doesn't have to have one. We only go to the next handler when successor is not null.

Then we need to implement the SetSuccessor method. This is pretty straightforward. We simply set the successor field, and we return the successor itself, which, as mentioned, will allow us to fluently chain another one. That is it for our handler. Now we have three more validation handlers left. The only thing they are different in is the actual validation rule.

Below IHandler{}

```

public class DocumentTitleHandler : IHandler<Document>
{
    private IHandler<Document>? _successor;

    public void Handle(Document document)
    {
        if (document.Title == string.Empty)
        {
            //validation doesn't check out
            throw new ValidationException(
                new ValidationResult(
                    "Title must be filled out",
                    new List<string>() { "Title" }), null, null);
        }

        //go to the next handler
        _successor?.Handle(document);
    }

    public IHandler<Document> SetSuccessor(IHandler<Document> successor)
    {
        _successor = successor;
        return successor;
    }
}

```

The first one is the documentLastModifiedHandler. We check whether the document was modified in the last 30 days. If it wasn't, we throw validation exception. Otherwise we potentially continue to the next handler in line. The SetSuccessor method simply sets the successor value as the previous handler.

Below DocumentTitleHandler{}

```

public class DocumentLastModifiedHandler : IHandler<Document>
{
    private IHandler<Document>? _successor;

```

```

public void Handle(Document document)
{
    if (document.LastModified < DateTime.UtcNow.AddDays(-30))
    {
        //validation doesn't check out
        throw new ValidationException(
            new ValidationResult(
                "Document must be modified in the last 30 days",
                new List<string>() { "LastModified" }), null, null);
    }

    //go to the next handler
    _successor?.Handle(document);
}

public IHandler<Document> SetSuccessor(IHandler<Document> successor)
{
    _successor = successor;
    return successor;
}
}

```

The next concrete handler is the `DocumentApprovedByLitigationHandler`. It follows likewise principles as the previous two handlers. The only difference is in the `Handle` method where we now check whether `ApprovedByLitigation` is true or false.

```

Below DocumentLastModifiedHandler {}
public class DocumentApprovedByLitigationHandler : IHandler<Document>
{
    private IHandler<Document>? _successor;

    public void Handle(Document document)
    {
        if (!document.ApprovedByLitigation)
        {
            //validation doesn't check out
            throw new ValidationException(
                new ValidationResult(
                    "Document must be approved by litigation",
                    new List<string>() { "ApprovedByLitigation" }), null, null);
        }

        //go to the next handler
        _successor?.Handle(document);
    }

    public IHandler<Document> SetSuccessor(IHandler<Document> successor)
    {
        _successor = successor;
        return successor;
    }
}

```

And then we add our last handler, the `DocumentApprovedByManagementHandler`, again following the same principles

as our other handlers. The only difference is that here we check the ApprovedByManagement Boolean. That is it for our implementation.

```
Below DocumentApprovedByLitigation{}
public class DocumentApprovedByManagementHandler : IHandler<Document>
{
    private IHandler<Document>? _successor;

    public void Handle(Document document)
    {
        if (!document.ApprovedByManagement)
        {
            //validation doesn't check out
            throw new ValidationException(
                new ValidationResult(
                    "Document must be approved by management",
                    new List<string>() { "ApprovedByManagement" } ), null, null);
        }

        //go to the next handler
        _successor?.Handle(document);
    }

    public IHandler<Document> SetSuccessor(IHandler<Document> successor)
    {
        _successor = successor;
        return successor;
    }
}
```

Let's write the code to test this. Let's create a valid document first. We need to import the chain of responsibility namespace for that. We pass through a title, a last modified date, and values stating that this document was approved by litigation and management. And now we create a second one, invalidDocument. As the name implies, this is invalid, and we can see that because the Boolean we passed through as value for the approved by litigation field is false. Now let's create our chain of handlers. We start with a DocumentTitleHandler. That was our first validation rule. And then, we chain the other three handler. For that, we can call into SetSuccessor, passing through instances of the handlers we want to add a successor. Seeing SetSuccessor returns an IHandler, we can chain these three statements. And with that we have got our chain. Let's effectively call Handle on the chain. It is surrounded by a try/catch statement in which can catch a validation exception, which is defined in System.ComponentModel.DataAnnotations. And this is done because, if all works out, we should hit a validation exception when handling the invalid document. So the first thing we do is try and handle the valid document, which should work out, and then we try and handle invalid document, which shouldn't work out and throw an exception, which we catch and then write out a message to the console output window. Let's add a ReadKey statement, and give this a try.

In Program.cs

```
var validDocument = new Document("How to Avoid Java Development", DateTimeOffset.UtcNow, true, true);
var invalidDocument = new Document("How to Avoid Java Development", DateTimeOffset.UtcNow, false, true);

var documentHandlerChain = new DocumentTitleHandler();
documentHandlerChain
    .SetSuccessor(new DocumentLastModifiedHandler())
    .SetSuccessor(new DocumentApprovedByLitigationHandler())
    .SetSuccessor(new DocumentApprovedByManagementHandler());
```



```

try
{
    documentHandlerChain.Handle(validDocument);
    Console.WriteLine("Valid document is valid.");
    documentHandlerChain.Handle(InvalidDocument);
    Console.WriteLine("Invalid document is valid.");
}
catch (ValidationException validationException)
{
    Console.WriteLine(validationException.Message);
}

Console.ReadKey();

```

And there we go. Our valid document is valid. We never see invalid document is valid written out to the console output window, but we do see a validation exception message, the document must be approved by litigation, which means it isn't valid.

Use Cases for the Chain of Responsibility pattern

There is three common use cases for this pattern.

Use it when more than one object may handle the request and the handler isn't known beforehand. In other words, it should be ascertained automatically. This is made possible because, with the chain of responsibility pattern you create a chain of linked handlers, and each handler gets a chance to process the request, but it doesn't have to. It can also simply pass it on.

Another use case is when you want to issue a request to one of several objects, which are the handlers, without specifying the receiver explicitly. You don't know beforehand whether a handler is effectively going to receive the request.

And lastly, when the set of objects that can handle a request should be specified dynamically. Translated to our handler example, all handlers and their order in the chain are specified at runtime.

Pattern Consequences

There is three main consequences to this pattern.

One is reduced coupling. Our client does not know which handler will handle the request. The object that invokes the operation is decoupled from the one that handles the operation. Like that, we can work towards a single responsibility per class.

The second consequence is added flexibility in regards to assigning responsibilities to objects. We can control the order in which handlers are executed at runtime, and we can control which handlers are in the chain.

The third is a disadvantage. Receipt isn't guaranteed. A request doesn't have an explicit receiver, so there is no guarantee that each handler will receive it or not or that it will be handled at all.

Related patterns

The chain of responsibility pattern is often applied in conjunction with the composite pattern. Here, the parent of a leaf component can act as the successor. In other words, when a leaf component gets a request, it can pass it on to the parent, which can pass it on to its parent, and so on. That is a chain of responsibility.

There is also a relation between the responsibility and the command pattern. The handlers we implemented can be

implemented as commands.

- Composite
The parent of a leaf can act as the successor.
- Command
Chain of responsibility handlers can be implemented as commands.

When we look at some of the other patterns in this course, we find others that connect senders and receivers of requests. In that regard, chain of responsibility, command, mediator, and observer are all related.

Chain of responsibility passes the request along a chain of receivers that can be defined at runtime, until at least one handles it, or none at all.

Command connects senders which receivers unidirectionally, for example from a button that is pushed in a UI, to the action that finally gets executed.

The mediator pattern, as we have just learned, eliminates direct connections altogether. Communication runs through that mediator.

And lastly, the observer allows receivers of requests to subscribe and unsubscribe them at run time.

Behavioral Pattern: Observer

Monday, October 31, 2022 1:46 PM

Coming up

In this module we will cover the observer pattern. Its intent is to define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically. It is also known as publish-subscribe, or pub-sub.

We will start by describing the pattern using a real-life example. As a real-life example we will use part of a ticket management system in which services have to communicate with each other. We will map that to the structure of the observer pattern, and implement it in a demo. We will continue to learn about the use cases for this pattern and the pattern consequences. And finish with related patterns.

Describing the Observer Pattern

The observer pattern is a behavioral pattern from the Gang of Four library of design patterns. The intent of the observer pattern is to define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

It is a pretty common pattern. For example, in Angular, you have got an Observable class that implements it. Also when you are working with microservices, you will often have services that subscribe to change events on other services to allow them to keep their data store in sync and so on.

What we will use as an example is part of a ticketing management system. It exposes a bunch of services, all with their own data store that needs to be kept in sync, not unlike what you'd expect in a microservices architecture. For example, there is an OrderService that has a notion of a set of tickets for a certain artist. When the order is fulfilled, the tickets are no longer available, and it updates its data store. But our system knows of other services as well.

There is a TicketStockService, which needs to be notified of the fact that the tickets are no longer there so it can update that in its data store. And then there is a TicketResellerService, which also needs to be notified of the fact that the tickets are no longer available. Other services that need to know about this can exist as well, but for this example we will use these three classes.

```
public class OrderService {}
```

```
public class TicketStockService {}
```

```
public class TicketResellerService {}
```

In any case, we are dealing with a system in which a set of related objects, which are our services, need to maintain consistency. We can easily achieve that. We could simply store a reference to the TicketStockService and TicketResellerService on our OrderService and let the OrderService call methods on them to notify them that the tickets have been sold. The problem is that this would introduce tight coupling. It would also become quite complex to manage once the amount of services that need to be notified started growing. And moreover, if we don't know in advance how many or which services need to be notified, this won't work at all.

```
public class OrderService
{
    private TicketStockService _ticketStockService;
```

```

        private TicketResellerService _ticketResellerService;
    }

```

```

public class TicketStockService {}

```

```

public class TicketResellerService {}

```

Enter the observer or pub/sub pattern. We will start with a base class, the TicketChangeNotifier. Our OrderService will be a subclass of this. On the TicketChangeNotifier we implement methods to add and remove observers and also want to notify them. We won't add, remove, or notify concrete observers though.

```

TicketChangeNotifier
|

```

```

OrderService

```

We will also work on an interface, say ITicketChangeListener. The interface defines one method, ReceiveTicketChangeNotification with the change in tickets as a parameter.

```

TicketChangeNotifier -----> ITicketChangeListener
|

```

```

OrderService

```

It is this interface that is implemented by our observers, the TicketStockService and TicketResellerService.

```

TicketChangeNotifier -----> ITicketChangeListener
|

```

```

|
| I----- TicketStockService |
|
OrderService <----- TicketResellerService

```

What happens now is that our two observers are added as observers on our OrderService. The OrderService is a concrete implementation of the TicketChangeNotifier. It calls into the methods defined on that, but next to that, it also holds its state, for example the data store containing the information it has on tickets. Once a sale happens, the OrderService adjusts its state by updating its data store. It then notifies all observers of this change through the Notify method on its base class. And that method runs through all observers and calls ReceiveTicketChangeNotification on them. Like that each ConcreteObserver can update its own state accordingly.

Structure of the Observer Pattern

The first player in the pattern structure is the Observer, that is our ITicketChangeListener. It defines an updating interface for objects that should be notified of changes in the subject.

```

TicketChangeNotifier -----> Observer
|

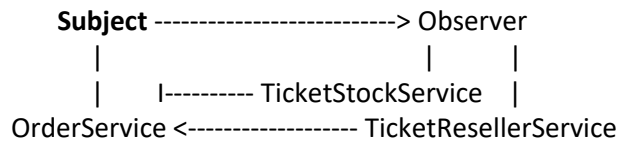
```

```

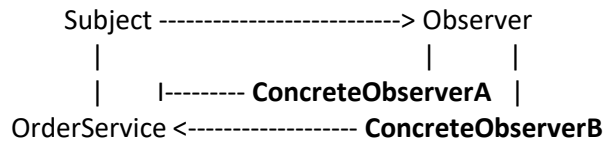
|
| I----- TicketStockService |
|
OrderService <----- TicketResellerService

```

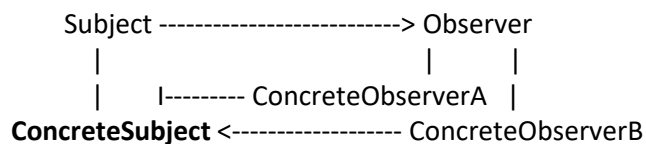
That subject, that is our TicketChangeNotifier base class. It knows its observers. Any number of observers can observe a subject. Next to that it provides an interface for attaching and detaching observer objects.



The ConcreteObserver maps to our TicketStockService and TicketResellerService. These store state that should remain consistent with the subject state, and they implement the Observer updating interface to keep its state consistent with the subjects.



Lastly the ConcreteSubject stores state that is of interest to ConcreteObserver objects, and it sends a notification to its observers when it states changes. That is the OrderService from our example.



The diagram above, slightly differs from the original pattern template. In our implementation we are passing through the state from ConcreteSubject to ConcreteObserver as a parameter via the Notify method. In the original template, the ConcreteObserver would hold a reference to ConcreteSubject, so it could query it for state changes. That is a valid implementation as well, but these days, the one used above is more common. The intent is the same in both, and that is what is important, not the exact implementation of the template.

Demo - Implementing the Observer pattern

In this demo we will implement the observer pattern, and we will start with the helper class, TicketChange. This will hold the state change information that needs to be sent to observers. For example, a ticket change for artists with ID 1 and amount 2 means that 2 tickets have been sold for this artist. It is an object of this type that will be received by observers.

```

In Implementation.cs
public class TicketChange
{
    public int Amount { get; private set; }
    public int ArtistId { get; private set; }

    public TicketChange(int artistId, int amount)
    {
        ArtistId = artistId;
        Amount = amount;
    }
}
  
```

Then let's add the subject. That is our TicketChangeNotifier. It is an abstract class. We don't want it to be used on its own because it doesn't contain the state information observers want to be notified about. That is contained in the concrete implementation. What this does contain is a list of observers, and those are objects that implement ITicketChangeListener. We haven't defined that interface yet, so let's

do that next.

Below TicketChange{}:

```
public abstract class TicketChangeNotifier
{
    private List<ITicketChangeListener> _observers = new();
}
```

When implementing the interface the code above will resolve. It is named ITicketChangeListener, and this is our observer. It contains just one method, ReceiveTicketChangeNotification, which is used by the subject to notify its observers about a change in the amount of tickets.

Below TicketChangeNotifier{}:

```
public interface ITicketChangeListener
{
    void ReceiveTicketChangeNotification(TicketChange ticketChange);
}
```

Then we can start adding methods on our TicketChangeNotifier. We need a few methods to manage observers. After all it is the TicketChangeNotifier that needs to keep a list of observers it needs to notify. So we call into AddObserver, which accepts an ITicketChangeListener as parameter. When the method is executed the observer gets added to the list of observers. RemoveObserver will remove an observer from the list. And when notify is called, all observers receive a TicketChange notification with the TicketChange object passed through as a parameter.

Change to TicketChangeNotifier:

```
public abstract class TicketChangeNotifier
{
    private List<ITicketChangeListener> _observers = new();

    public void AddObserver(ITicketChangeListener observer)
    {
        _observers.Add(observer);
    }

    public void RemoveObserver(ITicketChangeListener observer)
    {
        _observers.Remove(observer);
    }

    public void Notify(TicketChange ticketChange)
    {
        foreach (var observer in _observers)
        {
            observer.ReceiveTicketChangeNotification(ticketChange);
        }
    }
}
```

Onto the concrete implementations. Our subject is the TicketChangeNotifier, and we are going to start with the concrete subject, which is our order service. Its subclass is the subject, the TicketChangeNotifier.

What this class is concerned with is its state. For all registration and communication-related tasks, it relies on its base class, the TicketChangeNotifier. We are going to add one method to it, CompleteTicketSale. It is this method that is called by clients when a ticket sale is completed. That means that at this moment, the OrderService must update its state. It must update its data store so the correct amount of tickets for the correct artist is deducted.

For demo purposes, we will assume that when we see an OrderService is changing its state message that the local data store is adjusted. After that, it calls the Notify method on its base class, the TicketChangeNotifier, to notify the observers. And with that we have got our concrete subject.

Below ITicketChangeListener{}:

```
public class OrderService : TicketChangeNotifier
{
    public void CompleteTicketSale(int artistId, int amount)
    {
        //Change local datastore. Datastore omitted in demo implementation.
        Console.WriteLine($"{nameof(OrderService)} is changing its state.");
        //Notify observers
        Console.WriteLine($"{nameof(OrderService)} is notifying observers...");
        Notify(new TicketChange(artistId, amount));
    }
}
```

The last two parts are the concrete observers. The TicketResellerService is our first one. It needs to implement the ITicketChangeListener interface. That forces us to implement the ReceiveTicketChangeNotification method. This gets a state change from the concrete subject passed in, and that is the TicketChange object. Thanks to the info it gets from that object, it can adjust its own data store accordingly.

Below OrderService{}:

```
public class TicketResellerService : ITicketChangeListener
{
    public void ReceiveTicketChangeNotification(TicketChange ticketChange)
    {
        //Update local datastore (datastore omitted in demo implementation)
        Console.WriteLine($"{nameof(TicketResellerService)} notified " +
            $"of ticket change: artist {ticketChange.ArtistId}, amount " +
            $"{ticketChange.Amount}");
    }
}
```

Our last concrete observer was the TicketStockService. This too receives a ticket change notification, allowing it to update its data store. And that is it for the pattern implementation.

Below TicketResellerService{}:

```
public class TicketStockService : ITicketChangeListener
{
    public void ReceiveTicketChangeNotification(TicketChange ticketChange)
    {
        // update local datastore (datastore omitted in demo implementation)
        Console.WriteLine($"{nameof(TicketStockService)} notified " +
            $"of ticket change: artist {ticketChange.ArtistId}, amount " +
            $"{ticketChange.Amount}");
    }
}
```

```
}  
}
```

Now let's test this. First we create new instances of our three services. For that, we need to import our Observer namespace. So we have got our two concrete observers, and we have got our concrete subject. To that concrete subject, we add these two observers. For that, we can call AddObserver on the orderService. As a parameter to AddObserver, we need to pass through something that implements ITicketChangeListener, and both our TicketStockService and TicketResellerService do. Now let's say that a ticket sale happens for the artist with ID 1. Registering such a ticket sale on the orderService means calling the CompleteTicketSale method, passing through 1 as ID for the artist and 2 for the amount of tickets. This call should result in both observers being notified of this. Add a ReadKey statement at the bottom.

In Program.cs

```
TicketStockService ticketStockService = new();  
TicketResellerService ticketResellerService = new();  
OrderService orderservice = new();
```

```
//Add two observers  
orderservice.AddObserver(ticketStockService);  
orderservice.AddObserver(ticketResellerService);
```

```
//Notify  
orderService.CompleteTicketSale(1, 2);
```

```
Console.ReadKey();
```

That seems to work just fine. OrderService is changing its state after which it is notifying its observers, and then we see messages from our observers who are being notified of the ticket change. The nice thing here is that we can manage all of this at runtime. We added the observers at runtime, and that means we can also remove them. We remove one observer, the TicketResellerService, and then we complete another ticket sale.

Changes to Program:

```
TicketStockService ticketStockService = new();  
TicketResellerService ticketResellerService = new();  
OrderService orderservice = new();
```

```
//Add two observers  
orderservice.AddObserver(ticketStockService);  
orderservice.AddObserver(ticketResellerService);
```

```
//Notify  
orderService.CompleteTicketSale(1, 2);
```

```
Console.WriteLine();
```

```
//Remove one observer  
orderService.RemoveObserver(ticketResellerService);
```

```
//Notify  
orderService.CompleteTicketSale(2, 4);
```



```
Console.ReadKey();
```

When testing this, the first four lines being written out to the console output window remain the same. After that, the TicketResellerService observer is removed, and that results in only the TicketStockService being notified of the new ticket sale.

Use Cases for the Observer pattern

There are three main situations in which this pattern should be considered.

One, when a change to one object requires changing others and you don't know in advance how many objects need to be changed. That is a pretty common scenario in pub/sub use cases. It is common in microservice architectures where services that need to be notified can change dynamically.

Two, we use it when objects that observe others are not necessarily doing that for the total amount of time the application runs. The pattern allows you to remove and subscribe observers dynamically.

Three, consider it when an object should be able to notify other objects without making assumptions about who those objects are. In other words, use it when you want to avoid tight coupling between these objects.

Pattern Consequences

The most important consequence of this pattern is that subjects and observers can vary independently. In other words, you can change one or add a new subclass of one, without having to change the other. This is inline with the open/closed principle of the SOLID coding principles.

Very much related to that, subject and observer are loosely coupled. All the subject knows is that it has a list of observers that conform to an interface. It doesn't know about concrete implementations. This too is inline with the open/closed principle of the SOLID coding principles.

Something to look out for our unexpected updates. In some implementations of this pattern, observers hold a reference to the subject and can change its state. Observers don't know about the potential consequences of this. It can lead to a cascade of change notification. In our implementation, cannot do that.

Related patterns

When we look at some of the other patterns in this course, we find others that connect senders and receivers of requests.

Chain of responsibility, command, mediator, and observer are all related. Chain of responsibility passes the request along a chain of receivers that can be defined at runtime, until at least one handles it, or none at all.

Command connects senders which receivers unidirectionally, for example from a button that is pushed in a UI, to the action that finally gets executed.

The mediator pattern, as we have just learned, eliminates direct connections altogether. Communication runs through that mediator.

And lastly, the observer allows receivers of requests to subscribe and unsubscribe them at run time.

Behavioral Pattern: State

Monday, October 31, 2022 1:46 PM

Coming up

In this module we will cover the state pattern, also known as objects for states. It allows an object to alter its behavior when its internal state changes.

We will start by describing the pattern using a real-life example, withdrawing or depositing money from or to a bank account, which can result in the bank account changing state from regular to overdrawn. After that we will map it to the pattern structure and we will implement it in a demo. We will continue to learn about use cases for this pattern, and the pattern consequences. We will end the module with related patterns.

Describing the State Pattern

The state pattern is a behavioral pattern from the Gang of Four library of design patterns. Its intent is to allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

Let's clarify this with an example, withdrawing or depositing money from or on a bank account. This can result in the bank account changing state from regular to overdrawn or vice versa. If your bank account is defined by a class, `BankAccount`, you could add an enumeration property, `BankAccountState` to it.

```
public void Deposit(decimal amount)
{
    Balance += amount;
    if (Balance >= 0)
    {
        BankAccountState = BankAccountState.Regular;
    }
}
```

When you withdraw money, the state can potentially go from regular to overdraw. And when you deposit money it can potentially go from overdrawn back to regular. Depending on the state of the account, other rules might apply. You might not be able to withdraw money when your account is in overdrawn state. You can solve that with a conditional statement in the `Withdraw` method. That is the switch statement below.

```
public void Withdraw(decimal amount)
{
    switch (BankAccountState)
    {
        case BankAccountState.Regular:
            Balance -= amount;
            if (Balance < 0)
            {
                BankAccountState = BankAccountState.Overdrawn;
            }
            break;
        case BankAccountState.Overdrawn:
```

```

        // Cannot withdraw
        break;
    default:
        break;
    }
}

```

Now imagine an additional state, gold, which you get when your account has a lot of money in it. It allows you to withdraw more money at once. That means that the logic for transitioning becomes a bit more complex, and the conditional statement when withdrawing becomes a bit more complex as well, but it is still manageable.

Yet it is not hard to imagine that this can become very complex, very fast. Adding a few additional states or business rules, like only allowing you to take out a credit card when your account is in a certain state, changing state of the linked account when you overdraw a credit card, or even an interest rate that changes depending on the bank account state. The way the account behaves depends on the state, and the state can change at runtime.

Taking care of all these business rules and transitions and states in the way just shown will lead to large, difficult-to-manage conditional statements and state transitions. That is what the state pattern can help with.

We will start with just the two states from the initial example, regular and overdrawn. We still have a BankAccount class with methods on it to withdraw and deposit money.

BankAccount

What we could do is introduce an abstract class that represents the state of our bank account. It declares an interface common to all classes that represent different operational states of the bank account, which includes methods to handle requests that are coming from the bank account, like deposit and withdraw. We name that one BankAccountState. The BankAccount class has a BankAccountState property of this type.

BankAccount -----> BankAccountState

For each state, we have concrete implementations, like regular state and overdrawn state. In these implementations the logic resides to transition from one state to another. So, in the regular state object's Withdraw method, we have code that will change the state from regular state to overdrawn state when the balance goes below zero and vice versa.

The more states you have, the more you will notice this logic will be less conditional and complex than when working with enumeration. For each state, we have concrete implementations, like regular state and overdrawn state. In these implementations the logic resides to transfer from one state to another. So in the regular state object's Withdraw method, we have code that will change the state on the bank account from regular to overdrawn state when the balance goes below zero and vice versa.

```

BankAccount -----> BankAccountState
                    |           |
                RegularState   OverdrawnState

```

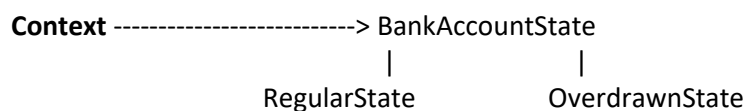
The more states you have, the more you will notice this logic will be less conditional and complex than when working with enumeration because typically you can't go from each state to all other states. The logic in the withdraw and deposit methods becomes easier as well. For example there is no conditional

statement anymore in the withdraw method on our overdrawn state to check whether we can withdraw money or not. We don't have to check for the state we are in as the state is represented by the object itself.

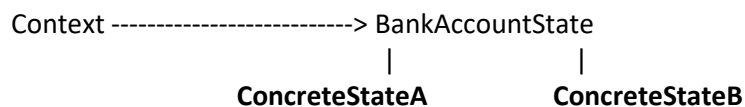
So when depositing money or withdrawing money via the bank account object, it calls into the underlying Deposit and Withdraw methods on the BankAccount state object. And as we just detailed, doing that takes care of state-specific logic and state transitions. By doing so, we vastly diminish complexity. Each state object handles just its own possible transitions and logic. The more different states there are, the more obvious the advantages become.

Structure of the State Pattern

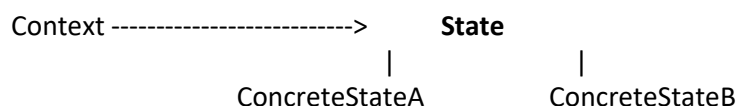
The first player in this pattern is the Context, that is our bank Account. This defines the interface that is of interest to clients. It maintains its state via a property of a concrete state subclass, which defines the current state.



The next player is the Concrete state subclasses. These map to our regular BankAccountState and OverdrawnBankAccountState classes. Each of these subclasses implements behavior associated with the state of the context.



Lastly, we have the state class. This defines an interface for encapsulating the behavior associated with a particular state of the context. That is our abstract base class, BankAccountState.



Demo - Implementing the State pattern

Let's start with the state class, BankAccountState. We will implement a pattern with an abstract base class, but you could use an interface for this as well. As we learned state objects will be responsible for transitioning between states. As state is defined on the context object, our BankAccount class, we need a reference to it. So we add a reference to the BankAccount.

We also need methods that handle the request on the context. We can deposit and withdraw money, so we add two abstract methods for that. Depositing and withdrawing money has an effect on the balance of the account, which in turn is used to decide on potential state transitions. We also need a Balance property. The setters of BankAccount and Balance was made protected. Like that, subclasses of our bank account state can set it, but outside of that, only getting the info is allowed. It is added a null-forgiving operator to the BankAccount. Otherwise we would get a warning stating that bank account might be null.

However, since we know that subclasses will ensure that when they are constructed, the BankAccount property is set, we can safely add this null-forgiving operator as to give a hint to the compiler that it shouldn't worry.

In Implementations.cs

```
public abstract class BankAccountState
{
    public BankAccount BankAccount { get; protected set; } = null!;

    public decimal Balance { get; protected set; }

    public abstract void Deposit(decimal amount);

    public abstract void Withdraw(decimal amount);
}
```

The context class is the BankAccount. Let's add an instance of BankAccountState to our BankAccount class. Important here is that we are working on the abstract base class. So we are not working on the implementation; we are coding against an interface to avoid tight coupling and enabling our state scenario.

We also add a Balance property as you would expect a client to want to access the balance from the BankAccount class. Important here is that this is a request to get the balance, which is handled by the state object. So the getter returns the balance field on the current state. In the constructor, we set the default state and initialize the account with a balance of 200. The remaining methods are easy. When a request for a deposit comes in, we pass it through to the state object to handle, and we do the exact same thing with requests for withdrawals.

Below BankAccountState{}:

```
public class BankAccount
{
    public BankAccountState BankAccountState { get; set; }

    public decimal Balance { get { return BankAccountState.Balance; } }

    public BankAccount()
    {
        BankAccountState = new RegularState(200, this);
    }

    public void Deposit(decimal amount)
    {
        //let the current state handle the deposit
        BankAccountState.Deposit(amount);
    }

    public void Withdraw(decimal amount)
    {
        //let the current state handle the withdraw
        BankAccountState.Withdraw(amount);
    }
}
```

Then let's add the two implementations of BankAccountState, RegularState and OverdrawnState. In the constructor of RegularState, we ensure that the Balance and BankAccount are set. That takes care of

ensuring non-null values. It is this constructor that will be used to and transitioning state. Let's add the two methods from the abstract base class, Deposit and Withdraw. Let's do the same with `overdrawnState` as well.

Back in `RegularState`. When depositing money, we are going to write out some info to the console window. That will make it easier for us to see what is going on. We also change the balance by the inputted amount. When you are in regular state and depositing money, there is no state transition that can happen at this moment. So, no further logic is required.

When withdrawing money, on the other hand, we can change state. When we enter the method, we write this to the console output window, and we change the balance. Now let's say that when the balance is below zero, after the amount has been deducted, we will change the state to `overdrawn` state. To be able to do that, we need to set the state on the `BankAccount` property to a new instance of `OverdrawnState`. In the `withdraw` method in the regular state, we change the state of the `BankAccount` to a new instance of `OverdrawnState`.

We construct that `OverdrawnState` object by passing through the current balance and the `BankAccount` object itself because the `OverdrawnState` object potentially has to change bank account values, like that state of the bank account.

```
Below BankAccountState{}
public class RegularState : BankAccountState
{
    public RegularState(decimal balance, BankAccount bankaccount)
    {
        Balance = balance;
        BankAccount = bankAccount;
    }

    public override void Deposit(decimal amount)
    {
        Console.WriteLine($"In {GetType()}, depositing {amount}");
        Balance += amount;
    }

    public override void Withdraw(decimal amount)
    {
        Console.WriteLine($"In {GetType()}, withdrawing {amount} from {Balance}");
        //Change state to overdrawn when withdrawing results in less than zero
        Balance -= amount;

        if (Balance < 0)
        {
            //Change state to overdrawn
            BankAccount.BankAccountState = new OverdrawnState(Balance, BankAccount);
        }
    }
}
```

In the `overdrawnState` object, when depositing money, we can potentially change back to regular state. If the balance after depositing is larger than or equal to zero, we change the state back to regular. And when we are in this `OverdrawnState` object, we won't allow withdrawing money anymore.

Below RegularState{}:

```
public class OverdrawnState : BankAccountState
{
    public OverdrawnState(decimal balance, BankAccount bankaccount)
    {
        Balance = balance;
        BankAccount = bankAccount;
    }

    public override void Deposit(decimal amount)
    {
        Console.WriteLine($"In {GetType()}, depositing {amount}");
        Balance += amount;

        if (Balance >= 0)
        {
            //Change state to regular
            BankAccount.BankAccountState = new RegularState(Balance, BankAccount);
        }
    }

    public override void Withdraw(decimal amount)
    {
        //Cannot withdraw
        Console.WriteLine($"In {GetType()}, can't withdraw, balance {Balance}");
    }
}
```

Let's test this. We make a new instance of the BankAccount. For that, we need to import the State namespace and then we see what happens when we deposit something, withdraw something and withdraw even more money. This should result in a state switch. Add a ReadKey statement. And let's run this.

In Program.cs

```
BankAccount bankAccount = new();
bankAccount.Deposit(100);
bankAccount.Withdraw(500);
bankAccount.Withdraw(100);
```

```
Console.ReadKey();
```

We start in regular state. And as you remember, we constructed the bank account with 200\$ in the account. We deposit 100, which means we are at 300. So the next time we do something, we are in a regular state. We withdraw 500 from 300, which means our balance is now beneath 0. That means that the state should transition to overdrawn state, which was indeed the case because the next time we try to do a withdraw we see that we are in the overdrawn state. As a result we are not allowed to withdraw money.

Demo - Extending the Bank Account sample with an additional state

In this demo, we will implement support for an additional state, gold state. An account gets gold status

when the balance is higher than 1000. We also need an extra rule for demo purposes.

Let's say that when you have gold status and you deposit money, you get an immediate interest of 10%. This is a nice example to illustrate less conditional statements are required. And let's make it a bit more complex.

You can transition from gold to regular or overdrawn and from regular to gold, but you can never directly transition from overdrawn to gold. You will need to be in regular status for at least one deposit. That means that the rules are a bit more complex than just looking at the balance. You can be in regular status even when you have a balance of more than 1000.

The GoldState derives from the BankAccountState, which means we should implement the Deposit and Withdrawal methods. And just as with the other ConcreteState implementations, we are going to add a constructor that accepts the balance and bank account and sets them.

The Deposit method is easy. We are in GoldState, so that means that when money is deposited, we get 10% interest applied immediately. So no conditional statement is required here.

In the Withdraw method we do need conditional statements to potentially transition to the two other states. So what we are saying here is that you can go from GoldState to either regular or overdrawn state, depending on your account balance after you withdraw money.

We start by adding the GoldState class.

Above RegularState{}:

```
public class GoldState : BankAccountState
{
    public GoldState(decimal balance, Bankaccount bankAccount)
    {
        Balance = balance;
        BankAccount = bankAccount;
    }

    public override void Deposit(decimal amount)
    {
        Console.WriteLine($"In {GetType()}, depositing " + $" {amount} + 10% bonus: {amount / 10}");
        Balance += amount + (amount / 10);
    }

    public override void Withdraw(decimal amount)
    {
        Console.WriteLine($"In {GetType()}, withdrawing {amount} from {Balance}");

        //Change state to overdrawn when withdrawing results in less than zero
        Balance -= amount;

        if (Balance < 1000 && Balance >= 0)
        {
            //Change to regular
            BankAccount.BankAccountState = new RegularState(Balance, BankAccount);
        }
        else if (Balance < 0)
        {
            //Change to overdrawn
            BankAccount.BankAccountState = new OverdrawnState(Balance, BankAccount);
        }
    }
}
```



```

        {
            //Change to regular
            BankAccount.BankAccountState = new OverdrawnState(Balance, BankAccount);
        }
    }
}

```

We also need to make a small change to our regular state, as you can potentially transition to GoldState from RegularState when depositing money. So if the balance is over 1000 or equal to it, we switch to GoldState.

Change RegularState:

```

public class RegularState : BankAccountState
{
    public RegularState(decimal balance, BankAccount bankAccount)
    {
        Balance = balance;
        BankAccount = bankAccount;
    }

    public override void Deposit(decimal amount)
    {
        Console.WriteLine($"In {GetType()}, depositing {amount}");
        Balance += amount;

        if (Balance >= 1000)
        {
            //Change to gold state
            BankAccount.BankAccountState = new GoldState(Balance, BankAccount);
        }
    }

    public override void Withdraw(decimal amount)
    {
        Console.WriteLine($"In {GetType()}, withdrawing {amount} from {Balance}");
        //Change state to overdrawn when withdrawing results in less than zero
        Balance -= amount;

        if (Balance < 0)
        {
            //Change state to overdrawn
            BankAccount.BankAccountState = new OverdrawnState(Nalance, BankAccount);
        }
    }
}

```

Another rule we want to implement is that you can't transition from overdrawn to gold directly. You will at least need to be in RegularState for one deposit. This rule is easy to implement. We don't have to change anything to our code base, as our overdrawnState object only implements a transition to regular state. All that is left is adding a few additional statements for testing to our program. First, we are going to deposit enough money to get to gold status, then we check whether that is indeed the case, and then we try to go back to overdrawn status. After we deposit more money, we should transition to regular

instead of gold, and we can check for that by doing a deposit of 100. Let's give this a try.

Change to Program:

```
BankAccount bankAccount = new();  
bankAccount.Deposit(100);  
bankAccount.Withdraw(500);  
bankAccount.Withdraw(100);
```

```
//deposit enough to go to gold  
bankAccount.Deposit(2000);  
//should be in gold now  
bankAccount.Deposit(100);  
//back to overdrawn  
bankAccount.Withdraw(3000);  
// should transition to regular  
bankAccount.Deposit(3000);  
//Should still be in regular  
bankAccount.Withdraw(100);
```

```
Console.ReadKey();
```

The first three statements are the same as in the previous demo. Then, we are in overdrawn state and we deposit 2000. That means we now have a balance of 1800. So we switch to regular state because we can't directly switch to gold state. Now we deposit 100 while in regular state.

We move to gold state because our balance is now 1900. And when we withdraw 3000 from 1900, we automatically go to overdrawn state. And when we are in overdrawn state and deposit 3000, we have a balance over 1000, but we go to regular state because of the rule.

So these are already quite complex rules, but all we had to do to implement all of this is create one new subclass and change the transitions. We didn't have to change the BankAccount class itself, and we didn't end up with as many conditional statements if we would have worked with an enumeration property for state on the BankAccount class. Even our rather complex transitioning rule was easy to implement.

Use Cases for the State pattern

There are two good use cases for this pattern.

The first case in which you should consider this pattern is when an object's behavior depends on its state, and it must change its behavior at runtime depending on that state. That is the use case we just implemented with the bank account example.

Another good use case is when your objects are dealing with large conditional switch or if statements that depends on the object state, for example by storing the state in an enumeration field on the object, also not unlike the use case from the demo. This often results in several of your object operations to contain the same conditional structure. That is what we would have ended up with in our naive implementation.

By using the state pattern, each branch of the conditional is put in a separate class, which lets you treat the object's state as an object in its own right. It can thus vary independently from other objects.

Pattern Consequences

This pattern localizes state-specific behavior and partitions behavior for different states. We saw that as all the behavior associated with a particular state was put in an object of its own. This works towards the single responsibility principle from the solid coding principles.

Another positive consequence is that because state-specific code lives in a state subclass, new states and transitions can easily be added by defining new subclasses. We saw that when we extended our example with the new state in the second demo. That is the open/closed principle from the solid coding principles at work.

But this can have a possible negative consequence as well. As the pattern distributes behavior across different state subclasses, the number of classes increased, which adds additional complexity. But seeing long conditional statements are not desirable, this becomes more of a positive consequences than a negative consequence once the amount of state subclasses is large.

Related patterns

There is a few related patterns.

The first one is the flyweight. When you implement the state pattern without instance variables in the state objects, you are essentially making these state objects shareable, which makes them flyweights.

There is also a relation to the singleton pattern as state objects often are singletons.

Strategy, bridge, and states are all related and look similar because they are based on composition, but they solve different problems.

- Flyweight
Without instance variables in the state objects, they become flyweights.
- Singleton
State objects are often singleton.
- Strategy
Also based on composition, but solves a different problem.
- Bridge
Also based on composition, but solves a different problem.

Behavioral Pattern: Iterator

Monday, October 31, 2022 1:46 PM

Coming up

In this module, we will dive into the iterator pattern. Its intent is to provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

We will start by describing the pattern, and you will notice very soon that this is a very common pattern, at least to use, not to implement. Yet we are going to implement the custom iterator by automatically iterating over a list in alphabetical order. We will use that as an example. After that, we will map that example to the pattern structure and implement it in the demo. We will continue with learning about use cases for this pattern and the pattern consequences. We will end the module with related patterns.

Describing the Iterator Pattern

The iterator pattern is a behavioral pattern from the Gang of Four library of design patterns. Iterators are everywhere. They are one of the most common patterns used by frameworks to traverse their collection.

Yet, even though it is very common, it is not often seen used that much in custom code anymore. By which, it means that the frameworks we use tend to already have the iterators in place we commonly need. Its intent is to provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

Sounds a little complicated maybe, but simply change aggregate object to innumerable or array or collection, and the intent all of a sudden becomes a lot more tangible. Whenever we use a for each statement over an innumerable, say a list of strings, it is an iterator that is used to provide the correct previous or next item.

And let's continue with that list example. A list is an aggregate object, which aggregates a set of variables, potentially other objects. Lists, but also other aggregate objects like collections, Dictionary, Stack, Queue, and so on keep their items in an internal structure. You don't want to expose this internal structure to avoid it being tampered with in unintended ways.

Also you might not always want to traverse the aggregate object in the same way. You might want to allow traversing it alphabetically or in reverse order, and you don't want to bloat the aggregate object interface with all these different ways of traversing it. And that is what the iterator solves. It takes the responsibility for traversing out of the aggregate list object and puts it in an iterator object.

Now as mentioned, most common iterators on the collections we use already exist. So an example, we are going to create a custom collection, a collection of people. On it, we are going to create an iterator a collection of people. On it, we are going to create an iterator that iterates over these people, but it is going to do that in alphabetical order by the person's name.

We will name our custom collection, PeopleCollection. On it common methods like you'd expect in a collection exist like Add, Remove, maybe Count, and so on. Next to that, it should also have a CreateIterator method. That will return our iterator and will define the need for such a method on an interface, IPeopleCollection.

PeopleCollection

The IPeopleCollection interface will define an interface for creating an iterator object.

```
IPeopleCollection
|
PeopleCollection
```

We will name that IPeopleIterator. That too is an interface. Like this, we avoid having to use concrete implementations on our PeopleCollection, which means we can easily provide multiple different iterators without having to change the class. The PeopleIterator interface contains four methods, First, Next, IsDone, and CurrentItem. These are the methods required for iterating over a collection.

```
IPeopleCollection      IPeopleIterator
|
PeopleCollection
```

The last part of the story is the concrete implementation of the iterator. The PeopleIterator, it implements the IPeopleIterator interface, and it will contain the actual logic for iterating over our PeopleCollection, sorted alphabetically by name.

```
IPeopleCollection      IPeopleIterator
|                      |
PeopleCollection <----- PeopleIterator
```

Structure of the Iterator Pattern

We will start with the aggregate. This defines an interface for creating an Iterator object, that maps to our IPeopleCollection.

```
Aggregate              IPeopleIterator
|                      |
PeopleCollection <----- PeopleIterator
```

That iterator object, that is what defines an interface for accessing and traversing elements. It maps to our IPeopleIterator interface.

```
Aggregate              Iterator
|                      |
PeopleCollection <----- PeopleIterator
```

And then we have the two concrete implementations. The ConcreteIterator implements the Iterator interface, and keeps track of the current position in the traversal of the Aggregate. If we map it to a real-life example, we see that it maps to our PeopleIterator.

```
Aggregate              Iterator
|                      |
PeopleCollection <----- ConcreteIterator
```

And lastly, the ConcreteAggregate implements the Iterator creation interface to return an instance of the proper ConcreteIterator, and that maps to our PeopleCollection.

```
Aggregate              Iterator
|                      |
ConcreteAggregate <----- ConcreteIterator
```

Demo - Implementing the Iterator pattern

In this demo we will implement our custom iterator over the PeopleCollection, which will iterate over it in alphabetical order. Let's start with a helper class, a Person. Person has a name and a country where the person is coming from.

In Implementation.cs

```
public class Person
{
    public string Name { get; set; }
    public string Country { get; set; }

    public Person(string name, string country)
    {
        Name = name;
        Country = country;
    }
}
```

Up next, we define our Iterator interface. It is named IPeopleIterator, and it is a copy from the pattern structure mapped to our example. So what it returns from its methods where applicable are Person objects as that is what we are going to iterate over.

Below Person{}:

```
public interface IPeopleIterator
{
    Person First();

    Person Next();

    bool IsDone { get; }

    Person CurrentItem { get; }
}
```

The next interface we want to define is our IPeopleCollection. That is the aggregate. As we learned, it should define one method, CreateIterator returning an IPeopleIterator.

Below IPeopleIterator{}:

```
public interface IPeopleCollection
{
    IPeopleIterator CreateIterator();
}
```

The Concrete aggregate is then our PeopleCollection. Now we could make this hard on ourselves. We could start from scratch with implementing a complete new collection with an internal data structure to hold our set of people with add and remove methods, indexers, and so on. However, we already have all that functionality in the built-in framework classes.

So, instead of starting from scratch, we are going to subclass the generic List class, we will make this derive from List of Person. The one thing we do need to add is an implementation for the CreateIterator method from our IPeopleCollection interface.

The signature of the method signifies that it returns an `IPeopleIterator`. The method itself should return a concrete implementation of that iterator. We haven't got that yet, but we know from the slides that we named it `PeopleIterator`. We also pass through the object itself as the `PeopleIterator` will require access to it to iterate over the items.

Below `IPeopleCollection{}`:

```
public class PeopleCollection : List<Person>, IPeopleCollection
{
    public IPeopleIterator CreateIterator()
    {
        return new PeopleIterator(this);
    }
}
```

The concrete iterator, the `PeopleIterator`. This is where most of the logic resides. Iterators can become quite complex, storing numerous in between positions and variables to return the correct item. Let's start by implementing the `IPeopleIterator` interface. In our case, we want to alphabetically order the list. We can again use built-in framework functionality for that.

The first thing we will do is create a constructor that accepts our `PeopleCollection`. Like that the code from our `PeopleCollection` that creates the iterator will resolve again. In our constructor, we accept that collection, and we store it in a private field. The other field we add is the current field, which will store the current position in our `PeopleCollection`. The idea is that now we are going to implement `IsDone`, `CurrentItem`, `First`, and `Next` in a way that returns our people in alphabetical order.

Let's start with `First`. When we want to get the first element in our collection, we can say that we want the current position to be set at 0, and then we return the person at the current position after having ordered the `peopleCollection` by the person's name.

When we want to get the next person in line, we add 1 to the current position. As long as we are not done, that means as long as there are still items in the list, we will implement `IsDone` immediately. As long as that is not the case, we can select the next person in line. We again order the `PeopleCollection` by name, and we get to one at the current position. If we are done, in other words if there are no more people in the list, we can potentially return null. What you want to do when someone tries to get the next item in the list and there is no next item kind of depends on your use case. Returning null is pretty common here.

Two things left, implementing `IsDone` and `CurrentItem`. These were defined as properties. Getting `IsDone` means checking whether the current position is larger than or equal to the amount of items in the `PeopleCollection`. So that statement will either return true or false. Getting the `Current Item` uses the indexer, passing through current as position after having ordered our `PeopleCollection` by name.

Below `PeopleCollection{}`

```
public class PeopleIterator : IPeopleIterator
{
    private PeopleCollection _peopleCollection;
    private int _current = 0;

    public bool IsDone
    {
        get { return _current >= _peopleCollection.Count; }
```

```

    }

    public Person CurrentItem
    {
        get
        {
            return _peopleCollection
                .OrderBy(p => p.Name).ToList()[_current];
        }
    }

    public Person First()
    {
        _current = 0;
        return _peopleCollection
            .OrderBy(p => p.Name).ToList()[_current];
    }

    public Person Next()
    {
        _current++;
        if (!IsDone)
        {
            return _peopleCollection
                .OrderBy(p => p.Name).ToList()[_current];
        }
        else
        {
            return null;
        }
    }
}

```

Let's write a little bit of code to test this. First, we create a new instance of `PeopleCollection`. For that, we need to import the iterator namespace. Now let's add a few people to it. These are all fellow authors living in Belgium or the surrounding countries. The next thing to do is create the iterator we will use to iterate over our collection. For that, we call in to `CreateIterator` on our `PeopleCollection`.

And now we use that iterator to run through all items in the collection. For each person, we write out a name. Like that, we will easily be able to see whether they are truly sorted alphabetically. As you notice, in our for loop, we also use the `First`, `IsDone`, and `Next` operations on our `PeopleIterator`. Add a `console.ReadKey` statement at the bottom.

In `Program.cs`

```
PeopleCollection people = new();
```

```

people.Add(new Person("Kevin Dockx", "Belgium"));
people.Add(new Person("Gill Cleeren", "Belgium"));
people.Add(new Person("Roland Guijt", "Netherlands"));
people.Add(new Person("Thomas Claudius Huber", "Germany"));

```

```
// create the iterator
```

```
var peopleIterator = people.CreateIterator();
```



```
//use the iterator to run through the people in the collection;
//they should come out in alphabetical order
for(Person person = peopleIterator.First();
    !peopleIterator.IsDone;
    person = peopleIterator.Next())
{
    Console.WriteLine(person?.Name);
}

Console.ReadKey();
```

And as you notice, the authors are now sorted alphabetically.

Use Cases for the Iterator pattern

Use this pattern to access an aggregate object's contents without exposing its internal representation. As mentioned before, this avoids unintended access to and manipulation of that internal representation. Next to that, it can also be used to hide away complexity when you are working with a complicated data structure. Think about complicated tree structures, for example.

Another good use case is when you want to support multiple ways of traversal for the same aggregate object.

And yet another one is avoiding code duplication. If the way you want to traverse a structure is complicated, chances are it is not trivial to write. If in such a case you want to traverse your structure in multiple places in your app, it is a good idea to put that code in an iterator to avoid code duplication.

Pattern Consequences

An important consequence is that iterators simplify the interface of your aggregate structure. The traversal code is separated out in separate classes. This follows the single responsibility principle.

You can implement new types of aggregate objects and iterators without them interfering with each other. Seeing we are working on interfaces and not implementations, the open/closed principle applies as well.

Iterators can exist next to each other at the same time on the same collection. In other words, you can be traversing a collection with an alphabetical iterator at the same time as you are traversing it in the order items were added. As the iterators store their own position, they won't interfere with each other.

A disadvantage is that using this pattern can be a bit of overkill when you only use simple traversals and collections. It is mostly useful for more complex situations.

Related patterns

The iterator is related to the composite pattern. This is a recursive structure, and iterators are often used to traverse those.

It can also be used in conjunction with a memento. In that case, the memento can be used to store the state of the iterator and potentially roll it back.

Lastly, the visitor pattern can be used in combination with the iterator. You could use an iterator to

traverse a potentially complex data structure and use the visitor to apply some logic to the items in that structure.

- Composite
Iterators are often used to traverse its recursive structure.
- Memento
The memento can be used to store the state of the iterator and, potentially, roll it back.
- Visitor
You can use an iterator to traverse a potentially complex data structure, and apply logic to the items in that structure with a visitor.

Behavioral Pattern: Visitor

Monday, October 31, 2022 1:46 PM

Coming up

In this module we will look into the Visitor pattern. A visitor represents an operation that has to be performed on the elements of an object structure.

We will start by describing the visitor pattern. For that, we will use a real-life example, calculating discounts for employees and customers. After that, we will map that to a structural example and implement it in a demo. We will also implement a variation of the pattern. We will continue with learning about use cases for this pattern, and the pattern consequences. We will end the module with related patterns.

Describing the Visitor Pattern

The Visitor Pattern is a behavioral pattern from the Gang of Four library of design patterns. Its intent is to represent an operation to be performed on the elements of an object structure. Visitor lets you define new operation without changing the classes of the elements on which it operates. That definition requires some further explanation.

Imagine the following use case. We are building part of an order management system. You have got a large set of objects, but they are not all the same. You have got an InternalCustomer class, a GovernmentCustomer class, and a PrivateCustomer class and so on, all deriving from Customer, but all with their own custom behavior and properties. At a certain moment, a requirement comes in to add new behavior to all these objects. You need to calculate a discount depending on the amount of money the customer previously spent on orders.

```
public class Customer {}
public class InternalCustomer : Customer {}
public class GovernmentCustomer : Customer {}
public class PrivateCustomer : Customer {}
```

To add this behavior, you can implement it in all these classes, or depending on what needs to be known about the customer, you can implement it in a base class.

```
public class Customer
{
    public decimal CalculateDiscount()
    {
        //Do calculations
    }
}
public class InternalCustomer : Customer {}
public class GovernmentCustomer : Customer {}
public class PrivateCustomer : Customer {}
```

But there is other types as well. Let's add an Employee class to the mix. Employees of companies also get discounts, but the calculations for these are completely different. They are based on the amount of years someone is employed. Employee and the customer don't have the same base class, so we can't use that approach anymore.

```

public class Customer
{
    public decimal CalculateDiscount()
    {
        //Do calculations
    }
}
public class InternalCustomer : Customer {}
public class GovernmentCustomer : Customer {}
public class PrivateCustomer : Customer {}
public class Employee
{
    public decimal CalculateDiscount()
    {
        //Do calculations
    }
}

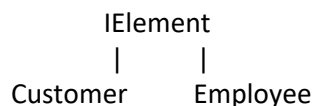
```

We can however still add the method to calculate the discount on the concrete Employee class. Yet, what if after some time a new requirement comes in? Like you now need to calculate some shipping costs for each customer and for each employee. And after that, you need to add JSON export to all objects, and so on. This becomes cumbersome. You always need to change these classes, which is risky. Also, we need to start wondering whether it makes sense to have all that added behavior in all of these classes. Maybe it doesn't belong there if we keep the single responsibility principle in mind.

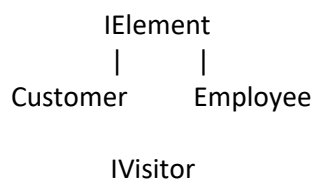
Enter the visitor pattern. This allows adding behavior to classes by adding the behavior to another class, the visitor, which will visit the original classes and execute that behavior. To avoid overloading the screen with a huge diagram, let's just stick to a customer class and an employee class, unrelated to each other, and with their own properties and operations. But there can of course be many more classes. They have a discount field, which we will calculate using our visitor.

Customer Employee

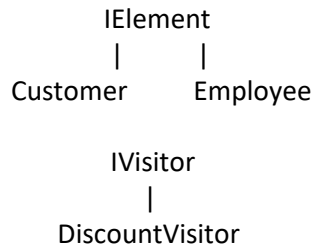
We have both of them implement an interface, the IElement interface, which defines one method, accept. This method accepts a visitor, or rather an IVisitor, as we want to work on an interface here.



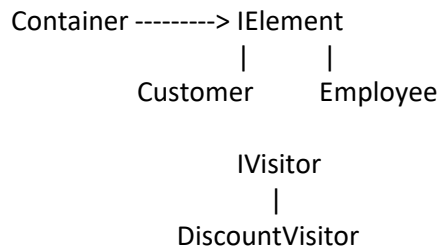
On that interface, we define a method to visit the customer, which accepts our customer, and another method to visit the employee, which accepts our employee. In other words, one method per class so the discount can be calculated correctly, as the algorithm may be different for each class.



A concrete class, DiscountVisitor, implements the IVisitor interface and contains the actual logic to calculate the discounts.

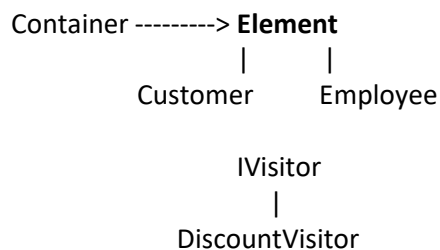


The last thing we need is some sort of object structure that accepts our DiscountVisitor and is responsible for running through the customers and employees and passing the visitor through to the Accept methods on them. This can be a list or a composite, depending on your use case. For our case we will name it Container.

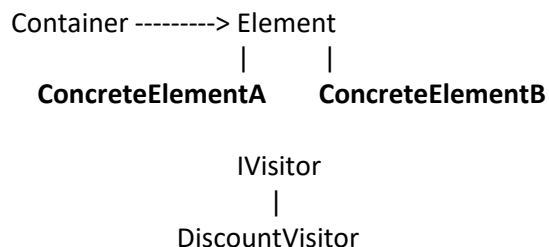


Structure of the Visitor Pattern

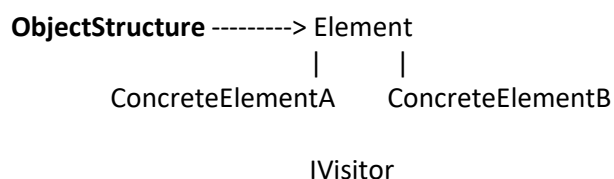
We will start with the Element, which maps to our IElement interface. This defines an accept operation that takes a Visitor as an argument.



Then we got ConcreteElement implementations. These are our Customer and Employee classes. A ConcreteElement implements the accept operation that takes a Visitor as an argument.

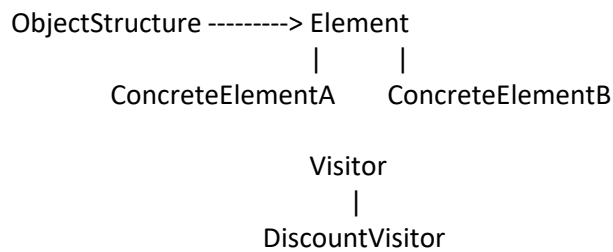


The objectStructure then can enumerate its elements. It typically provides an interface to allow a visitor to visit its elements, and it can be a composite or collection. That maps to our Container

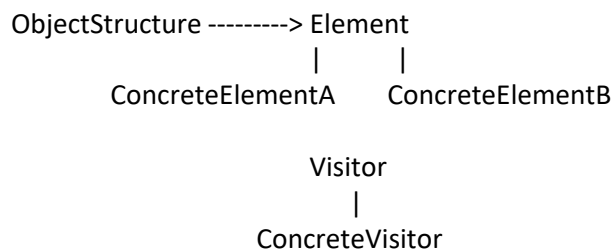




Up next is the Visitor, which maps to IVisitor. This declares a Visit operation for each class of ConcreteElement in the ObjectStructure.



And then we have got ConcreteVisitor, which maps to our DiscountVisitor, as we learned in the previous clip, but others are possible as well. These ConcreteVisitors implement each operation declared by the visitor, so they contain the actual algorithm. Often, they also contain state that accumulates while traversing all the objects, which can be useful for callers like storing the total amount of discount given.



Demo - Implementing the Visitor pattern

In this demo, we will implement the visitor pattern.

Let's start with our concrete elements, the customer and the employee classes. We will give our customer and employee a name. The way discounts are calculated for these are different. For the customer, it is 10% of the total amount ordered, so we add a property for that. We also add a discount property, which will be set by the visitor. Therefore the setter can't be private. We want to ensure that a customer is constructed with the amount ordered and name passed through, so we add a constructor to that avail.

In implementations.cs

```

public class Customer
{
    public decimal AmountOrdered { get; private set; }
    public decimal Discount { get; set; }
    public string Name { get; private set; }

    public Customer(string name, decimal amountOrdered)
    {
        Name = name;
        AmountOrdered = amountOrdered;
    }
}
  
```

Likewise, we create an Employee class. For an employee class the discount will be calculated depending on the amount of years the employee has been employed.

```

Below customer{}
public class Employee
{
    public int YearsEmployed { get; private set; }
    public decimal Discount { get; set; }
    public string Name {get; private set; }

    public Employee(string name, int yearsEmployed)
    {
        Name = name;
        YearsEmployed = yearsEmployed;
    }
}

```

Up next are two interfaces, the IVisitor interface and the IElement interface. Note that these can be implemented as abstract base classes as well if you wish to provide some out of the box functionality. For our use case, that is not applicable.

Let's start with the IVisitor interface. As we learned on the slides, it needs methods to visit our concrete elements, VisitCustomer and VisitEmployee.

```

Below Employee{}
public interface IVisitor
{
    void VisitCustomer(Customer customer);
    void VisitEmployee(Employee employee);
}

```

The IElement interface then contains one method, Accept. That method must accept an IVisitor so it can visit the concrete elements and execute its algorithm on them. Our concrete elements must thus implement this IElement interface,

```

Below IVisitor{}:
public interface IElement
{
    void Accept(IVisitor visitor);
}

```

Let's scroll back to our Customer class. We have it implement the IElement interface leading to an Accept method on this customer class. The IVisitor gets passed through via the Accept method. So, what we can do now on this IVisitor interface is call VisitCustomer because that was one of the methods that was created on it. We pass through the customer object itself so the visitor has access to the customer properties it needs to do its calculation. Also for demo purposes, we write out some info to the console output window.

Change to Customer class:

```

public class Customer : IElement
{
    public decimal AmountOrdered { get; private set; }
    public decimal Discount { get; set; }
    public string Name {get; private set; }

    public Customer(string name, decimal amountOrdered)

```

```

    {
        Name = name;
        AmountOrdered = amountOrdered;
    }

    public void Accept(IVisitor visitor)
    {
        visitor.VisitCustomer(this);
        Console.WriteLine($"Visited {nameof(Customer)} {Name}, " +
            $"discount given: {Discount}");
    }
}

```

And we do something likewise for our Employee class. We implement the IElement interface. That leads us to implementing the Accept method. IVisitor gets passed through. So on IVisitor, we can call VisitEmployee, passing through this, the CurrentEmployee object. Here we write out some info to the console output window.

Change to Employee class:

```

public class Employee : IElement
{
    public int YearsEmployed { get; private set; }
    public decimal Discount { get; set; }
    public string Name {get; private set; }

    public Employee(string name, int yearsEmployed)
    {
        Name = name;
        YearsEmployed = yearsEmployed;
    }

    public void Accept(IVisitor visitor)
    {
        visitor.VisitEmployee(this);
        Console.WriteLine($"Visited {nameof(Employee)} {Name}, " +
            $"discount given: {Discount}");
    }
}

```

We haven't got an actual visitor yet though, so let's create that next. We will name it DiscountVisitor. It must implement the IVisitor interface. That means we need to implement two methods, VisitCustomer and VisitEmployee. In the VisitCustomer method, we can calculate the discount for a customer, 10% of what the customer ordered. The visitor access the customer properties it needs, and it also changes the customer object. It sets the discount. So, visitors can change values on the object they visit, but they also often store information about their operation results that can be useful for callers. So let's add a property to our visitor, TotalDiscountGiven. It will contain a summation of the discounts provided across all elements. So, that means that when we visit a customer and we calculate the discount, we add that to our TotalDiscountGiven property. For employees, the calculation is a bit different. When someone has been employed for less than 1+ years, a €100 discount is given. Otherwise, a €200 discount is given. Here we also add the discount that is given to the total amount.

Below Employee{}:


```

public class DiscountVisitor : IVisitor
{
    public decimal TotalDiscountGiven { get; set; }

    public void VisitCustomer(Customer customer)
    {
        //Percentage of total amount
        var discount = customer.AmountOrdered / 10;
        //Set it on the customer
        customer.Discount = discount;
        //Add it to the total amount
        TotalDiscountGiven += discount;
    }

    public void VisitEmployee(Employee employee)
    {
        //Fixed value depending on the amount of years employed
        var discount = employee.YearsEmployed < 10 ? 100 : 200;
        //set it on the employee
        employee.Discount = discount;
        //Add it to the total amount
        TotalDiscountGiven += discount;
    }
}

```

Lastly, we need a structure to hold employees and customers. That is the object structure from the pattern structure, we name it Container, and it is a pretty straight forward class. It stores lists of employees and customers, and it allows passing through a visitor to each of them. This is thus a composite. With that, our pattern implementation is done.

Below DiscountVisitor{}:

```

public class Container
{
    public List<Employee> Employees { get; set; } = new();
    public List<Customer> Customers { get; set; } = new();

    public void Accept(IVisitor visitor)
    {
        foreach (var employee in Employees)
        {
            employee.Accept(visitor);
        }

        foreach (var customer in Customers)
        {
            customer.Accept(visitor);
        }
    }
}

```

Let's test this. We create a new instance of a container. For that we need to import the Visitor namespace, and then we add a few customers and employees to it. For these the discounts will be calculated. We create a new instance of a DiscountVisitor, pass it through the container, and write out

the gathered amount of discount given in total. Once the DiscountVisitor is passed through, it will visit all our customers and employees, which means that once we reach this last WriteLine statement, TotalDiscountGiven should effectively contain the total discount given. Let's add a ReadKey statement, and try this.

In Program.cs

```
//Create container and add concrete elements.
```

```
var container = new Container();
```

```
container.Customers.Add(new Customer("Sophie", 500));
```

```
container.Customers.Add(new Customer("Karen", 1000));
```

```
container.Customers.Add(new Customer("Sven", 800));
```

```
container.Employees.Add(new Employee("Kevin", 18));
```

```
container.Employees.Add(new Employee("Tom", 5));
```

```
//Create visitor
```

```
DiscountVisitor discountVisitor = new();
```

```
//pass it through
```

```
container.Accept(discountVisitor);
```

```
//Write out gathered amount
```

```
Console.WriteLine($"Total discount: {discountVisitor.TotalDiscountGiven}");
```

```
Console.ReadKey();
```

There we go. As we noticed, all employees and customers were visited. Discounts were calculated by the discountVisitor, and all these discounts added together results in a total discount given of 530.

Demo - Simplifying the Visitor interface

We are looking at our IVisitor interface. One of the issues with this approach is that this interface needs to be changed whenever a new concrete element is added because it contains methods that work on these concrete elements like Customer and Employee. To avoid that, this interface is often simplified. Instead of having it contain methods for each concrete type, we have it contain one method, Visit, that accepts an IElement instead of a concrete implementation. All our concrete implementations have to implement IElement anyway, so this should work, but it also has consequences.

In implementation.cs

```
/* public interface IVisitor {} */
```

```
Below /* ... */:
```

```
public interface IVisitor
```

```
{
```

```
    void Visit(IElement element);
```

```
}
```

First of all, we need to make a change to our Customer and Employee classes. In their Accept methods, it is now the Visit method that needs to be called.

Change Customer{}:

```
public class Customer : IElement
```

```
{
```

```

    public decimal AmountOrdered { get; private set; }
    public decimal Discount { get; set; }
    public string Name {get; private set; }

    public Customer(string name, decimal amountOrdered)
    {
        Name = name;
        AmountOrdered = amountOrdered;
    }

    public void Accept(Visitor visitor)
    {
        //visitor.VisitCustomer(this);
        visitor.Visit(this);
        Console.WriteLine($"Visited {nameof(Customer)} {Name}, " +
            $"discount given: {Discount}");
    }
}

```

Change to Employee{}:

```

public class Employee : IElement
{
    public int YearsEmployed { get; private set; }
    public decimal Discount { get; set; }
    public string Name {get; private set; }

    public Employee(string name, int yearsEmployed)
    {
        Name = name;
        YearsEmployed = yearsEmployed;
    }

    public void Accept(Visitor visitor)
    {
        //visitor.VisitEmployee(this);
        visitor.Visit(this);
        Console.WriteLine($"Visited {nameof(Employee)} {Name}, " +
            $"discount given: {Discount}");
    }
}

```

We will also have to change the DiscountVisitor itself. If we implement IVisitor now we see we only have one method to implement, Visit. So that means it is up to the discount visitor to check the actual concrete type of the element, so it can execute the correct logic. So we adjust our old methods by making them private. And in our Visit method, we check for the actual type, and depending on that, execute the correct algorithm.

Change to DiscountVisitor:

```

public class DiscountVisitor : IVisitor
{
    public decimal TotalDiscountGiven { get; set; }
}

```

```

public void Visit(IElement element)
{
    if (element is Customer)
    {
        VisitCustomer((Customer)element);
    }

    else if (element is Employee)
    {
        VisitEmployee((Employee)element);
    }
}

private void VisitCustomer(Customer customer)
{
    //Percentage of total amount
    var discount = customer.AmountOrdered / 10;
    //Set it on the customer
    customer.Discount = discount;
    //Add it to the total amount
    TotalDiscountGiven += discount;
}

private void VisitEmployee(Employee employee)
{
    //Fixed value depending on the amount of years employed
    var discount = employee.YearsEmployed < 10 ? 100 : 200;
    //set it on the employee
    employee.Discount = discount;
    //Add it to the total amount
    TotalDiscountGiven += discount;
}
}

```

No need to change anything to the client. That should still work as before, so let's give this a try. There we go still works as expected. We have made our interface simpler, so we won't have to change that whenever the new concrete element is added to our ObjectStructure. Yet, we have introduced additional logic in the visitor implementation. It's this implementation that is seen more often in the wild, but both implementations we made are valid. The intent is the same. It is just a change in the template.

Use Cases for the Visitor pattern

A good use case for this pattern is when an object's structure contains many classes with objects with differing interfaces and you want to perform operations on them that depend on their concrete classes. In our example, these concrete classes were customers and employee.

Another good use case is when you are in a situation where the classes defined in your object structure don't have to change often, but you do often want to define new operations over the structure, a discount visitor, a JSON export visitor, and so on. Using this pattern avoids having to change the classes, employee and customer, that define your object structure.

Lastly, use it when you have got potentially many operations that need to be performed on objects in

your object structure, but not necessarily on all of them. The visitor pattern avoids that you have to pollute all your objects with these operations, even when they don't apply to them. Of course, a bit of reflection can help here as well, but that is typically a costly operation you want to avoid.

Pattern Consequences

The first consequence is that the visitor pattern makes adding new operations easy. You can define a new operation simply by creating a new visitor, and there is no need to change the classes the visitor visits. This is the open/closed principle at work.

A definite advantage is that you can use the visitor to accumulate into on the objects it visits. We did that in our example by accumulating the total discount.

What is also an advantage is that the visitor gathers related operations together and potentially separates unrelated ones. For example, all operations related to calculating discounts are grouped together in the same visitor class, all operations for calculating shipping costs, as well, and so on. Otherwise, they might be separated out into the classes that are visited, where they would represent a group of unrelated operations. This advantage works towards the single responsibility principle.

There is also a few disadvantages. Adding a new concrete element can be hard, as you will potentially have to change all visitor interfaces and concrete implementations.

And lastly, the visitor pattern might require you to break encapsulation. The visitor needs access to all properties or operations on each concrete element to do its job, which might require you to make more operations accessible on these classes than would otherwise be required.

Related patterns

The visitor pattern is related to the composite pattern. We know that, as the object structure we used in our example was a composite. So the visitor pattern can be used to apply an operation over an object structure defined by the composite pattern.

It can also be used together with an iterator. You could use an iterator to traverse a potentially complex data structure and then use the visitor to apply some logic to the items in that structure.

- Composite

A visitor can be used to apply an operation over an object structure defined by the composite pattern

- Iterator

You can use an iterator to traverse a potentially complex data structure, and apply logic to the items in that structure with a visitor.

Behavioral Pattern: Interpreter

Monday, October 31, 2022 1:47 PM

Coming up

In this module, we will dive into the interpreter pattern. It is not one you'd often use, yet it is pretty cool. It is used to define grammar and interpret languages.

We will start by describing the pattern. As an example, we will write a piece of code to translate Arabic numerals to Roman numerals. We will map that to a structural example and implement it in a demo. We will continue learning about the use cases for this pattern and the pattern consequences. We will end the module with related patterns.

Describing the Interpreter Pattern

The interpreter pattern is a behavioral pattern from the Gang of Four library of design patterns. Its intent is to, given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentence in the language. In other words it defines a grammatical representation for a language and an interpreter to interpret the grammar. And that is also why it is not something you will often create yourself. You don't define a language every day, yet it is something we use a lot in our day-to-day programming life.

When you use LINQ queries, behind the scenes an interpreter-like pattern can handle translation between LINQ and, for example, SQL, or the interpreter for regular expressions or even the way the C# compiler interprets the code we write. All of these could use a variation of this interpreter pattern. There are other patterns and possibilities. We will cover that once we get to the use cases, but know that it would be possible to use the interpreter pattern for all these examples.

Imagine the use case in which we want to translate Arabic numerals to Roman numerals. Say that we have the number 181. How would that go around? If we stick to the hundreds as a maximum, there is 999 possible outcomes, or 1000 if we include 0. Imagine the switch statement we'd have to write. That is completely unmanageable. Enter the interpreter pattern.

We can look at 181 as a sentence in a grammar we are going to define. That grammar exists of a set of expressions. The first expression would be the 100 expression, the second one the then expression, the third one the one expression. Each expression is then responsible for translating the word, which is one number in our case, that is passed in. In other words, it is responsible for interpreting the word. So we define a grammar through a set of expressions. You can call that a syntax tree, which is a word. That means we don't need that huge switch statement anymore.

What we do need is a way to define these expressions. Let's start with a RomanExpression of strike class or interface that declares an interpret operation.

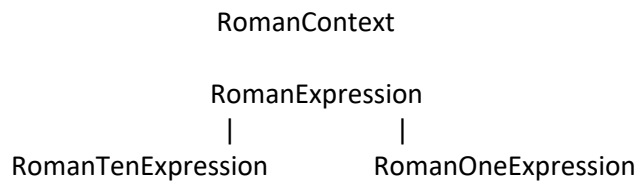
RomanExpression

This operation gets some information passed through, an object that contains info on what we want to translate and typically a way to store the result. Say we name that RomanContext.

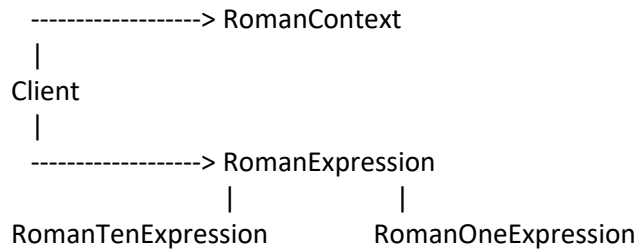
RomanContext

RomanExpression

And now we have three implementations of that expression, RomanOneHundrerExpression, which interprets the hundreds, RomanTenExpression, and RomanOneExpression . Let's stick to two for the example, on our diagram to keep it more readable.

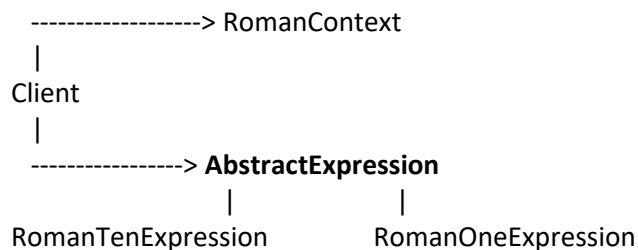


It is up to the client then to define the grammar, the syntax tree, and provide RomanContext.



Structure of the Interpreter Pattern

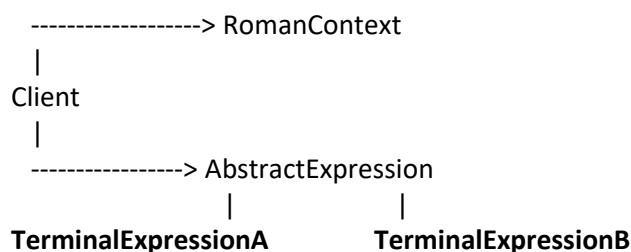
We will start with the first player, the AbstractExpression. This maps to our RomanExpression abstract base class. What it does is declare an abstract interpret operation that is common to all nodes in the abstract syntax tree.



Up next is the TerminalExpression. That is an implementation of the Abstract Expression. This implements an Interpret operation associated with terminal symbols in the grammar.

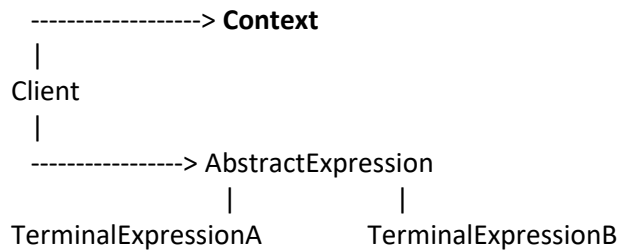
An instance is required for every terminal symbol in a sentence. This maps maps to our RomanTen and RomanOne Expressions. But what is a terminal symbol?

And if there is something called a terminal expression, there is probably also something called a nonterminal expression, right? That is indeed the case, but we will put it aside for now, to not overcomplicate things. We will get back to terminal and non-terminal expressions after the demo.

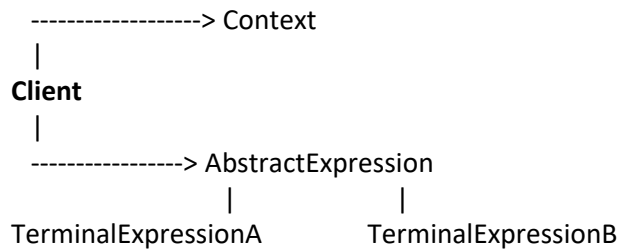


The next player is the context which maps to our Roman context. It contains information that is global to

the interpreter. Input and output properties are quite common on this.



Lastly, we have the client. This builds or is given the abstract syntax tree, which represents a sentence in the language that the grammar defines. That tree is assembled from expressions. The client is also responsible for invoking the interpret operation.



Demo - Implementing the Interpreter pattern

In this demo, we will create a translator which will translate Arabic numerals to Roman numerals. Let's start with the basics. We add a Context class, RomanContext, containing an Input and Output property. This is what will get passed through to our expressions.

```
In Implementation.cs
public class RomanContext
{
    public int Input { get; set; }
    public string Output { get; set; } = string.Empty;

    public RomanContext(int input)
    {
        Input = input;
    }
}
```

For that we create an AbstractExpression base class. In our case, we name that RomanExpression. Here we use abstract class, you could also use an interface for this, as long as you don't have the need to provide default implementation logic. The RomanExpression contains one abstract method, Interpret, which accepts the context.

```
Below RomanContext{}:
public abstract class RomanExpression
{
    public abstract void Interpret(RomanContext value);
}
```

Then it is onto the actual logic, the implementations of our abstract RomanExpression class. In this example these are the terminal expression classes from the pattern structure. Let's start simple, with a 1

expression, so numbers 1 to 9. But how do we translate that to Roman numerals?

Well, numbers 1 to 9 are represented as such in Roman numerals, so we could write a switch statement for all 9 possibilities. But if you think about it, we can simplify this. Take 3 for example, translating that to Roman literals is 3 times capital I. That is the same as translating 1 to Roman literals 3 times. And 2, that is 2 times 1.

So instead of having a switch statement for 3, we could say that we write a statement for dealing with 1 and subtract 1 each time. That means if we have our number 3 we will hit our logic to write 1 as capital I three times, resulting in III, which is what we need. With that in mind, we can simplify our logic to just 4 cases, 9, 5, 4, and 1.

All the rest of these are combinations we can reach with simple subtraction, putting code that translates to what you see on screen now. That takes care of our Roman 1 expression.

Below RomanExpression{}:

```
public class RomanOneExpression : RomanExpression
{
    //1 = I, 2 = II, 3 = III, 4 = IV, 5 = V, 6 = VI, 7 = VII, 8 = VIII, 9 = IX.

    public override void Interpret(RomanContext value)
    {
        while ((value.Input - 9) >= 0)
        {
            value.Output += "IX";
            value.Input -= 9;
        }

        while ((value.Input - 5) >= 0)
        {
            value.Output += "V";
            value.Input -= 5;
        }

        while ((value.Input - 4) >= 0)
        {
            value.Output += "IV";
            value.Input -= 4;
        }

        while ((value.Input - 1) >= 0)
        {
            value.Output += "I";
            value.Input -= 1;
        }
    }
}
```

The next one in line is the Roman 10 expression, and if we think about it this follows likewise principles as the Roman 1 expression. These nine different numbers don't necessarily result in a switch statement with nine potential cases. They can be simplified to four. So that results in a Roman 10 expression deriving from RomanExpression, in which the Interpret method takes care of these 4 potential cases.

Below RomanOneExpression{}:

```
public class RomanTenExpression : RomanExpression
{
    //10 = X, 20 = XX, 30 = XXX, 40 = XL, 50 = 50 L, 60 = LX, 70 = LII, 80 = LIII, 90 = XC.

    public override void Interpret(RomanContext value)
    {
        while ((value.Input - 90) >= 0)
        {
            value.Output += "XC";
            value.Input -= 90;
        }

        while ((value.Input - 50) >= 0)
        {
            value.Output += "L";
            value.Input -= 50;
        }

        while ((value.Input - 40) >= 0)
        {
            value.Output += "XL";
            value.Input -= 40;
        }

        while ((value.Input - 10) >= 0)
        {
            value.Output += "X";
            value.Input -= 10;
        }
    }
}
```

That leaves us with the Roman 100 expression, we can implement likewise logic for this. And that takes care of the Roman 100 expression.

Below RomanTenExpression{}:

```
public class RomanHundredExpression : RomanExpression
{
    //100 = C, 200 = CC, 300 = CCC, 400 = CD, 500 = D, 600 = DC, 700 = DCC, 800 = DCCC,
    //900 = CM.

    public override void Interpret(RomanContext value)
    {
        while ((value.Input - 900) >= 0)
        {
            value.Output += "CM";
            value.Input -= 900;
        }

        while ((value.Input - 500) >= 0)
```

```

        {
            value.Output += "D";
            value.Input -= 500;
        }

        while ((value.Input - 400) >= 0)
        {
            value.Output += "CD";
            value.Input -= 400;
        }

        while ((value.Input - 100) >= 0)
        {
            value.Output += "C";
            value.Input -= 100;
        }
    }
}

```

Now onto the client to write codes for testing this. First we create a syntax tree. We do need to import the Interpreter namespace for that. We create a new list of Roman expressions, stating we are going to start with the hundred expression, then the ten expression, and then the one expression that is the order in which our sentence will be translated, word by word.

Then we create a context. When creating that context, we pass through the input, 5, in our case. We run through our tree of expressions, and we interpret the input.

After that we write out the result. In this case, RomanHundredExpression won't result in anything, and neither will the RomanTenExpression, but RomanOneExpression will result in 5 being translated into 5 (V) in Roman numerals.

Now let's do the same for a few other Arabic numbers. So we try and translate 81 and 733. Then we add a ReadKey statement at the moment and run the code.

In Program.cs

```

var expressions = new List<RomanExpression>
{
    new RomanHundredExpression(),
    new RomanTenExpression(),
    new RomanOneExpression(),
};

var context = new RomanContext(5);
foreach (var expression in expressions)
{
    expression.Interpret(context);
}

Console.WriteLine($"Translating Arabic numerals to Roman numerals: 5 = {context.Output}");

context = new RomanContext(81);
foreach (var expression in expressions)
{

```

```

        expression.Interpret(context);
    }
    Console.WriteLine($"Translating Arabic numerals to Roman numerals: 81 = {context.Output}");

    context = new RomanContext(733);
    foreach (var expression in expressions)
    {
        expression.Interpret(context);
    }
    Console.WriteLine($"Translating Arabic numerals to Roman numerals: 733 = {context.Output}");

    Console.ReadKey();

```

There we go, our numbers has been translated correctly.

Terminal vs. NonTerminal Expression

When we looked into the pattern structure, we learned that one possible implementation of an abstract expression is a terminal expression.

There is another possible implementation, a non-terminal expression. It is not applicable to our example, but it is nevertheless interesting to know about.

All language consist of symbols. A symbol is the smallest meaningful part or unit of language, for example, a word, a number, an operator, things like that. These symbols are called terminals implemented through a terminal expression. When you think about it, you notice that all these terminals make up statements.

For example, $5 + 10 = 15$. That is a statement made up of a set of terminals. In a programming language, you could have a while or a conditional statement made up of terminals. That is the statements that are made up of terminals and that are allowed by a language, those are called nonterminals. Nonterminals can consist of terminals and/or additional nonterminals.

So that is what these two mean and how they differ.

Terminal expression:

- A symbol (the smallest meaningful part or unit of a language) is called a Terminal.
- Implemented through a Terminal expression.

NonTerminal expression:

- Statements that are made up of terminals and are allowed by a language are called NonTerminals.
- Can consist of Terminal(s) and/or NonTerminal(s).

So that is what these two mean and how they differ.

A non-terminal expression is symbolized by a non-terminal expression in the pattern structure. It maintains a list of abstract expressions that make up the non-terminal expression. Interpreting a non-terminal expression is typically done via recursion on all the variables that represented.



|
NonTerminalExpression <-----|

Use Cases for the Interpreter pattern

The use case for this pattern is when there is a language you can interpret, and you can represent statements in the language as an abstract syntax tree. But that use case would apply to many types of languages, even though this pattern isn't necessarily a good fit for all of them.

Keep in mind that this is a good fit when the grammar is simple. When it becomes complex, the class hierarchy of the grammar becomes large and unmanageable.

Also, it is not the most efficient interpreter. The most efficient ones are usually implemented by translating parsed trees to another form first. Regular expressions, for example, are often transformed into state machines first. Yet even then, the translator itself can be implemented by the interpreter pattern. So, it is still applicable.

Pattern Consequences

The first consequence is an advantage it is easy to change and extend the grammar. Grammar rules are represented by classes and these can be extended via inheritance.

Also implementing the grammar is easy too, as all classes that implement the abstract expression have similar implementations.

The disadvantage is that complex grammars are hard to maintain. There is at least one class for every rule in the grammar. So, if you got a large set of rules, you have got a large set of classes to manage and maintain.

Related patterns

The interpreter is related to two other patterns. The first one is the composite as the abstract syntax tree is, in fact, an instance of the composite pattern.

The second related pattern is the iterator because, in regards to traversing the structure, an iterator could be used.

- Composite
The abstract syntax tree is an instance of the composite pattern.
- Iterator
You can use an iterator to traverse the tree.